

Nand2Tetris

Ela Fallik

להערות ותיקונים:

ela.fallik@mail.huji.ac.il

Based on the course's book - version 2005

תוכן עניינים

3	Hardware Platform	I
4	Boolean Logic (Basic Boolean Gates)	1
4	Unary op	1.1
4	Binary op	1.2
5	Mux & DMux	1.3
5	Multi-Bit	1.4
6	Multi-Way	1.5
8	Boolean Arithmetic (Arithmetic Logical Unit - ALU)	2
8	Background	2.1
10	Add	2.2
12	ALU	2.3
13	Additional Functionality - Shift	2.4
14	Additional Functionality - Multiplier	2.5
15	Sequential Logic (DFF, RAM)	3
15	DFF - Data Flip-Flops	3.1
15	Registers	3.2
17	Memory	3.3
18	PC - Program Counter	3.4
19	Machine Language (HACK Lang. & Assembly specification)	4
19	Background	4.1
20	HACK Machine Language	4.2
20	Machine Language & Assembly Syntax	4.3
25	Project	4.4
26	Computer Architecture (CPU, Computer)	5
26	Background	5.1
26	CPU	5.2
29	Instruction Memory - ROM	5.3
30	Data Memory	5.4
32	Computer	5.5
33	Additional Functionality - Extended ALU & CPU	5.6
34	Software Hierarchy	II
35	Assembler	6
35	Background	6.1
35	Implementation	6.2
37	Virtual Machine I: Stack Arithmetic	7
37	Background	7.1
39	VM Specification I	7.2
42	Standard VM Mapping on the Hack Platform	7.3
43	Implementation	7.4
44	Code module	7.5

47	Virtual Machine II: Program Control	8
47	Background	8.1
48	VM Specification II	8.2
50	Implementation	8.3
53	High-Level Language (Jack)	9
53	Background	9.1
54	Jack Specification	9.2
63	Compiler I: Syntax Analysis	10
63	Background	10.1
66	Specification & Implementation	10.2
68	Compiler II: Code Generation	11
68	Background	11.1
71	Specification & Implementation	11.2
73	Operating System	12
73	Background	12.1
73	General	12.2
73	Math - Mathematical Operation	12.3
76	Memory - Memory Management	12.4
78	Array	12.5
79	String	12.6
80	Screen - Graphics Output	12.7
83	Output - Character Output	12.8
84	Keyboard - Input Handling	12.9
86	Sys -Initialization	12.10

Hardware Platform

המטרה: פיתוח פלטפורמת ה-Hack משערי Nand.

הדרך:

1. Boolean Logic: בניית ציפים בסיסיים.
 2. Boolean Arithmetic: בניית ה-ALU, ציפ לחישובים בסיסיים.
 3. Sequential Logic: בניית ציפים המשכיים וציפים של זיכרון.
 4. Machine Language: תיאור המפרט של שפת אסמבלי עבור פלטפורמת ה-Hack.
 5. Computer Architecture: בניית המעבד והמחשב.
- הערה: המימושים בחלק הזה אמורים להיות אופטימליים. אם אתם מוצאים משהו יותר טוב - תעדכנו במייל (:)

Boolean Logic (Basic Boolean Gates) 1

* HDL Syntax - appendix A in the book.

Unary op 1.1

	Not	
	in	out
input: in	0	1
output: out	1	0
num of Nands:		1

Implementation:

$\text{Not } [in] = (in \text{ Nand } in)$

Binary op 1.2

			Nand	And	Or	Xor
	a	b	out	out	out	out
input: a, b	0	0	1	0	0	0
output: out	0	1	1	0	1	1
	1	0	1	0	1	1
	1	1	0	1	1	0
num of Nands:			1	2	3	4

Implementation:

$a \text{ And } b = \text{Not}[a \text{ Nand } b]$

$a \text{ Or } b = \text{Not}[a \text{ Nand } \text{Not}[b]]$

$a \text{ Xor } b = ((a \text{ Nand } b) \text{ Nand } a) \text{ Nand } ((a \text{ Nand } b) \text{ Nand } b)$

Mux & DMux 1,3

Mux 1,3,1

	sel	Mux
input: a, b, sel	0	a
output: out	1	b
num of Nands:		4

Implementation:

$$\text{Mux}[a,b,\text{sel}] = ((\text{Not}[\text{sel}] \text{ Nand } a) \text{ Nand } (\text{sel} \text{ Nand } b))$$

DMux 1,3,2

		DMux	
	sel	a	b
input: in, sel	0	in	0
output: a, b	1	0	in
num of Nands:		4	

Implementation:

$$\text{DMux}[\text{in},\text{sel}] = \begin{cases} a = \text{Not}[\text{sel} \text{ Nand } \text{in}] \\ b = \text{Not}[(\text{sel} \text{ Nand } \text{in}) \text{ Nand } \text{in}] \end{cases}$$

Multi-Bit 1,4

Implementation: use the basic chip for all i (16 chips overall).

Not, Or & And 1,4,1

	Not16	Or16	And16
input:	in[16]	a[16], b[16]	a[16], b[16]
output:	out[16]	out[16]	out[16]
$\forall 0 \leq i \leq 15:$	out[i]=Not[in[i]]	out[i]=a[i] Or b[i]	out[i]=a[i] And b[i]

Mux 1,4,2

	sel	Mux16
input: a[16], b[16], sel	0	a
output: out[16]	1	b

Multi-Way 1,5

Or 1,5,1

	Or8Way
input:	in[8]
output:	out=Or(in[0],...,in[7])

Implementation:

7 Or gates, $\text{out_i} = \text{Or}(\text{out_}(i-1), \text{in}[i])$ for $1 \leq i \leq 7$ ($\text{out_0} = \text{in}[0]$).

Mux 1,5,2

Mux4Way16	sel	out
input: a[16], b[16], c[16], d[16], sel[2]	00	a
output: out[16]	01	b
	10	c
	11	d

Implementation:

outA = Mux16(a,b,sel[0])

outB = Mux16(c,d,sel[0])

out = Mux16(outA,outB,sel[1])

Mux8Way16	sel	out
input: a[16], b[16], c[16], d[16], e[16], f[16], g[16], h[16], sel[3]	000	a
output: out[16]	001	b
	010	c
	011	d
	100	e
	101	f
	110	g
	111	h

Implementation:

outA = Mux4Way16(a,b,c,d,sel[0..1])

outB = Mux4Way16(e,f,g,h,sel[0..1])

out = Mux16(outA,outB,sel[2])

DMux4Way		out			
	sel	a	b	c	d
input: in, sel[2]	00	in	0	0	0
output: a, b, c, d	01	0	in	0	0
	10	0	0	in	0
	11	0	0	0	in

Implementation:

```
outA = Mux4Way16(a,b,c,d,sel[0..1])
```

```
outB = Mux4Way16(e,f,g,h,sel[0..1])
```

```
out = Mux16(outA,outB,sel[2])
```

DMux8Way		out							
	sel	a	b	c	d	e	f	g	h
input: in, sel[3]	000	in	0	0	0	0	0	0	0
output: a, b, c, d, e, f, g, h	001	0	in	0	0	0	0	0	0
	010	0	0	in	0	0	0	0	0
	011	0	0	0	in	0	0	0	0
	100	0	0	0	0	in	0	0	0
	101	0	0	0	0	0	in	0	0
	110	0	0	0	0	0	0	in	0
	111	0	0	0	0	0	0	0	in

Implementation:

```
outA, outB = DMux(in,sel[2])
```

```
a, b, c, d = DMux4Way(outA,sel[0..1])
```

```
e, f, g, h = DMux4Way(outB,sel[0..1])
```


Boolean Arithmetic (Arithmetic Logical Unit - ALU) 2

Background 2.1

Binary Numbers 2.1.1

מעבר בסיסים: נניח $x = x_n x_{n-1} \dots x_0$ מספר בבסיס b . מעבר בסיסים מבסיס b לבסיס 10 מתבצע ע"י:

$$(x)_b = (x_n x_{n-1} \dots x_0)_b = \sum_{i=0}^n x_i \cdot b^i$$

מספר בינארי הוא בבסיס 2, כלומר $b = 2$. לדוגמא:

$$(10011)_2 = 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 19$$

מינוח: נניח $y = y_{n-1} \dots y_0$ מספר בבסיס 2 (עם n ספרות). נאמר על y_0 (הספרה הכי ימנית) שהיא ה least significant bit, ועל y_{n-1} (הספרה הכי שמאלית) שהיא ה MSB - most significant bit.

Binary Addition 2.1.2

חיבור שני מספרים בינאריים עם n ספרות מתבצע ע"י חיבור ספרה ספרה מימין לשמאל, תוך העברת השארית (carry). אם השארית היא 1 עבור חיבור ה MSB זה אומר שקיבלנו overflow, ונתעלם ממנה. דוגמא עבור שני מספרים בינאריים עם 4 ביטים:

no overflow:	0	0	0	1	carry	overflow:	1	1	1	1	carry			
		1	0	0	1	x		1	0	1	1	x		
	+	0	1	0	1	y		+	0	1	1	y		
		0	1	1	1	0	$x + y$			0	0	1	0	$x + y$

Signed Binary Numbers - 2's Complement Method 2.1.3

במערכת בינארית עם n ביטים ניתן לייצג 2^n מספרים - מאפס עד $2^n - 1$. כדי לאפשר מספרים שליליים נרצה לחלק את המספרים לשניים: שליליים ואי שליליים (חיוביים + אפס). נגדיר:

$$\text{minus}(x) := \bar{x} = \begin{cases} 2^n - x & x \neq 0 \\ 0 & x = 0 \end{cases}$$

באופן זה, $x + (-x) = 2^n$, שזה בדיוק 1 ואחריו n אפסים בייצוג בינארי. מכיוון שהמערכת משתמשת ב n ביטים, נתעלם מהשארית (הביט השמאלי ביותר) ונקבל $(x)_2 + (-x)_2 = 0$, שזה מה שאנחנו מחפשים.

דוגמא עבור מערכת עם 4 ביטים:

Positive		Negative	
0	$(0000)_2$		
1	$(0001)_2$	$(1111)_2 = 15 = 2^4 - 1$	-1
2	$(0010)_2$	$(1110)_2 = 14 = 2^4 - 2$	-2
3	$(0011)_2$	$(1101)_2 = 13 = 2^4 - 3$	-3
4	$(0100)_2$	$(1100)_2 = 12 = 2^4 - 4$	-4
5	$(0101)_2$	$(1011)_2 = 11 = 2^4 - 5$	-5
6	$(0110)_2$	$(1010)_2 = 10 = 2^4 - 6$	-6
7	$(0111)_2$	$(1001)_2 = 9 = 2^4 - 7$	-7
		$(1000)_2 = 8 = 2^4 - 8$	-8

תכונות:

1. השיטה הזו מאפשרת לייצג במערכת בינארית עם n ביטים 2^n מספרים בין -2^{n-1} ל- $2^{n-1} - 1$.
2. כל המספרים החיוביים (ואפס) מתחילים ב 0.
3. כל המספרים השליליים מתחילים ב 1.
4. חיבור של מספרים שליליים או של חיובי עם שלילי מתבצע בדיוק כמו שתואר למעלה (Binary Addition). כך גם ניתן לממש חיסור: $x - y = x + (-y)$.

שיטות החישוב:

כדי לקבל את $-x$ מ x :

1. נשאיר את כל האפסים עד ה 1 הראשון מימין (כולל) כמו שהם, ונהפוך את כל הספרות מימין לו (כלומר, 1 הופך ל- 0 ולהיפך).
2. קיצור דרך: נהפוך את כל הביטים של x (1 הופך ל- 0 ולהיפך) ונוסיף 1 לתוצאה.

Add 2,2

Half-Adder 2,2,1

Designed to add two bits.

HalfAdder	a	b	carry	sum
input: a, b	0	0	0	0
output: carry, sum	0	1	0	1
sum = LSB of a+b	1	0	0	1
carry = MSB of a+b	1	1	1	0
num of Nands: 5				

Implementation:

$\text{sum} = a \text{ Xor } b = ((a \text{ Nand } b) \text{ Nand } a) \text{ Nand } ((a \text{ Nand } b) \text{ Nand } b)$
 $\text{carry} = a \text{ And } b = \text{Not}[a \text{ Nand } b]$

Full-Adder 2,2,2

Designed to add three bits.

FullAdder	a	b	c	carry	sum
input: a, b	0	0	0	0	0
output: carry, sum	0	0	1	0	1
sum = LSB of a+b+c	0	1	0	0	1
carry = MSB of a+b+c	0	1	1	1	0
	1	0	0	0	1
	1	0	1	1	0
	1	1	0	1	0
	1	1	1	1	1
num of Nands: 9					

Implementation:

$x = a + b = a \text{ Xor } b = ((a \text{ Nand } b) \text{ Nand } a) \text{ Nand } ((a \text{ Nand } b) \text{ Nand } b)$
 $\text{sum} = x + c = x \text{ Xor } c = ((x \text{ Nand } c) \text{ Nand } x) \text{ Nand } ((x \text{ Nand } c) \text{ Nand } c)$
 $\text{carry} = (a \text{ Nand } b) \text{ Nand } (x \text{ Nand } c) (*)$

(*) $\text{carry}(a+b+c) = 1$ iff $\text{carry}(a+b) = 1$ or $\text{carry}(x+c) = 1$.

$\text{carry}(a+b) = a \text{ And } b = \text{Not}[a \text{ Nand } b]$, $\text{carry}(x+c) = x \text{ And } c = \text{Not}[x \text{ Nand } c]$,

$\Rightarrow \text{carry}(a+b+c) = 1$ iff $(a \text{ Nand } b) = 0$ or $(x \text{ Nand } c) = 0$, iff $(a \text{ Nand } b) \text{ Nand } (x \text{ Nand } c) = 1$.

Designed to add two 16-bit numbers.

Add16
input: a[16], b[16]
output: out[16]
out = a+b
overflow ignored
num of Nands: $5 + 9 \cdot 14 + 8 = 139$

Implementation:

carry_0, sum[0] = HalfAdder(a[0],b[0]) - 5 Nands

for $1 \leq i \leq 14$, carry_i, sum[i] = FullAdder(a[i],b[i],carry_(i-1)) - 9 Nands for each i

x = a[15] + b[15] = a[15] Xor b[15] - 4 Nands

sum[15] = x + carry_14 = x Xor carry_14 - 4 Nands

(*) שתי השורות האחרונות מקבילות ל- FullAdder(a[i],b[i],carry_(i-1)), אך מכיוון שאין צורך בשארית נשתמש במימוש של FullAdder בלי השורה האחרונה של חישוב השארית ונחסוך שער Nand אחד.

Designed to add 1 to a given 16-bit number.

Inc16
input: in[16]
output: out[16]
out = in+1
overflow ignored
num of Nands: $1 + 5 \cdot 14 + 4 = 75$

Implementation:

out[0] = Not(in[0]) - 1 Nand

carry_0 = in[0]

for $1 \leq i \leq 14$, carry_i, out[i] = HalfAdder(carry_(i-1), in[i]) - 5 Nands for each i

out[15] = carry_14 + in[15] = carry_14 Xor in[15] - 4 Nands

(*) 1 במערכת בינארית עם 16 ספרות הוא 0000000000000001. לכן $out[0] = in[0] + 1 = \text{Not}(in[0])$, והשארית מהחיבור הזה היא 1 אם $in[0] = 1$. כעת, מכיוון שלכל $1 \leq i \leq 14$, $in[i] = 0$, נוכל לחבר לכל i רק את השארית מהחיבור הקודם ואת $in[i]$ באמצעות HalfAdder. לבסוף בחישוב של הספרה השמאלית ביותר אין צורך בחישוב השארית, לכן נוכל לחשב ישירות באמצעות Xor ולחסוך שער Nand אחד.

API:

```

Chip name: ALU
Inputs:    x[16], y[16],    // Two 16-bit data inputs
              zx,              // Zero the x input
              nx,              // Negate the x input
              zy,              // Zero the y input
              ny,              // Negate the y input
              f,               // Function code: 1 for Add, 0 for And
              no                // Negate the out output
Outputs:  out[16],         // 16-bit output
              zr,              // True iff out=0
              ng                // True iff out<0
Function: if zx then x = 0      // 16-bit zero constant
              if nx then x = !x    // Bit-wise negation
              if zy then y = 0      // 16-bit zero constant
              if ny then y = !y    // Bit-wise negation
              if f then out = x + y // Integer 2's complement addition
                  else out = x & y // Bit-wise And
              if no then out = !out // Bit-wise negation
              if out=0 then zr = 1 else zr = 0 // 16-bit eq. comparison
              if out<0 then ng = 1 else ng = 0 // 16-bit neg. comparison
Comment:  Overflow is neither detected nor handled.

```

Truth Table:

These bits instruct how to preset the x input		These bits instruct how to preset the y input		This bit selects between + / And	This bit inst. how to postset out	Resulting ALU output
zx	nx	zy	ny	f	no	out=
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x&y	if no then out=!out	f(x,y)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x y

Implementation:

```
// zero the x input?
Mux16 (a=x, b=false, sel=zx, out=Xzx);

// negate the x input?
Not16 (in=Xzx, out=notXzx);
Mux16 (a=Xzx, b=notXzx, sel=nx, out=Xnx);

// zero the y input?
Mux16 (a=y, b=false, sel=zy, out=Yzy);

// negate the y input?
Not16 (in=Yzy, out=notYzy);
Mux16 (a=Yzy, b=notYzy, sel=ny, out=Yny);

// compute x + y
Add16 (a=Xnx, b=Yny, out=AddOut);
// compute x & y
And16 (a=Xnx, b=Yny, out=AndOut);
// out = x + y (if f=1) or x & y (if f=0)
Mux16 (a=AndOut, b=AddOut, sel=f, out=FOut);

// negate the out output? and check if negative
Not16 (in=FOut, out=notFOut);
Mux16 (a=FOut, b=notFOut, sel=no, out=out, out[0..7]=out1, out[8..15]=out2, out[15]=ng);

// check if zero
Or8Way (in=out1, out=zr1);
Or8Way (in=out2, out=zr2);
Or (a=zr1, b=zr2, out=zr3);
Not (in=zr3, out=nr);
```

Additional Functionality - Shift 2.4

ShiftLeft 2.4.1

Multiplies the input by 2, equivalent to one shift left without changing the sign (the MSB). The LSB is always zero.

Implementation: $2 \cdot 14 + 2 = 30$ Nands

for $1 \leq i \leq 14$, $\text{out}[i] = \text{in}[i-1] = \text{true}$ And $\text{in}[i-1] - 2$ Nands for each i

$\text{out}[15] = \text{in}[15] = \text{true}$ And $\text{in}[15] - 2$ Nands

UnsignedShiftLeft 2.4.2

Multiplies the input by 2, equivalent to one shift left all bits. The LSB is always zero.

Implementation: $2 \cdot 15 = 30$ Nands

for $1 \leq i \leq 15$, $\text{out}[i] = \text{in}[i-1] = \text{true}$ And $\text{in}[i-1] - 2$ Nands for each i

ShiftRight 2.4.3

Divides its input by 2, equivalent to one shift right without changing the sign (the MSB). Odd numbers are handled as follows: $-5/2 \rightarrow -3$, $5/2 \rightarrow 2$

Implementation: 32 Nands

out = And16 (a[0..14]=in[1..15], a[15]=in[15], b=true)

(*) נקבל $out[15]=in[15]$, ולכל $1 \leq i \leq 14$, $out[i]=in[i+1]$

Additional Functionality - Multiplier 2.5

Mul
input: a[16], b[16]
output: out[16]
out = a*b
overflow ignored

ציפ שמכפיל שני מספרים של 16 ביטים באמצעות "כפל ארוך": לכל $0 \leq i \leq 15$, מחשב את $shift_i = a \ll i$ (UnsignedShiftLeft של a לאורך i צעדים), מחשב את $digit_i = shift_i \cdot b[i]$ (Mux16) ולבסוף מחבר את כל התוצאות.

Implementation:

Shift: $shift_1 = \text{UnsignedShiftLeft}(a)$
for $2 \leq i \leq 15$, $shift_i = \text{UnsignedShiftLeft}(shift_{i-1})$

Next Addition: $digit_0 = \text{Mux16}(\text{false}, a, \text{sel}=b[0]);$
for $1 \leq i \leq 15$, $digit_i = \text{Mux16}(\text{false}, shift_i, \text{sel}=b[i]);$

Adding: $out_0 = \text{Add16}(digit_0, digit_1)$
for $1 \leq i \leq 13$, $out_i = \text{Add16}(out_{i-1}, digit_{i+1})$
 $out = \text{Add16}(out_{13}, digit_{15})$

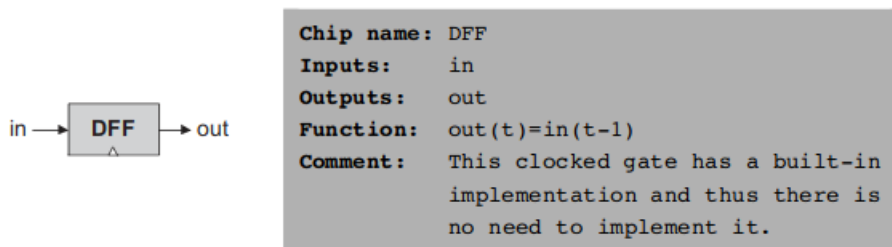
Sequential Logic (DFF, RAM) 3

DFF - Data Flip-Flops 3.1

עד עכשיו התעסקנו עם ציפים קומבינטוריים (combinational chips), כלומר כאלו שהפלט שלהם תלוי רק בקלט שקיבלו. כעת נתעסק בציפים המשכיים (sequential chips). ציפים אלו הם ציפים שהתוצאה שלהם תלויה לא רק בקלט, אלא גם בזמן שבו אנחנו נמצאים.

השעון: ברוב המחשבים, מעבר הזמן מתואם ע"י "שעון מאסטר" שעובר בין שני מצבים: 0 ו 1, או tick ו tock. הזמן שעובר בין תחילת ה "tick" לסוף ה "tock" נקרא מעגל (cycle), וכל מעגל נחשב כיחידת הזמן של המחשב. כעת נוכל לומר שכל ציף שפעולתו קשורה לשעון הוא ציף המשכי.

DFF: זהו ציף המשכי שמחובר לשעון. בכל יחידת זמן t , $out[t]=in[t-1]$, כלומר הפלט בזמן t הוא הקלט בזמן $t-1$. הציף הזה מסופק לנו כאבן בניין בסיסית של המחשב (כמו הציף Nand).



מכיוון שהפלט של ציף המשכי בזמן t לא תלוי בקלט בזמן t אלא בקלט בזמן $t-1$, אין בעיה להכניס את הפלט שלו בזמן t כקלט שלו בזמן t , וניתן גם להכניס אותו כקלט לציפים קומבינטוריים כלשהם ואז להשתמש בפלט שלהם כקלט של הציף ההמשכי, כלומר לעשות "מעגל".

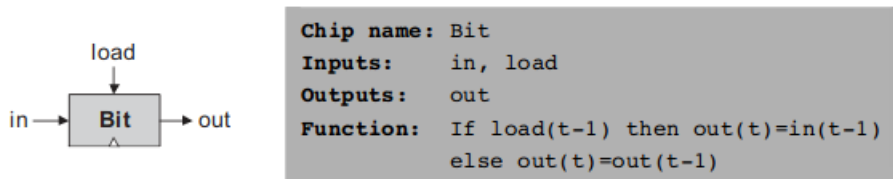
Registers 3.2

ציף שמסוגל לזכור, לאחסן מידע לאורך זמן, וכן יכול לשנות את המידע שהוא מאחסן. מבוסס על DFF.

Bit 3.2.1

רגיסטר שמאחסן ביט אחד בלבד.

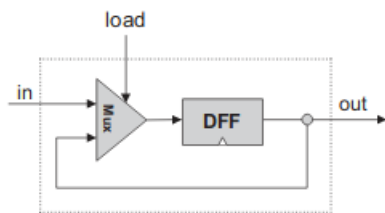
API:



Output the data that the Bit currently store.

If $load = 1$ then load the Bit with the input, else change nothing.

Implementation:



if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

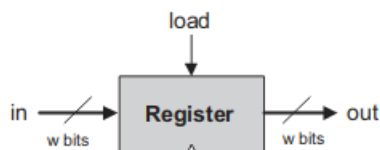
1-bit register (Bit)

```
Mux(a=in, b=DFFout, sel=load, out=muxOut);
DFF(in=muxOut, out=DFFout, out=out);
```

Register 3.2.2

רגיסטר שמאחסן 16 ביטים (אצלנו יכול גם לאחסן יותר. מסמנים את מס' הביטים שהוא מאחסן ב-w, רוחב - width). סדרת ביטים המאוחסנת ברגיסטר נקראת גם מילה.

API:

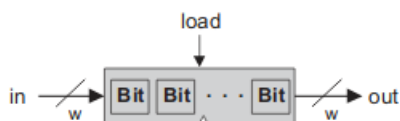


Chip name: Register
Inputs: in[16], load
Outputs: out[16]
Function: If load($t-1$) then out(t)=in($t-1$)
else out(t)=out($t-1$)
Comment: "=" is a 16-bit operation.

Output the data that the Register currently store.

If load = 1 then load the Register with the input, else change nothing.

Implementation:



if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

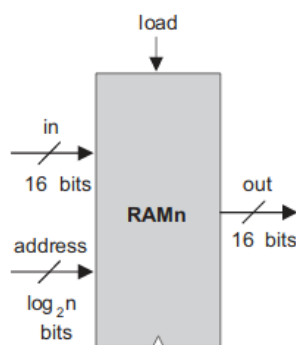
w-bit register

```
for  $0 \leq i \leq 15$ , Bit(in=in[i], load=load, out=out[i]);
```

Memory 3.3

צ'יפ זיכרון נקרא גם Direct-Access Memory או RAM. זהו מערך של n רגיסטרים, שניתן לגשת אליהם באופן ישיר, כלומר זמן הגישה לכל הרגיסטרים הוא זהה. מספר הרגיסטרים בצ'יפ, n , נקרא הגודל, size, של הצ'יפ, והרוחב של הרגיסטרים בצ'יפ (זהה בכל הרגיסטרים) נקרא ה- width של הצ'יפ. אצלנו הרוחב הוא 16 בכל הצ'יפים, ויש לנו מספר גדלים: RAM8, RAM16, RAM512, RAM4K, RAM16K. לכולם אותו API מלבד שני פרמטרים (בהמשך). הגישה הישירה לרגיסטרים מתבצעת באמצעות כתובת שניתנת כקלט. עבור צ'יפ עם 2^n רגיסטרים, נקבל כתובת עם n ביטים. הכתובת היא בעצם המקום של הרגיסטר הרצוי במערך. ה"יחידה" שניגשת לרגיסטר הנכון בהינתן הכתובת נקראת direct-access logic. בכל זמן t , הצ'יפ מקבל קלט $in[16]$, כתובת הרגיסטר הרצוי וביט נוסף load שמסמן האם לכתוב ברגיסטר או רק לקרוא.

API:



```
Chip name: RAMn // n and k are listed below
Inputs:   in[16], address[k], load
Outputs:  out[16]
Function: out(t)=RAM[address(t)](t)
          If load(t-1) then
              RAM[address(t-1)](t)=in(t-1)
Comment:  "=" is a 16-bit operation.

The specific RAM chips needed for the Hack platform are:
```

Chip name	n	k
RAM8	8	3
RAM64	64	6
RAM512	512	9
RAM4K	4096	12
RAM16K	16384	14

Output the data that the Register = RAM[address] currently store.

If load = 1 then load the Register with the input, else change nothing.

Implementation: for RAM8

```
// route the load param to the right address, all the rest are 0
DMux8Way (in=load, sel=address, a=load0, b=load1, c=load2, d=load3, e=load4, f=load5, g=load6, h=load7);

// update the value in the right address (only if it's load = 1),
// in all other registers nothing happens
Register (in=in, load=load0, out=out0);
Register (in=in, load=load1, out=out1);
Register (in=in, load=load2, out=out2);
Register (in=in, load=load3, out=out3);
Register (in=in, load=load4, out=out4);
Register (in=in, load=load5, out=out5);
Register (in=in, load=load6, out=out6);
Register (in=in, load=load7, out=out7);

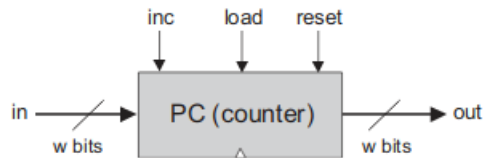
// get the out data according to the address
Mux8Way16 (a=out0, b=out1, c=out2, d=out3, e=out4, f=out5, g=out6, h=out7, sel=address, out=out);
```

כל שאר הצ'יפים מממשים באותו אופן, רק במקום Register שמים את הצ'יפ הקודם בסדר, ומשתמשים בחלק המתאים ב- address (מ-0 עד k עבור הצ'יפ הקודם והשלושה הנותרים ל DMux).

3.4 PC - Program Counter

צ'יפ המשכי, קאונטר של 16-ביט. מאחסן 16 ביט כמו רגיסטר, אבל עם תכונות נוספות.

API:



```

Chip name: PC // 16-bit counter
Inputs:   in[16], inc, load, reset
Outputs:  out[16]
Function: If reset(t-1) then out(t)=0
          else if load(t-1) then out(t)=in(t-1)
          else if inc(t-1) then out(t)=out(t-1)+1
          else out(t)=out(t-1)
Comment:  "=" is 16-bit assignment.
          "+" is 16-bit arithmetic addition.
    
```

מקבל קלט: inc , $load$, $reset$, $in[16]$. בכל זמן t :

- אם $reset=1$ אז מאפס את המידע השמור ומחזיר 0.
- אחרת, אם $load=1$ אז הוא מתנהג בדיוק כמו רגיסטר.
- אחרת, אם $inc=1$ אז הוא מעלה את המידע השמור ב-1 ומחזיר את הערך הישן.
- אחרת רק מחזיר את הערך השמור.

Implementation:

```

Register(in=outReset, load=true, out=out, out=outReg);
Inc16(in=outReg, out=tempInc);
Mux16(a=outReg, b=tempInc, sel=inc, out=outInc);
Mux16(a=outInc, b=in, sel=load, out=outLoad);
Mux16(a=outLoad, b=false, sel=reset, out=outReset);
    
```

Machine Language (HACK Lang. & Assembly specification) 4

Background 4.1

שפה (פורמליזם) המורכבת מהוראות כתובות כמילים בינאריות, במקרה שלנו באורך 16 ביט. מתקשרת ישירות עם החומרה ומפעילה את הזיכרון באמצעות המעבד (processor) וקבוצה קטנה של רגיסטרים. כאשר מדברים עם שפת מכונה מדברים בעצם גם על אסמבלי וגם על הייצוג הבינארי. הייצוג הבינארי הוא שפת מכונה ש"מדברת" ישירות עם המחשב, ואסמבלי היא שפת מכונה סמלית, ודרוש תרגום לשפת המכונה הבינארית כדי להכניס אותה למחשב (נעשה את זה בפרק 6).

Machine Language - Machines 4.1.1

- הזיכרון: קבוצת מכשירים (צ'יפים) שמאחסנים מידע והוראות במחשב. מבחינת המתכנת בשפת מכונה, זהו מערך של רגיסטרים (במקרה שלנו בגודל K32) ברוחב קבוע (במקרה שלנו 16), לכל אחד כתובת קבועה וייחודית. לכן נוכל להתייחס למילה בזיכרון לפי הכתובת שלה. בהמשך נשתמש בנוטציות $Memory[address]$, $RAM[address]$, $M[address]$, כולן שקולות.

- המעבד: יחידת החישוב המרכזית (Central Processing Unit) או CPU. מכשיר (צ'יפ) שמסוגל לבצע מספר קבוע של פעולות: פעולות אריתמטיות ולוגיות, גישה לזיכרון ו-control (branching) operation. המידע עליו מתבצעת הפעולה (דאטה או כתובת) היא מילה בינארית שמגיעה מאחד הרגיסטרים או מכתובת זיכרון, והתוצאה של הפעולה יכולה להיות משוכנת באחד הרגיסטרים או בכתובת זיכרון כלשהי.

- רגיסטרים: מכיוון שגישה לזיכרון היא פעולה ארוכה, ישנם מספר רגיסטרים שיושבים (פיזית) קרוב למעבד, כך שהוא יכול לגשת אליהם במהירות.

Machine Language - Language 4.1.2

כל הוראה בשפה מתורגמת לאחת מהפעולות של ה-CPU.

- פעולות אריתמטיות ולוגיות.

- גישה לזיכרון: יש שתי קטגוריות - גישה לשם ביצוע פעולה אריתמטית או לוגית על מידע המאוחסן בזיכרון, וגישה לשם איחסון מידע בכתובת מסוימת. יש מספר addressing modes - דרכים לפרט את הכתובת של המידע הדרוש, בד"כ משתמשים (לפחות) בשלוש הדרכים הבאות:

1. גישה ישירה - direct addressing: גישה באמצעות כתובת ספציפית - $R1 < -Memory[67]$.

2. גישה מיידיית - immediate addressing: מציבה קבוע ברגיסטר או בכתובת הזיכרון. כלומר, הערך הדרוש מופיע בהוראה ולא דרושה גישה מיוחדת כדי לגשת אליו - $R1 < -67$.

3. גישה עקיפה - indirect addressing: הכתובת אליה רוצים לגשת אינה מפורשת בסדרת ההוראות. ניתנת כתובת בזיכרון בה הכתובת הדרושה מאוחסנת, ויש צורך לגשת אל מקום האחסון שלה, לקרוא את הכתובת משם ואז לגשת אל הכתובת - $R1 < -i+j$, $R2 < -Memory[R1]$.

- Flow of control: תוכנה בדר"כ רצה באופן לינארי - שורה אחר שורה. אך לפעמים נרצה לשנות את זה ולקפוץ למקום אחר ברצף ההוראות, למשל במקרה שנרצה לחזור על קוד (לולאה), לבצע משהו בתנאי כלשהו (if-then) או לקפוץ לשורה הראשונה של חלק אחר בקוד (קריאה לפונקציה).

4.2 HACK Machine Language

המחשב שלנו (the HACK computer) הוא מכונה עם אורך מילה של 16 ביטים, שמורכבת מ-CPU, יחידת זיכרון שמאחסנת מידע, RAM, (נתייחס לזה פשוט כ"זיכרון"), יחידת זיכרון שמאחסנת הוראות ושני I/O devices (מכשירי קלט - פלט): מקלדת ומסך.

- זיכרון: ה-CPU מסוגל להריץ רק הוראות שמאוחסנות בחלק הזיכרון המיועד להן. זהו זיכרון לקריאה בלבד (Read-only Memory), ROM, שמועלה למחשב באופן חיצוני, למשל באמצעות צ'יפ ROM. בכל פעם שנרצה לטעון תוכנה חדשה, נצטרך להכניס למחשב צ'יפ חדש שעליו צרובות ההוראות. אצלנו ה-Hardware simulator מספק דרך לסמלץ את התהליך הזה באמצעות העלאת קובץ טקסט המכיל את ההוראות. כתובות הזיכרון מורכבות מ-15 ביטים.

- רגיסטרים: במחשב שלנו יש שני רגיסטרים - A ו-D. שניהם יכולים להכיל מידע וניתן לבצע עליהם פעולות אריתמטיות ולוגיות. בנוסף, הרגיסטר A מתפקד גם בתור address register, כלומר בהינתן הוראה ניתן לפרש את התוכן של A כערך מידע כלשהו או ככתובת זיכרון (של הזיכרון הרגיל או זיכרון ההוראות): ראשית, ניתן להשתמש ב-A לגישה ישירה לזיכרון. מכיוון שכתובת זיכרון דורשת 15 ביט, ומילים בשפה הן באורך 16 ביט, לא ניתן לכתוב בהוראה אחת גם כתובת מסויימת וגם לפרט פעולה נוספת. לשם כך, גישה לכתובת זיכרון פועלת על כתובת זיכרון לא מפורשת בשם M. העלאה של כתובת זיכרון ל-A מעלה אוטומטית את התוכן של הכתובת הזו ל-M, וכך ניתן גם להשתמש בתוכן של הכתובת וגם לשנות אותה (באמצעות הצבת ערך שונה ב-M). בנוסף, ניתן להשתמש ב-A לגישה ישירה לזיכרון ההוראות, עבור קפיצה (Flow of control). ההוראה של קפיצה למקום אחר בזיכרון ההוראות (בהמשך) לא מכילה כתובת, אלא מכילה הוראת קפיצה לכתובת (בזיכרון ההוראות) המאוחסנת באותו רגע ב-A.

4.3 Machine Language & Assembly Syntax

בשפת המכונה שלנו יש שני סוגים של הוראות: A-instruction & C-instruction. * Mnemonic: סמל ששמו רומז על פעולתו. במקרה שלנו זהו הסמל הניתן בשפת אסמבלי לפעולה בשפת מכונה. דוגמאת קוד אסמבלי: בצד שמאל קוד בשפת C, בצד ימין התרגום לאסמבלי.

```
// Adds 1+...+100.
int i = 1;
int sum = 0;
While (i <= 100){
    sum += i;
    i++;
}
```

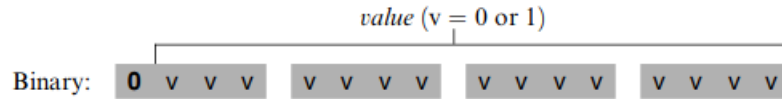
```
// Adds 1+...+100.
@i      // i refers to some mem. location.
M=1     // i=1
@sum    // sum refers to some mem. location.
M=0     // sum=0
(LLOOP)
@i
D=M     // D=i
@100
D=D-A   // D=i-100
@END
D;JGT   // If (i-100)>0 goto END
@i
D=M     // D=i
@sum
M=D+M   // sum=sum+i
@i
M=M+1   // i=i+1
@LOOP
0;JMP   // Goto LOOP
(END)
@END
0;JMP   // Infinite loop
```

4.3.1 A-instruction

כאשר הביט הימני הוא 0.

משמשת להעלאה של ערך בינארי באורך 15 ביט לרגיסטר A.

A-instruction: @value // Where value is either a non-negative decimal number
// or a symbol referring to such number.



שימושים:

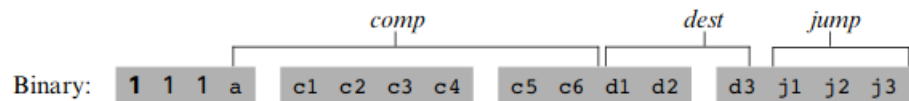
- הכנסת קבוע - @5 יקבע את הערך של A ל-5. קבועים חייבים להיות אי שליליים, נכתבים בבסיס 10.
- קובע את M ל- Memory[A].
- מאפשר קפיצה לשורה מסויימת בזיכרון ההוראות.

4.3.2 C-instruction

כאשר הביט הימני הוא 1.

משמשת לכל פעולות החישוב, ההשמה והקפיצה.

C-instruction: dest=comp;jump // Either the dest or jump fields may be empty.
// If dest is empty, the "=" is omitted;
// If jump is empty, the ";" is omitted.



1. Computation: ה-ALU מעוצב לחשב מספר קבוע של פעולות על הרגיסטרים A, D ו-M (M=Memory[A]). הפונקציה לחישוב נקבעת לפי הביט a וששת הביטים c:

(when a=0)		c1	c2	c3	c4	c5	c6	(when a=1)	
comp	mnemonic							comp	mnemonic
0		1	0	1	0	1	0		
1		1	1	1	1	1	1		
-1		1	1	1	0	1	0		
D		0	0	1	1	0	0	M	
A		1	1	0	0	0	0		
!D		0	0	1	1	0	1		
!A		1	1	0	0	0	1	!M	
-D		0	0	1	1	1	1		
-A		1	1	0	0	1	1	-M	
D+1		0	1	1	1	1	1		
A+1		1	1	0	1	1	1	M+1	
D-1		0	0	1	1	1	0		
A-1		1	1	0	0	1	0	M-1	
D+A		0	0	0	0	1	0	D+M	
D-A		0	1	0	0	1	1	D-M	
A-D		0	0	0	1	1	1	M-D	
D&A		0	0	0	0	0	0	D&M	
D A		0	1	0	1	0	1	D M	

2. **Destination**: הערך שמחושב ע"י ה-ALU בחלק 1 של ה- C-instruction יכול להיות מאוחסן ברגיסטרים A ו- D, או להיות מושם ב- M, כלומר בתא הזיכרון עם הכתובת שנמצאת ב- A. ניתן לאחסן את התוצאה באחד מהם, בכמה מהם או באף אחד מהם. המקום לאחסון נקבע ע"י שלושת הביטים d: d1 - אם d1 דולק נאחסן את תוצאת החישוב ב- A.
d2 - אם d2 דולק נאחסן את תוצאת החישוב ב- D.
d3 - אם d3 דולק נאחסן את תוצאת החישוב ב- M.

d1	d2	d3	Mnemonic	Destination (where to store the computed value)
0	0	0	null	The value is not stored anywhere
0	0	1	M	Memory[A] (memory register addressed by A)
0	1	0	D	D register
0	1	1	MD	Memory[A] and D register
1	0	0	A	A register
1	0	1	AM	A register and Memory[A]
1	1	0	AD	A register and D register
1	1	1	AMD	A register, Memory[A], and D register

3. **Jump**: אומר למחשב מה לעשות אחרי הצעד הנוכחי: להמשיך להוראה הבאה (דיפולטיבי) או לקפוץ להוראה שממוקמת במקום אחר בתוכנית. במקרה השני, אנחנו מניחים שהרגיסטר A מכיל את הכתובת להוראה בזיכרון ההוראות שאליה אנחנו רוצים לקפוץ. שלושת הביטים j קובעים באיזה מקרה נקפוץ, לפי הערך שמחושב ע"י ה-ALU בחלק 1:

- אם j1 דולק נקפוץ אם הערך שלילי.
- אם j2 דולק נקפוץ אם הערך אפס.
- אם j1 דולק נקפוץ אם הערך חיובי.

j1 (out < 0)	j2 (out = 0)	j3 (out > 0)	Mnemonic	Effect
0	0	0	null	No jump
0	0	1	JGT	If out > 0 jump
0	1	0	JEQ	If out = 0 jump
0	1	1	JGE	If out ≥ 0 jump
1	0	0	JLT	If out < 0 jump
1	0	1	JNE	If out ≠ 0 jump
1	1	0	JLE	If out ≤ 0 jump
1	1	1	JMP	Jump

- * במקרה שנוכל לקפוץ בלי קשר לערך החישוב, נכתוב JMP;0, כאשר מתעלמים מה-0 (הוא שם כי חייבים את החלק comp).
- * כדי למנוע בלבול בין הזיכרון הרגיל לזיכרון הכתובות כיעד לקפיצה, כאשר ההוראה עלולה לגרום לקפיצה (כלומר כאשר הביטים של jump לא ריקים) נימנע מלהשתמש ב- M (ולהיפך).

4.3.3 Additional Functionality - Multiplication and shift

הרחבה של C-instruction: תמיכה בפעולות כפל ו- shift. כמו בכל C-instruction, ביט 15 יהיה דולק.

1. כפל: אם ביט 14 דולק וביט 13 כבוי, נבצע פעולת כפל בין D ל- A/M (נקבע לפי הביט a).

Instruction	14	13	12,a	11-6,c1-c6
D*A	1	0	0	0
D*M	1	0	1	0

2. shift: אם ביט 14 כבוי וביט 13 דולק נבצע פעולת שיפט:

אם c2 דולק נבצע שיפט על D. אחרת, אם a דולק נבצע שיפט על M, אחרת נבצע שיפט על A.

אם c1 דולק נבצע שיפט שמאלה, אחרת ימינה.

Instruction	Command	Description	14	13	12,a	11,c1	10,c2	9-6,c3-c6
Shift Left D	D<<	Shift left the content of D	0	1	0	1	1	0
Shift Left A	A<<	Shift left the content of A	0	1	0	1	0	0
Shift Left M	M<<	Shift left the content of M	0	1	1	1	0	0
Shift Right D	D>>	Shift right the content of D	0	1	0	0	1	0
Shift Right A	A>>	Shift right the content of A	0	1	0	0	0	0
Shift Right M	M>>	Shift right the content of M	0	1	1	0	0	0

4.3.4 Assembly - Symbols

פקודות אסמבלי יכולות להתייחס לכתובות זיכרון כקבועים או כסמלים. יש שלושה סוגים של סמלים:

1. **Predefined symbols**: קבוצה של כתובות RAM (הזיכרון הרגיל) שניתן להתייחס אליהן בכל תוכנת אסמבלי באמצעות הסמלים הבאים:

- **Virtual registers**: הסמלים R0 עד R15 מייחסים לכתובות ה- RAM 0 עד 15 בהתאמה.

- **Predefined pointers**: הסמלים SP, LCL, ARG, THIS, THAT מתייחסים לכתובות ה- RAM 0 עד 4 בהתאמה.

- **I/O pointers**: הסמל SCREEN מתייחס לכתובת ה- RAM 16384 והסמל KBD מתייחס לכתובת ה- RAM 24576. אלו הכתובות שהן הבסיס למפות הזיכרון (בהמשך) של המסך והמקלדת בהתאמה.

2. **Label symbols**: סמלים שהמשתמש יכול להגדיר, שמשרתים את פקודת ה- jump.

מצהירים על לייבל באמצעות "(LabelName)". שורה זו מגדירה תווית LabelName שמתייחסת למיקום בזיכרון ההוראות שמחזיק את ההוראה שבאה אחרי שורת ההצהרה. לאחר מכן, כדי לקפוץ לשורה זו, נעלה לרגיסטר A את הכתובת הזו באמצעות "@LabelName" ואז נכתוב שורה של C-instruction עם פקודת קפיצה מתאימה. כל תווית יכולה להיות מוצהרת פעם אחת, וניתן להשתמש בה כמה פעמים שרוצים.

3. **Variable symbols**: סמלים שהמשתמש יכול להגדיר. כל סמל שאין אחד מהקבועים ואינו מוגדר במקום

אחר בתוך תווית נחשב משתנה. במקרה זה הוא מקבל כתובת זיכרון ייחודית ב- RAM ע"י האסמבלר (התוכנה שמתרגמת מאסמבלי לשפת מכונה), החל מ- 16.

* סמלים של המשתמש הם סדרה של אותיות, ספרות, וסימנים: "_", ".", "\$", ":", ". לא מתחילים עם ספרה. השמות רגישים ל-case.

למחשב שלנו יכולים להיות מחוברים מקלדת ומסך. עם כל אחד מהם המחשב מתקשר באמצעות מפת זיכרון. כלומר, מוקצה להם חלק בזיכרון שאיתו המכשירים מסונכרנים. כתיבה במפת הזיכרון של המסך תשנה את הפיקסלים על המסך, וקריאה ממפת הזיכרון של המקלדת תאפשר לנו לדעת על איזה מקש המשתמש לוחץ ברגע נתון.

1. מסך: מסך שחור-לבן עם 256 שורות של 512 פיקסלים כל אחת. מפת הזיכרון של המסך משוכנת בצ'יפ זיכרון של 8K, ומתחילה בכתובת ה-RAM 16394. כל שורה במסך, החל מהשורה השמאלית העליונה, מיוצגת ע"י 32 תאים רציפים בזיכרון.

גישה לפיקסל בשורה r ובעמודה c תתקבל באמצעות גישה למילה שמאוחסנת ב-RAM בכתובת $16384 + r \cdot 32 + \frac{c}{16}$, ואז גישה לפיקסל מספר $c \% 16 = c \bmod 16$ במילה זו (כאשר סופרים מימין לשמאל). כדי לשנות את הפיקסל על המסך צריך פשוט לשנות את הביט בזיכרון: שחור=1 ולבן=0.
לדוגמא:

```
// Draw a single black dot at the screen's top left corner:
@SCREEN // Set the A register to point to the memory
        // word that is mapped to the 16 left-most
        // pixels of the top row of the screen.
M=1     // Blacken the left-most pixel.
```

2. מקלדת: מפת הזיכרון של המקלדת היא מילה אחת שממוקמת בכתובת ה-RAM 24576. כאשר מקש נלחץ במקלדת הפיזית, קוד ה-ASCII (בבינארי) מופיע במפת הזיכרון של המקלדת. כאשר לא נלחץ שום מקש, יופיע הערך 0. בנוסף ישנם עוד מספר ערכים נוספים.

טבלת ASCII: <http://facweb.cs.depaul.edu/sjost/it212/documents/ascii-pr.htm>

תווים נוספים:

Key pressed	Code	Key pressed	Code
newline	128	end	135
backspace	129	page up	136
left arrow	130	page down	137
up arrow	131	insert	138
right arrow	132	delete	139
down arrow	133	esc	140
home	134	f1-f12	141-152

Project 4.4

כתיבת מספר תוכניות באסמבלי: fill, mult, sort, divide. האלגוריתם ל-mult לקוח מהחלק האחרון של הספר (מערכת ההפעלה, פרק 12). האלגוריתם ל-divide הוא חילוק ארוך, והאלגוריתם הכי פקוט ל-sort הוא bubble sort.

אלגוריתם חילוק ארוך ל-divide:

- מחשבים: $R13/R14$ ומציבים את התוצאה ב- $R15$ ($R13, R14 > 0$).

if $R14 > R13$, jump to END ($R15 = 0$)

digit = $16384 // 2^{15}$

remainder = 0

while digit $\neq 0$:

- temp = digit AND $R13 //$ get the i-th bit, $i = 15, \dots, 0$

- if temp $\neq 0$:

- - - remainder += 1

- if remainder $\geq R14$:

- - - remainder -= $R14$

- - - $R15 +=$ digit

shift right digit

shift left remainder

5 Computer Architecture (CPU, Computer)

5.1 Background

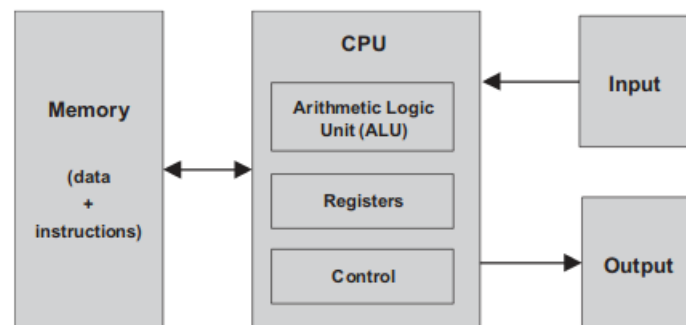
בפרק זה בונים את המחשב עצמו מכל הצ'יפים שבנינו עד עכשיו.

5.1.1 The Stored Program Concept

המחשב עצמו מורכב מחומרה קבועה ומסוגל לבצע מספר קבוע של פעולות. ניתן לחבר את הפעולות האלו כמו אבני בניין וכך לקבל תוכנות מורכבות. התוכנה עצמה (סדרת הפעולות) לא מבוססת בחומרה אלא נשמרת בזיכרון המחשב, כמו דאטה רגיל. כך המחשב מסוגל להפעיל תוכנות שונות, לפי מה שנמצא בזיכרון שלו (שאותו ניתן לשנות).

5.1.2 Von Neumann Architecture

מבוססת על יחידת חישוב מרכזית - Central Processing Unit (CPU) שמתקשר עם הזיכרון (Memory chip) שמכיל את הדאטה (RAM) ואת מפות הזיכרון של הקלט/פלט ועם הצ'יפ שמכיל את סדרת ההוראות הנוכחית (ROM), מקבל מידע דרך מכשיר קלט (input device) ושולח מידע דרך מכשיר פלט (output device).



5.2 CPU

אחראי לביצוע ההוראות בתכנית הנוכחית. מבצע לולאה שכל איטרציה בה מורכבת ממספר פעולות: קבל הוראה, תפרש אותה, בצע אותה ועבור לאיטרציה הבאה.
ה-CPU מורכב משלושה חלקים:

1. ALU: מבצע חישובים אריתמטיים ולוגיים.

2. Registers: מכיוון שגישה לזיכרון היא פעולה ארוכה, ישנם מספר רגיסטרים שיושבים (פיזית) קרוב למעבד, כך שהוא יכול לגשת אליהם במהירות. אצלנו יש שלושה:

- Data register: הרגיסטר D שתואר בפרק הקודם.

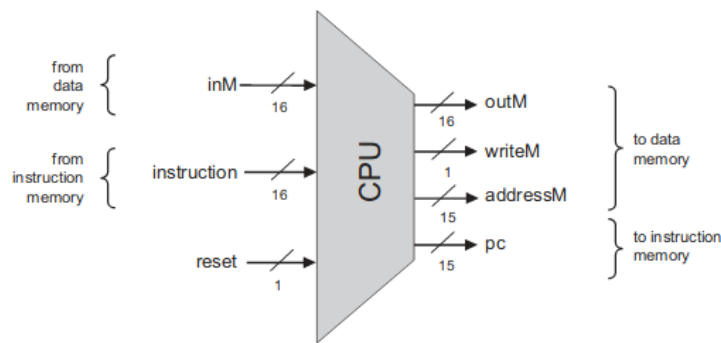
- Address register: הרגיסטר A שתואר בפרק הקודם.

- Program counter: שומר את הכתובת של ההוראה הבאה לביצוע ע"י ה-CPU. ה-CPU יכול לשנות אותה אם מתבצעת קפיצה, אחרת הוא מעלה אותה ב-1 בסוף הקריאה הנוכחית. הפלט של הקאונטר מחובר ישירות ל-address input של הצ'יפ ROM, ששומר את ההוראות. כלומר בכל שלב, ה-ROM פולט את הערך שנמצא ב-ROM[PC]. הערך הזה יהיה מילה עם 16 ביט שתהיה ההוראה הנוכחית של ה-CPU.

3. Control Unit: מילה (16 ביטים) שמחזיקה את ההוראה הנוכחית.

הוא מחובר לשתי יחידות זיכרון נפרדות: RAM ו-ROM.

API:



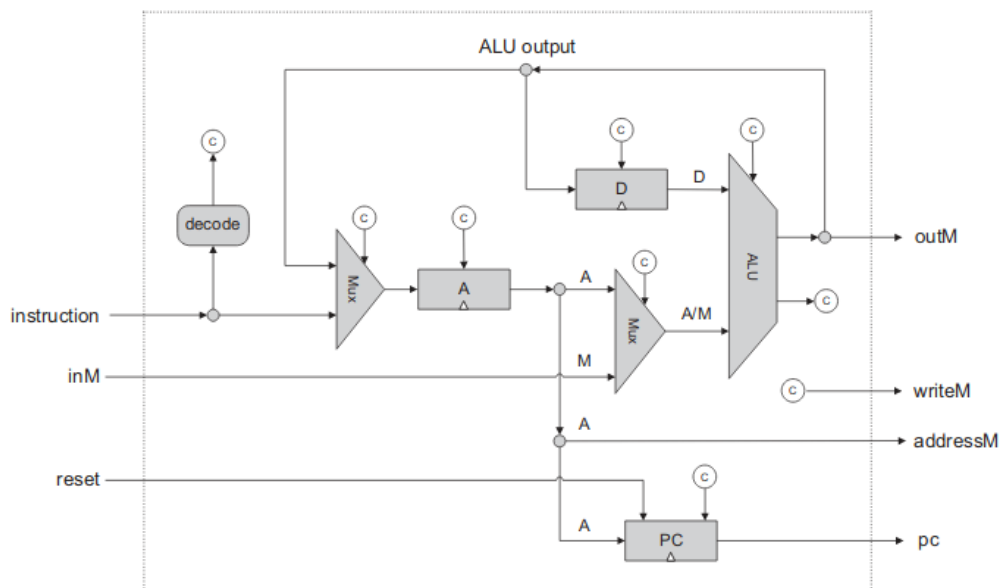
```

Chip Name: CPU           // Central Processing Unit
Inputs:   inM[16],       // M value input (M = contents of RAM[A])
              instruction[16], // Instruction for execution
              reset          // Signals whether to restart the current
                              // program (reset=1) or continue executing
                              // the current program (reset=0)
Outputs:  outM[16],     // M value output
              writeM,       // Write to M?
              addressM[15],  // Address of M in data memory
              pc[15]        // Address of next instruction
Function: Executes the instruction according to the Hack machine language
              specification. The D and A in the language specification refer to
              CPU-resident registers, while M refers to the memory location
              addressed by A (inM holds the value of this location).

              If the instruction needs to write a value to M, the value is
              placed in outM, the address is placed in addressM, and the writeM
              bit is asserted. (When writeM=0, any value may appear in outM.)

              If reset=1, then the CPU jumps to address 0 (i.e., sets pc=0 in
              the next time unit) rather than to the address resulting from
              executing the current instruction.
  
```

Implementation:



1. Instruction Decoding: פירוש ההוראה הנוכחית (לפי הסינטקס של שפת מכונה).

```
// choose the input to ALU: A or M. choose M iff a bit = inst[12] = 1.
Mux16(a=outA, b=inM, sel=inst[12], out=ALUinput);
// choose the input to Aregister: ALUout or inst. choose ALUout iff inst[15] =1 (C instruction).
Mux16(a=inst, b=ALUout, sel=inst[15], out=Ainput);
// decide if to load data to Aregister: yes iff inst[15] = 0 (A instruction) or d1 bit = inst[5] = 1
Not(in=inst[15], out=Ainst);
Or(a=Ainst,b=inst[5],out=loadA);
// decide if to load data to Dregister: yes iff inst[15] = 1 (C instruction) and d2 bit = inst[4] = 1
And(a=inst[15],b=inst[4],out=loadD);
// decide if to write to M: yes iff inst[15] = 1 (C instruction) and d3 bit = inst[3] = 1
And(a=inst[15],b=inst[3],out=writeM);
```

2. Instruction Execution: הפעלת כל החלקים ב-CPU לפי ההוראה.

```
// update registers
ARegister(in=Ainput, load=loadA, out=outA, out[0..14]=addressM);
DRegister(in=ALUout, load=loadD, out=outD);
// calculate with ALU: inst[11]=c1,...,inst[6]=c6.
ALU(x=outD, y=ALUinput, zx=inst[11], nx=inst[10], zy=inst[9], ny=inst[8], f=inst[7], no=inst[6],
out=ALUout, out=outM, zr=zrRes, ng=negRes);
```

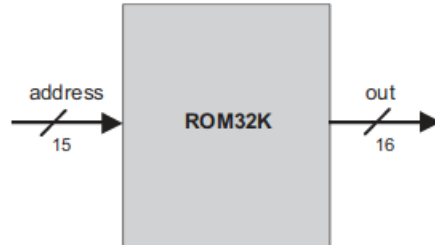
3. Next Instruction Fetching: מציאת הכתובת ב-ROM של ההוראה הבאה שיש לבצע (פונקציה של ההוראה ושל פלט ה-ALU).

```
// choose the input to ALU: A or M. choose M iff a bit = inst[12] = 1.
Mux16(a=outA, b=inM, sel=inst[12], out=ALUinput);
// choose the input to Aregister: ALUout or inst. choose ALUout iff inst[15] =1 (C instruction).
Mux16(a=inst, b=ALUout, sel=inst[15], out=Ainput);
// decide if to load data to Aregister: yes iff inst[15] = 0 (A instruction) or d1 bit = inst[5] = 1
Not(in=inst[15], out=Ainst);
Or(a=Ainst,b=inst[5],out=loadA);
// decide if to load data to Dregister: yes iff inst[15] = 1 (C instruction) and d2 bit = inst[4] = 1
And(a=inst[15],b=inst[4],out=loadD);
// decide if to write to M: yes iff inst[15] = 1 (C instruction) and d3 bit = inst[3] = 1
And(a=inst[15],b=inst[3],out=writeM);
```

5.3 Instruction Memory - ROM

צ'יפ זיכרון לקריאה בלבד (direct-access Read-Only Memory). שומר את סדרת ההוראות של התוכנית הנוכחית שהוכנסה למחשב. אצלנו זהו צ'יפ זיכרון 32K. אנחנו נשתמש בצ'יפ ROM32K - Built-in.

API:



```
Chip Name: ROM32K           // 16-bit read-only 32K memory
Input:      address[15]      // Address in the ROM
Output:     out[16]          // Value of ROM[address]
Function:   out=ROM[address] // 16-bit assignment
Comment:    The ROM is preloaded with a machine language program.
             Hardware implementations can treat the ROM as a
             built-in chip. Software simulators must supply a
             mechanism for loading a program into the ROM.
```

5.4 Data Memory

מכיל שלושה חלקים: צ'יפ RAM16K שמכיל דאטה רגיל, צ'יפ Screen שמכיל את מפת הזיכרון של המסך וצ'יפ Keyboard שמכיל את מפת הזיכרון של המקלדת. מתנהג כמו צ'יפ RAM רגיל.

- כתובות 0 עד 16383 (0 עד $2^{14} - 1 = 0011111111111111$ בבינארי) נמצאות ב-RAM. בכתובות אלו ביט 15 = 0, ביט 14 = 0 וביט 13 = 0.

- כתובות 16384 עד 24575 ($2^{14} = 0100000000000000$ עד $2^{14} + 2^{13} - 1 = 0101111111111111$ בבינארי) נמצאות במפת הזיכרון של המסך. בכתובות אלו ביט 15 = 0, ביט 14 = 1 וביט 13 = 0.

- הכתובות 24576 ($0110000000000000 = 2^{13} + 2^{14}$ בבינארי) נמצאת במפת הזיכרון של המקלדת. בכתובות אלו ביט 15 = 0, ביט 14 = 1 וביט 13 = 1.

- כל כתובת מעל 24576 אינה חוקית.

עבור מפות הזיכרון של המסך והמקלדת נשתמש בצ'יפים Built-in.

API:

Screen:

```

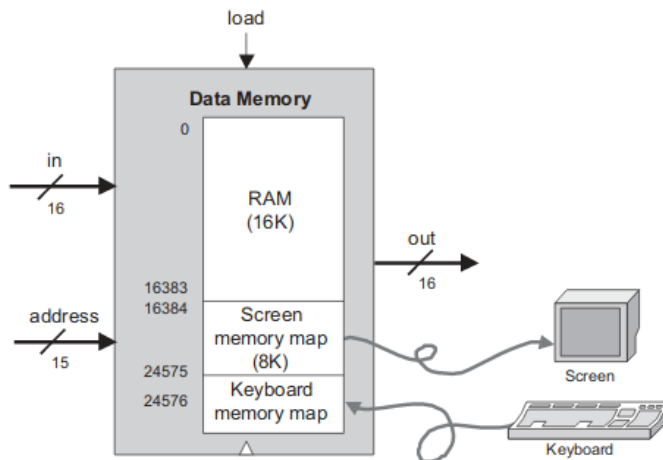
Chip Name: Screen      // Memory map of the physical screen
Inputs:    in[16],     // What to write
              load,       // Write-enable bit
              address[13] // Where to write
Output:    out[16]     // Screen value at the given address
Function: Functions exactly like a 16-bit 8K RAM:
              1. out(t)=Screen[address(t)](t)
              2. If load(t-1) then Screen[address(t-1)](t)=in(t-1)
                 (t is the current time unit, or cycle)
Comment:   Has the side effect of continuously refreshing a 256
              by 512 black-and-white screen (simulators must
              simulate this device). Each row in the physical
              screen is represented by 32 consecutive 16-bit words,
              starting at the top left corner of the screen. Thus
              the pixel at row r from the top and column c from the
              left ( $0 \leq r \leq 255$ ,  $0 \leq c \leq 511$ ) reflects the  $c \div 16$  bit
              (counting from LSB to MSB) of the word found at
              Screen[r*32+c/16].
    
```

Keyboard:

```

Chip Name: Keyboard    // Memory map of the physical keyboard.
                          // Outputs the code of the currently
                          // pressed key.
Output:    out[16]     // The ASCII code of the pressed key, or
                          // one of the special codes listed in
                          // figure 5.6, or 0 if no key is pressed.
Function: Outputs the code of the key presently pressed on the
              physical keyboard.
Comment:   This chip is continuously being refreshed from a
              physical keyboard unit (simulators must simulate this
              service).
    
```

Memory:



```

Chip Name: Memory      // Complete memory address space
Inputs:   in[16],      // What to write
              load,        // Write-enable bit
              address[15] // Where to write
Output:   out[16]      // Memory value at the given address
Function: 1. out(t)=Memory[address(t)](t)
              2. If load(t-1) then Memory[address(t-1)](t)=in(t-1)
                 (t is the current time unit, or cycle)
Comment:  Access to any address>24576 (0x6000) is invalid.
               Access to any address in the range 16384-24575
               (0x4000-0x5FFF) results in accessing the screen
               memory map. Access to address 24576 (0x6000) results
               in accessing the keyboard memory map. The behavior
               in these addresses is described in the Screen and
               Keyboard chip specifications.
  
```

Implementation:

- אם ביט 14 בכתובת כבוי הכתובת נמצאת ב-RAM.

- אם ביט 14 בכתובת דולק הכתובת נמצאת באחד ממכשירי ה-IO. במקרה זה:

- - - אם ביט 13 בכתובת כבוי הכתובת נמצאת במפת הזיכרון של המסך.

- - - אם ביט 13 בכתובת דולק הכתובת נמצאת במפת הזיכרון של המקלדת.

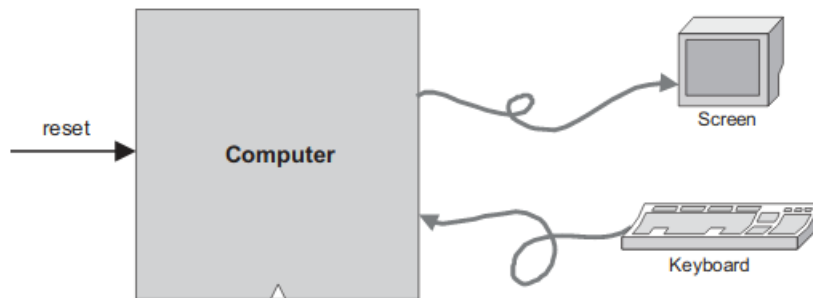
```

// choose the chip to send the load bit to
DMux(in=load, sel=address[14], a=loadRAM, b=loadScreen);
// get the outputs from all the chips
RAM16K(in=in, load=loadRAM, address=address[0..13], out=outRAM);
Screen(in=in, load=loadScreen, address=address[0..12], out=outScreen);
Keyboard(out=outKBD);
// choose the output
Mux16(a=outScreen, b=outKBD, sel=address[13], out=outIO);
Mux16(a=outRAM, b=outIO, sel=address[14], out=out);
  
```


Computer 5.5

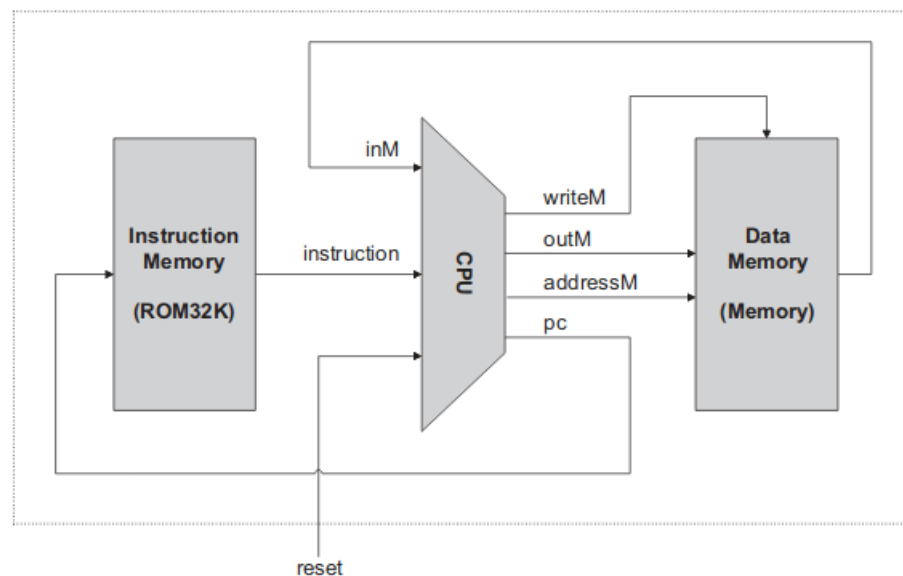
מכיל את כל החומרה של המחשב, הצ'יפ העליון.

API:



Chip Name: Computer // Topmost chip in the Hack platform
Input: reset
Function: When reset is 0, the program stored in the computer's ROM executes. When reset is 1, the execution of the program restarts. Thus, to start a program's execution, reset must be pushed "up" (1) and then "down" (0).
 From this point onward the user is at the mercy of the software. In particular, depending on the program's code, the screen may show some output and the user may be able to interact with the computer via the keyboard.

Implementation:



ממומש באמצעות שלושה צ'יפים: CPU, Memory, ROM32K.

Additional Functionality - Extended ALU & CPU 5.6

הרחבת ה-ALU וה-CPU כדי שיכללו גם כפל ופעולות shift.

ExtendALU 5.6.1

API:

input: x[16], y[16], instruction[9]
output: out[16], zr, ng
if inst[7..8] = 1 then output = ALU (with input x, y and inst[0..6])
else, if inst[7] = 0 then out = x * y
else (inst[8] = 0, inst[7] = 1), then the output will be shift:
- if inst[4] = 0, then out = shift y, else out = shift x.
- if inst[5] = 0, then out = shift right, else out = shift left.

Implementation:

מחשבים הכל באמצעות הצ'יפ ALU הרגיל, הצ'יפים של Mul ו-ShiftRight, ShiftLeft.
בוחרים את התוצאה הנכונה של shift לפי inst[4..5] באמצעות Mux4Way16.
בוחרים את התוצאה הנכונה בסה"כ לפי inst[7..8] באמצעות Mux4Way16.
בודקים אם אפס ואם שלילי כמו ב-ALU הרגיל.

CpuMul 5.6.2

API:

אותו דבר כמו CPU רגיל, עם תוספת של כפל ושיפט באמצעות C-instruction שתוארו ב-4.3.3.

Implementation:

בדיוק אותו דבר כמו ה-CPU הרגיל רק שמשתמשים ב-ExtendAlu במקום ה-ALU הרגיל.

חלק II

Software Hierarchy

המטרה: פיתוח קומפיילר ומערכת הפעלה בסיסית עבור שפת תכנות high-level מונחית עצמים (Jack).

הדרך:

6. Assembler: תרגום משפת מכונה בינארית לשפת מכונה סמלית - אסמבלי.
 - 7-8. Virtual Machine: בניית VM-Translator שמתרגם תוכניות משפת VM (Virtual Machine) לאסמבלי.
 9. Jack Specification: תיאור המפרט לשפת Jack, high-level.
 - 10-11. Compiler: בניית Jack Compiler שמתרגם תוכניות משפת Jack ל-VM.
 12. Operation System: בניית מערכת הפעלה עבור פלטפורמת ב-Hack, תיאור אלגוריתמים יעילים.
- הערה: המימושים בחלק הזה אינם בהכרח אופטימליים.

6 Assembler

6.1 Background

המטרה: לתרגם תכנית בשפת מכונה סמלית (אסמבלי) לשפת מכונה בינארית (נקרא לזה פשוט שפת מכונה). עושים את זה באמצעות כתיבת "מתרגם" - Assembler שיקבל קובץ שכתוב באסמבלי - סיומת asm, ויוציא קובץ שכתוב בשפת מכונה - סיומת hack.

הדרך: בעיקרון, שפת אסמבלי היא תרגום ישיר של שפת מכונה - לכל פקודה (למשל, A+D) יש מקבילה בינארית, כלומר הוראה אחת שמקבילה לפעולה בפקודה. לכן בקלות נוכל לעבור שורה שורה בקובץ הקלט ולתרגם כל שורה למקבילה הבינארית שלה, לפי המפרט בפרק 4 (4.3).

הבעיה היחידה שניתקל בה היא הסמלים - symbols. שפת אסמבלי מאפשרת שימוש בסמלים שונים בתור סימונים קבועים למקומות בזיכרון (כמו R0-R15, ARGS, ...), תוויות (labels) ומשתנים (variables). כל אחד מהסמלים האלו מקבל מקום קבוע בזיכרון שמייצג אותו: במקרה של הסמלים הקבועים והמשתנים - כתובת בזיכרון הרגיל, RAM. המשתנים מקבלים כתובת החלק מתא 16 בזיכרון והלאה. במקרה של תוויות - כתובת בזיכרון ההוראות, ROM, שמייצגת את המקום ברשימת ההוראות של התכנית בו הוצהרה התווית, ואליה צריך לחזור בכל פעם שקופצים לתווית הזו. בשפת מכונה אין משמעות לשמות, אלא רק לכתובות מפורשות בזיכרון. לכן, נצטרך להחליף את השמות האלו בכל מקום בכתובת המפורשת שלהם.

פתרון: נבנה טבלה שלכל שם של סמל (תווית או משתנה) מחזיקה את הכתובת המוקצית לו בזיכרון (ב-RAM או ב-ROM). בכל פעם שניתקל במשתנה חדש נוסיף שורה בטבלה עם שם המשתמש והכתובת המתאימה. עבור הסמלים הקבועים יש כתובות קבועות שנוכל להוסיף לטבלה בהתחלה:

Label	RAM address	(hexa)
SP	0	0x0000
LCL	1	0x0001
ARG	2	0x0002
THIS	3	0x0003
THAT	4	0x0004
R0-R15	0-15	0x0000-f
SCREEN	16384	0x4000
KBD	24576	0x6000

6.2 Implementation

התכנית Assembler מורכבת מארבעה מודולים (מחלקות): Assembler, Parser, Code, SymbolTable. ריצה של התכנית על קובץ תכלול שני מעברים. * יש API מפורט בספר בפרק 6.

6.2.1 המעברים:

מעבר ראשון: עוברים על הקובץ ובונים את טבלת הסמלים. ראשית נוסיף את כל הסמלים הקבועים. כעת, עוברים על הקובץ שורה שורה. מחזיקים קאונטר (מתחיל ב-0) שמחזיק את מספר השורה של הפקודה הנוכחית ב-ROM (כלומר כל שורה נעלה את הקאונטר ב-1, חוץ מבשורות של הערות או בשורות של הצהרה על תווית). בכל פעם שניתקל בהצהרה על תווית "(LabelName)" נבדוק מה מספר השורה בקאונטר. זו תהיה הכתובת של התווית ב-ROM. נוסיף את שם התווית לטבלה עם הכתובת הזו.

מעבר שני: עוברים שוב על הקובץ ומתרגמים כל שורה:

1. C-instruction: בכל פעם שנתקלים בהוראה `dest=comp;jump`, מתרגמים אותה לפי המפרט שבפרק 4 להוראה בינארית, וכותבים אותה בקובץ החדש.

2. A-instruction: בכל פעם שנתקלים בהוראה `@value`: אם `value` הוא מספר, מתרגמים אותו למספר בינארי עם 15 ספרות, מוסיפים 0 מימין (כדי לומר שזו פקודת A) ומוסיפים את ההוראה לקובץ. אחרת, `value` הוא משתנה (חדש או שהוגדר כבר) או שם של תווית שהוגדרה כבר. ראשית, נחפש את השם בטבלת הסמלים. אם מצאנו, נתייחס אל `value` לפי הכתובת שהוקצאה לסמל הזה בטבלה ונתרגם כרגיל. אחרת, השם הזה הוא משתנה חדש, ונוסיף אותו לטבלה, כאשר הכתובת שלו תהיה הכתובת של המשתנה האחרון שהוכנס לטבלה ועוד 1, ואז נתרגם כרגיל לפי הכתובת הזו.

6.2.2 המודולים:

SymbolTable: מחזיק את טבלת הסמלים שמחזיקה לכל סמל את הכתובת שלו. יכול לבדוק האם סמל מסויים נמצא בטבלה ויכול להוסיף תווית עם כתובת נתונה. בנוסף, עבור המעבר השני, מאתחל משתנה `next_address` שמחזיק את הכתובת של המשתנה האחרון שהוכנס לטבלה ועוד 1. המשתנה הזה מאותחל ל-16, כי אנחנו מתחילים להקצות זיכרון ב-RAM למשתנים החל מהתא ה-16. בכל פעם שנוסיף משתנה לטבלה, הוא יתווסף עם הכתובת שנמצאת כרגע ב `next_address`, ואז `next_address` יעלה באחד.

Code: מודול סטטי. מחזיק את הטבלאות שמפרטות את המשמעות של פקודות C שבפרק 4, ומסוגל לתרגם את כל אחד מהחלקים של פקודת C (`dest, comp, jump`) למקבילה הבינארית שלה.

Parser: קורא את קובץ הקלט (אסמבלי) שורה-שורה. בכל שלב מפרסר את השורה ומאפשר גישה נוחה לכל אחד מהמרכיבים של הפקודה הכתובה בה: איזו סוג של פקודה זו, `dest, comp, jump` במקרה של פקודת C, `value` במקרה של פקודת A ו-`LabelName` במקרה של הצהרה על תווית.

Assembler: המודול המרכזי, מריץ את התכנית: בונה טבלת סמלים, מנחה את הפרסר במעבר הראשון למצוא את כל ההצהרות על תוויות (תוך מעקב אחרי מספר השורה) ומוסיף אותן לטבלה. לאחר מכן מתחיל את המעבר השני: פותח את הקובץ החדש עם סיומת `hack`, מנחה את הפרסר לעבור שורה שורה על הקובץ `asm` ומתרגם את השורה באמצעות המודול `Code`.

Virtual Machine I: Stack Arithmetic 7

Background 7.1

באופן כללי, מכונה וירטואלית (VM) היא מחשב אבסטרקטי שלא קיים באמת, אבל ניתן לדמות את פעולתו על מחשב אמיתי. כלומר, בהינתן קוד שכתוב ב"שפת המכונה" של המכונה הוירטואלית, VM code, ניתן יהיה להריץ אותו על המחשב המקורי, כאשר ההתאמה בין השפה של הקוד לבין שפת המכונה של המחשב מתבצעת ע"י ה-VM. באופן זה ה-VM מאפשרת התאמה בין תוכניות חיצוניות למחשב בו היא נמצאת ובכך מאפשרת transportability: בעיקרון שפת ה-VM תהיה יותר פשוטה משפה high-level, לכן ניתן לתרגם אותה לשפות מכונה שונות של מחשבים שונים יחסית בקלות. לאחר מכן, כל תוכנה שנרצה להפעיל על המחשבים האלו נצטרך לתרגם רק פעם, לשפת VM. בפרק זה ובפרק הבא נגדיר את שפת ה-VM שנשתמש בה ונבנה מתרגם משפה זו לשפת המכונה שלנו, אסמבלי.

The Virtual Machine paradigm 7.1.1

המטרה הכללית שלנו היא לבנות קומפיילר משפת Jack, שפת high-level, לשפת המכונה של Hack (אסמבלי), כדי שנוכל להריץ תוכניות שכתובות בשפה הזו על המחשב שלנו.

בניית קומפיילר כזו נדרשת עבור רוב שפות התכנות שאנחנו מכירים, למשל C ו-JAVA. ניתן לבנות קומפיילר שיתרגם את תוכנית בשפה מסויימת ישירות לאסמבלי. הבעיה היא שלכל מחשב יכולה להיות שפת מכונה שונה (ובאמת יש המון שפות כאלו), לכן לכל זוג כזה של מחשב ושפת תכנות נצטרך לבנות קומפיילר ספציפי, מה שיוצר המון המון קומפיילרים שונים לכל שפת תכנות. בפרט, אם יש משהו מתעדכן בשפת התכנות, יש צורך להתאים את כל מגוון הקומפיילרים האלו. אחד הפתרונות לבעיה הזו (שמשמשים בו, למשל, ב-JAVA) הוא פירוק של תהליך הקומפילציה לשני חלקים. בחלק הראשון שפת התכנות מתורגמת לשפת ביניים מפשטת יחסית, אך לא מפשטת כמו שפת מכונה (משהו בין high-level ל-low-level), ובשלב השני שפת הביניים מתורגמת לשפת המכונה של המחשב הספציפי.

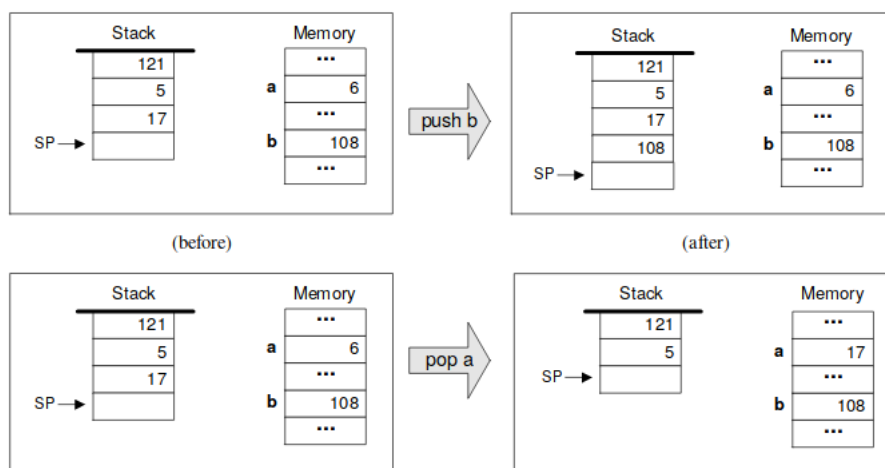
The Stack Machine Model 7.1.2

שפת ה-VM מורכבת מפעולות אריתמטיות ולוגיות, פעולות גישה לזיכרון, קריאות לפונקציות ופעולות program flow (לולאות, if-else, ...). השאלה המרכזית היא איך ואיפה לאחסן את האופרנדים ואת התוצאות של הפעולות האלו. אנחנו נשתמש במודל של stack - מחסנית, שמממשת את עיקרון ה-LIFO (last in first out). באותו רעיון משתמשים גם במכונה הוירטואלית של JAVA.

פעולות בסיסיות: מחסנית היא מבנה נתונים אבסטרקטי שתומך בשתי פעולות בסיסיות: push ו-pop: push מוסיף איבר לראש המחסנית, והאיבר שהיא קודם בראש נדחף מתחתיו. pop מוציא את האיבר שנמצא כרגע בראש המחסנית, והאיבר שהיה מתחתיו נמצא כעת בראש.

ישנן דרכים רבות לממש מחסנית. הפשוטה מכולן היא באמצעות מערך, שנקרא לו stack, ופויינטר, שנקרא לו stack pointer או sp בקצרה. הפויינטר הזה מצביע תמיד למקום במערך שנמצא מעל האיבר העליון במחסנית. הפעולה push x תתבצע באמצעות: $stack[sp]=x$, $sp=sp+1$, והפעולה $pop\ x$ תתבצע באמצעות: $sp=sp-1$, $return\ stack[sp]$.

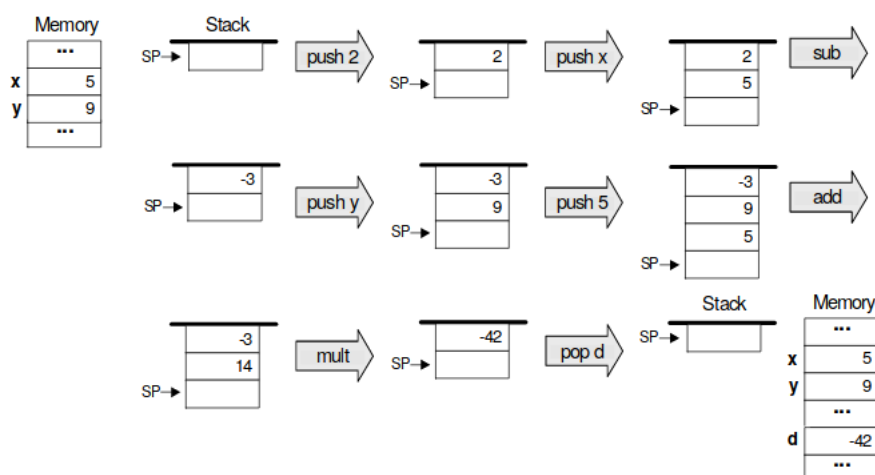
דוגמא:



פעולות אריתמטיות ולוגיות: התהליך פשוט: האופרנדים מוצאים מהמחסנית (באמצעות pop), מבוצעת עליהם הפעולה הדרושה, והתוצאה מוחזרת לראש המחסנית (באמצעות push).
דוגמאות:



```
// d=(2-x)*(y+5)
push 2
push x
sub
push y
push 5
add
mult
pop d
```



VM Specification I 7.2

General 7.2.1

תכנית VM בנויה ממספר קבצים עם סיומת .vm. בכל קובץ מספר פונקציות, כל אחת היא יחידה נפרדת שמכילה קוד בשפת VM. השפה בנויה מ- data type אחד באורך 16 ביט שיכול להתפרש כ- int, בוליאני או פוינטר. יש 4 סוגי פקודות:

1. פקודות אריתמטיות שמבצעות פעולות אריתמטיות או לוגיות על המחסנית.
2. פקודות גישה לזיכרון שמעבירות מידע בין המחסנית ומקטעי הזיכרון (memory segments).
3. פקודות program flow שמבצעות מעברים וקפיצות עם או בלי תנאים (loops, if-else,...).
4. פקודות קריאה לפונקציות שקוראות לפונקציות ומחזירות את התוכנית בחזרה לשורה הבאה.

כל פקודה בקוד VM מופיעה בשורה נפרדת, באחד מהפורמטים הבאים:

command (e.g. add)

command arg (e.g. goto loop)

command arg1 arg2 (e.g. push local 3)

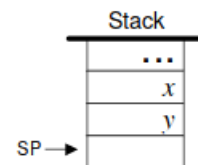
וניתן להוסיף שורות ריקות או לכתוב הערות, אחרי הסימן '\\', שמהן המתרגם מתעלם.

הבנייה של המתרגם משפת VM לאסמבלי היא מורכבת, לכן מחולקת לשני חלקים: בפרק זה מפורט המבנה של פקודות אריתמטיות ופקודות גישה לזיכרון, ובונים מתרגם שלהן לאסמבלי, ובפרק הבא משלימים את הסינטקס והמתרגם לכלול גם את שתי סוגי הפקודות האחרונות.

Arithmetic and Logical Commands 7.2.2

ישנן 9 פקודות אריתמטיות ולוגיות, כולן מבוססות על המחסנית. שבע מהן הן בינאריות: הן מוציאות (pop) מהמחסנית את שני הערכים האחרונים, מחשבת פונקציה בינארית עליהם ומחזירה (push) את התוצאה למחסנית. שתי הפקודות הנותרות הן אונאריות: הן מוציאות (pop) מהמחסנית את הערך האחרון, מחשבת פונקציה אונארית עליו ומחזירה (push) את התוצאה למחסנית.

Command	Return value (after popping the operand/s)	Comment
add	$x + y$	Integer addition (2's complement)
sub	$x - y$	Integer subtraction (2's complement)
neg	$-y$	Arithmetic negation (2's complement)
eq	true if $x = y$ and false otherwise	Equality
gt	true if $x > y$ and false otherwise	Greater than
lt	true if $x < y$ and false otherwise	Less than
and	$x \text{ And } y$	Bit-wise
or	$x \text{ Or } y$	Bit-wise
not	Not y	Bit-wise



שלוש מהפעולות האלו (eq, lt, gt) מחזירות ערך בוליאני. בשפת VM, true מתורגם לערך 1- (שבשיטה שלנו מיוצג כ- 16 אחדות בייצוג בינארי), ו- false מתורגם לערך 0.

הפעולות `push x` ו-`pop x` דורשות מיקום כלשהו בזיכרון הגלובאלי (מחוץ למחסנית) ממנו מגיע ואליו הולך `x`. שפת VM משתמשת בשמונה מקטעי זיכרון (memory segments) שונים:

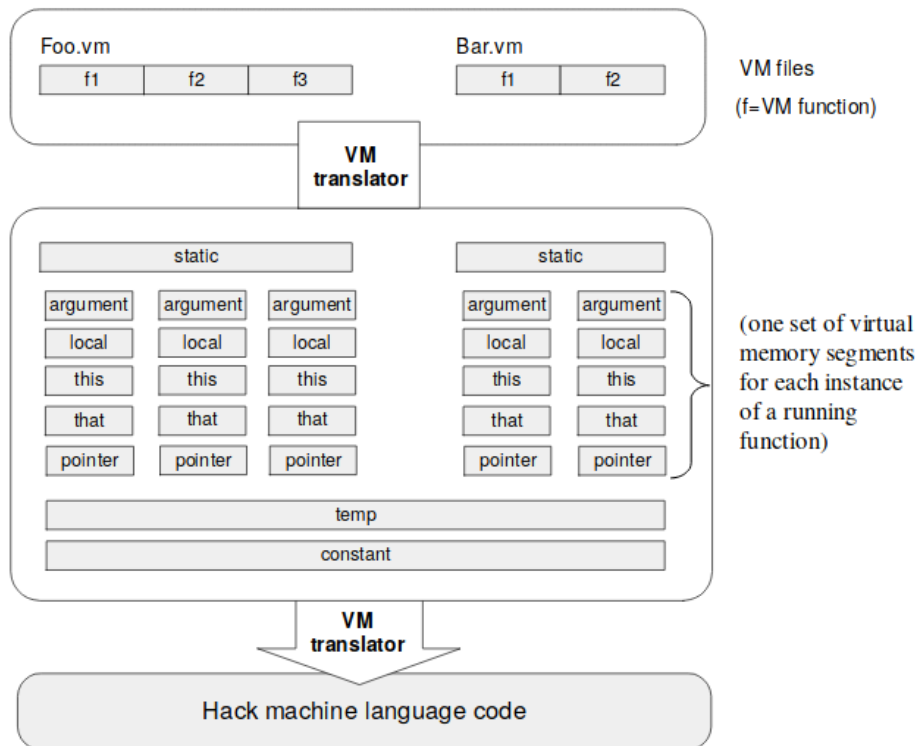
Segment	Purpose	Comments
argument	Stores the function's arguments.	Allocated dynamically by the VM implementation when the function is entered.
local	Stores the function's local variables.	Allocated dynamically by the VM implementation and initialized to 0 when the function is entered.
static	Stores static variables shared by all functions in the same .vm file.	Allocated by the VM implementation for each .vm file; shared by all functions in the .vm file.
constant	Pseudo-segment that holds all the constants in the range 0 ... 32767.	Emulated by the VM implementation; Seen by all the functions in the program.
this that	General-purpose segments. Can be made to correspond to different areas in the heap. Serve various programming needs.	Any VM function can use these segments to manipulate selected areas on the heap.
pointer	A two-entry segment that holds the base addresses of the <code>this</code> and <code>that</code> segments.	Any VM function can set <code>Pointer 0</code> (or <code>1</code>) to some address; this has the effect of aligning the <code>this</code> (or <code>that</code>) segment to the area on the heap beginning in that address.
temp	Fixed eight-entry segment that holds temporary variables for general use.	May be used by any VM function for any purpose. Shared by all functions in the program.

ישנן שתי פעולות גישה לזיכרון:

- `push segment index` Push the value of `segment[index]` onto the stack;
- `pop segment index` Pop the topmost stack item and store its value in `segment[index]`.

כאשר `segment` הוא אחד מהמקטעים בטבלה, ו-`index` הוא מספר התא (אי שלילי) בתוך המקטע שאליו נרצה לגשת (החל מ-0).

את היחסים בין קבצי vm, פונקציות vm ומקטעי הזיכרון של כל אחת מהן ניתן לייצג באופן הבא:



בנוסף לשמונה מקטעי הזיכרון שניתן לגשת אליהם במפורש ע"י פקודות VM, ה- VM מממש שני מבני נתונים באופן לא מפורש, stack ו-heap (מחסנית וערימה). לא מתייחסים אליהם מפורשות, אבל המצב שלהם משתנה ברגע בעקבות פקודות VM שונות. המחסנית - Stack: מרחב בזיכרון שמממש את מבנה הנתונים של המחסנית שתואר מקודם (בצורת מערך ופוינטר). הערימה - Heap: מרחב נוסף בזיכרון שמוקדש לאיחסון אובייקטים ומערכים.

7.2.4 Program Flow and Function Calling Commands

ישנן 6 פקודות נוספות שעליהן נפרט בפרק הבא:

Program Flow Commands

- `label symbol` // Label declaration
- `goto symbol` // Unconditional branching
- `if-goto symbol` // Conditional branching

Function Calling Commands

- `function functionName nLocals` // Function declaration; must specify the
// number of the function's local variables
- `call functionName nArgs` // Function invocation; must specify the
// number of the function's arguments
- `return` // Transfer control back to the calling function

(In the above, *functionName* is a symbol and *nLocals* and *nArgs* are non-negative integers.)

7.3 Standard VM Mapping on the Hack Platform

כעת נרצה לממש את הרעיון האבסטרקטי של מכונה וירטואלית במחשב שלנו. לשם כך, אנחנו צריכים למצוא דרך לדמות את העולם של המכונה הוירטואלית באמצעות המחשב שלנו, כלומר להקצות זיכרון פיזי ב-RAM למבני הנתונים של ה-VM (מחסנית וערימה). באופן כללי ניתן לממש את זה בכל דרך שרוצים, אבל כאן נתאר מיפוי סטנדרטי (standard mapping) של האלמנטים והפעולות של ה-VM במחשב שלנו.

מיפוי ל - RAM: הזיכרון של מחשב ה-Hack מורכב מ- K32 רגיסטרים של 16 ביט. ה- 16K הראשונים הם ה-RAM הכללי, וה- 16K הבאים מוקצים למפות הזיכרון של המכשירים הפרפריאליים, כלומר מכשירי הקלט-פלט. ה-VM משתמש בזיכרון זה באופן הבא:

<i>RAM addresses</i>	<i>Usage</i>
0–15	Sixteen virtual registers, whose usage is described below
16–255	Static variables (of all the VM functions in the VM program)
256–2047	Stack
2048–16483	Heap (used to store objects and arrays)
16384–24575	Memory mapped I/O

נזכור כי לפי המפרט של אסמבלי, ניתן להשתמש בסמלים R0 עד R15 כהתייחסות לכתובות RAM 0 עד 15 בהתאמה, ובנוסף כי ניתן להשתמש בסמלים SP, LCL, ARG, THIS, THAT כהתייחסות לכתובות 0,1,2,3,4 בהתאמה. העניין הזה תוכנן כדי לאפשר מימוש נוח של ה-VM, ונשתמש בו כאן באופן הבא:

<i>Register</i>	<i>Name</i>	<i>Usage</i>
RAM[0]	SP	Stack pointer: points to the next topmost location in the stack;
RAM[1]	LCL	Points to the base of the current VM function's local segment;
RAM[2]	ARG	Points to the base of the current VM function's argument segment;
RAM[3]	THIS	Points to the base of the current this segment (within the heap);
RAM[4]	THAT	Points to the base of the current that segment (within the heap);
RAM[5-12]		Holds the contents of the temp segment;
RAM[13-15]		Can be used by the VM implementation as general-purpose registers.

מיפוי מקטעי הזיכרון:

local, argument, this, that: כל אחד מהמקטעים האלו ממופה ישירות ל-RAM, וכתובת הבסיס הפיזית שלהם שם נשמרת ברגיסטר המתאים להם (LCL, ARG, THIS, THAT בהתאמה). כך גישה לתא ה- i באחד מהמקטעים האלו מתורגמת לגישה לכתובת $base+i$ ב-RAM, כאשר $base$ היא כתובת הבסיס של המקטע. למשל, במקרה של local, נרצה לגשת ל- $RAM[RAM[LCL]+i]$.

pointer, temp: כל אחד מהמקטעים האלו ממופה ישירות לאזור קבוע ב-RAM. pointer ממופה לכתובות 3 ו-4 (THIS, THAT) ו-temp ממופה לכתובות 5-12 (R5-R12). כך גישה ל- i pointer מתורגמת לגישה לכתובת $3+i$ ב-RAM, כלומר $RAM[3+i]$, וגישה ל- i temp מתורגמת לגישה לכתובת $5+i$ ב-RAM, כלומר $RAM[5+i]$.

constant: המקטע הזה הוא באמת וירטואלי, במובן שאין מיקום פיזי בזיכרון ה-RAM שמחזיק אותו. במקום זאת, גישה ל- `constant i` מתורגמת פשוט למספר `i`.

static: כאשר תירגמנו מאסמבלי לשפת מכונה בינארית, כל משתנה שהוגדר ע"י המשתמש קיבל מקום בזיכרון, החל מכתובת 16. באופן זה נוכל להקצות מקום למשתנים הסטטיים: משתנה סטטי מספר `j` בקובץ `vm` ששמו `f` יתורגם לסמל אסמבלי `f.j`, והאסמבלר יקצה לו מקום בזיכרון החל מכתובת 16, כלומר בתוך המקטע שהוקצה למשתנים סטטיים, כנדרש. לסיכום:

<i>Symbol</i>	<i>Usage</i>
SP, LCL, ARG, THIS, THAT	These predefined symbols point, respectively, to the stack top and to the base addresses of the virtual segments <code>local</code> , <code>argument</code> , <code>this</code> , and <code>that</code> .
R13-R15	These predefined symbols can be used for any purpose.
<code>Xxx.j</code> symbols	Each static variable <code>j</code> in file <code>Xxx.vm</code> is translated into the assembly symbol <code>Xxx.j</code> . In the subsequent assembly process, these symbolic variables will be allocated RAM space by the Hack assembler.
Flow of control symbols	The implementation of the VM commands <code>function</code> , <code>call</code> , and <code>label</code> involves generating special label symbols, to be discussed in chapter 8.

7.4 Implementation

התכנית `VMtranslator` מורכבת משלושה מודולים (מחלקות): `VMtranslator`, `Parser`, `Code`. * יש API מפורט בספר בפרק 7.

Code: מתרגם פקודות אריתמטיות וגישה לזיכרון (`push`, `pop`). פירוק בהמשך (7.5).

Parser: קורא את קובץ הקלט (`vm`) שורה-שורה. בכל שלב מפרסר את השורה ומאפשר גישה נוחה לכל אחד מהמרכיבים של הפקודה הכתובה בה: פקודה אריתמטית, פקודת `push` או פקודת `pop`, וגישה לארגומנטים 1 ו-2 של הפקודה (אם יש, כרגע רק במקרה של `push` ו-`pop`).

VMtranslator: המודול המרכזי, מריץ את התכנית: פותח את הקובץ החדש עם סיומת `asm`, מנחה את הפרסר לעבור שורה שורה על כל אחד מקבצי ה-`vm` אחד אחרי השני ומתרגם את השורה באמצעות המודול `Code`.

Code module 7.5

פקודות אריתמטיות:

7.5.1 הוצאת אופרנד מהמחסנית לרגיסטר M

נסמן בהמשך פקודה זו כ- Action 1.

```
// SP- -
@SP
M=M-1
// => A=SP, M=RAM[SP]-1= the address of first element in the stack
A=M
// => A=RAM[SP]-1, M=RAM[RAM[SP]-1]= the first element in the stack
```

7.5.2 פקודה אריתמטית אינארית: neg, not

```
Action 1 // extracts x from the stack
// M now contain x so A still points to the one-after-the-head of the stack, so M=RAM[SP]
// do the command, the result now stored in the one-after-the-head in the stack
neg: M=-M
not: M=!M
// increment SP
@SP
M=M+1
```

7.5.3 פקודה אריתמטית בינארית: add, sub, and, or

```
Action 1 // extracts y from the stack and keep it in D
D=M
Action 1 // extracts x from the stack
// now M=x, D=y
// M now contain x so A still points to the one-after-the-head of the stack, so M=RAM[SP]
// do the command, the result will be stored in the one-after-the-head in the stack
add: M=M+D
sub: M=M-D
and: M=M&D
or: M=M|D
// increment SP
@SP
M=M+1
```

7.5.4 פקודת השוואה: gt, lt, eq

בעיקרון מה שעושים זה להוציא את x ו- y מהמחסנית, לחסר את x מ- y ואז לבדוק מה היחס של זה ל-0, ולפי סוג הפקודה להכניס true (-1) או false (0) למחסנית.

זה פחות או יותר מה שעושים, חוץ מזה שצריך לדאוג לדבר אחד: אנחנו מניחים שהמספרים שמוכנסים לנו הם תקינים, אבל מה קורה אם יש לנו y מספר חיובי מאוד גדול ו- x מספר שלילי מאוד קטן? מה שיכול לקרות זה שכאשר נעשה $y-x$ יתקבל overflow (כאשר נריץ את תוכנת האסמבלי המתורגמת) ואז כאשר נשווה את זה לאפס נוכל לקבל תוצאה שגויה.

כדי למנוע את זה נצטרך לעשות כמה בדיקות: ראשית, נשים לב שזה יכול לקרות רק כאשר $x < 0 < y$ או כאשר $y < 0 < x$. את שני המקרים האלו נוכל לבדוק ישירות ולהחזיר תשובה מיידית לשאלת ההשוואה, מבלי לחסר.

ניתן כאן את התרגום בלי הבדיקות של ה-overflow, אך יש צורך להשלים אותן כדי שהתרגום יהיה נכון. בתרגום זה נשתמש בתוויות: נשמור משתנה, conditionNum, שיספור את מספר ההשוואות שנעשו עד עכשיו, ובכל השוואה נעשה תווית שמסמלת את סוף התווית - END_CONDITION + conditionNum. כך נוכל ראשית לחשב את $y-x$, לשים במחסנית true=-1, לבדוק האם ההשוואה נכונה, אם כן לקפוץ לסוף ההשוואה באמצעות התווית, ואם לא, לשנות את הערך ששמנו במחסנית ל- false=0 ולסיים.

```

Action 1 // extracts y from the stack and keep it in D
D=M
Action 1 // extract x from the stack
// now M=x, D=y
D=D-M // D=y-x
// M now contain x so A still points to the one-after-the-head of the stack, so M=RAM[SP]
// push true=-1 onto the stack.
M=-1
// jump to a label at the end if the comparison is true:
@END_CONDITION + conditionNum
// write the command
eq: D;JEQ
lt: D;JGT // x < y iff D=y-x>0
gt: D;JLT // x > y iff D=y-x<0
// if false - set M=RAM[SP] to false=0
M=0
// end the condition
(END_CONDITION + conditionNum)
// the result now stored in the one-after-the-head in the stack
// increment SP
@SP
M=M+1

```

פקודות push ו-pop:

7.5.5 מציאת הכתובת

נסמן פעולה זו בהמשך כ- Action 2.

תרגום הפקודה הזו תלוי במקטע הרצוי, ונעשה לפי הפירוט של המיפוי הסטנדרטי ב-7.3. נמצא את הכתובת ונאחסן אותה ב-D.

7.5.6 החלפת ערכים בין שני מקומות בזיכרון

7.5.7 פקודת pop

נמצא את הכתובת אליה נרצה להוציא את x ונאחסן אותה ב- D באמצעות Action 2. נאחסן אותה בכתובת ה-RAM R13 (סמל קבוע שמסמל את הכתובת 13 בזיכרון. הכתובות 13-15 משמשות למימוש המתרגם). נוציא את x מהמחסנית ונכניס אותו למקום בזיכרון שכתובתו שמורה כעת ב- R13.

Action 2 // find the right address and store it in D: D=address

// store the address in RAM[13]

@R13

M=D

Action 1 // extract x from the stack

D=M // RAM[SP] = RAM[SP]-1, D=RAM[RAM[SP]]=x

@R13

A=M // M=RAM[RAM[R13]]=RAM[address]

M=D // RAM[address]=D=x

7.5.8 פקודת push

נמצא את הכתובת ממנה נרצה להעתיק את x ונאחסן אותה ב- R13 באמצעות Action 2. נעתיק את x ל-R13, נכניס אותו לראש המחסנית ונעלה את SP ב-1.

Action 2 // find the right address and store it in D: D=address

// copy x from this address

A=D // A=address

D=M // D=M-RAM[A]=RAM[address]=x

// store x in the top of the stack

@SP

A=M // M=RAM[RAM[SP]]

// now

M=D // RAM[RAM[SP]]=D=x

// increment SP

@SP

M=M+1

Virtual Machine II: Program Control 8

Background 8.1

בפרק זה נשלים את המפרט של שפת ה-VM ואת בניית המתרגם: קריאות לפונקציות ו-Program flow. פעולות אלו מאפשרות לנו לשנות את סדר המעבר על ההוראות בקוד, עם קפיצות, לולאות ואזורי קוד מוגדרים שמבצעים פעולה מסויימת (פונקציות), וניתן להשתמש בהם כפעולה high-level. כאשר פונקציה מסויימת (the caller) קוראת לפונקציה אחרת (the called subroutine), יש צורך לטפל במספר פרטים:

- העברת הארגומנטים מהפונקציה הקוראת לפונקציה הנקראת.
- שמירת המצב של הפונקציה הקוראת לפני הקריאה.
- הקצאת זיכרון למשתנים הלוקאליים של הפונקציה הנקראת.
- קפיצה לקוד של הפונקציה הנקראת.
- החזרת ערכי החזרה מהפונקציה הנקראת לפונקציה הקוראת.
- מתזור הזיכרון שהוקצה לפונקציה הנקראת.
- החזרת ה"עולם" של הפונקציה הקוראת למצב שהיה לפני הקריאה.
- קפיצה לקוד שבא מיד לאחר הקריאה לפונקציה הנקראת.

בכל הפרטים האלו יש צורך לטפל במהלך מימוש ה-low level שבו אנחנו עוסקים עכשיו. אנחנו מממשים את זה באמצעות המחסנית.

VM Specification II 8.2

Program Flow Commands 8.2.1

הפעולה הדיפולטית של תוכנית היא לעבור שורה שורה על ההוראות ולבצע אותן. לעיתים נרצה לשנות את הסדר הזה ולקפוץ למקום אחר בקוד (עם קיום תנאי כלשהו או בכל מקרה).
ב-VM הקפיצות האלו ממומשות באמצעות הפקודות הבאות:

<code>label c</code>	This command labels the current location in the function's code. Only labeled locations can be jumped to from other parts of the program. The label <code>c</code> is an arbitrary string composed of letters, numbers, and the special characters “_”, “:”, and “.”. The scope of the label is the current function.
<code>goto c</code>	This command effects a "goto" operation, causing execution to continue from the location marked by the <code>c</code> label. The jump destination must be located in the same function.
<code>if-goto c</code>	This command effects a "conditional goto" operation. The stack's topmost value is popped; if the value is not zero, execution continues from the location marked by the <code>c</code> label; otherwise, execution continues from the next command in the program. The jump destination must be located in the same function.

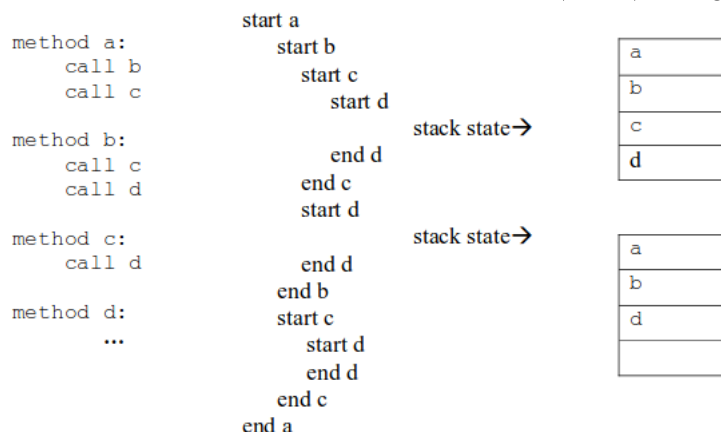
המימוש כאן די פשוט ומתבסס על שימוש בתוויות של אסמבלי, כאשר הפקודה `label labelName` בפונקציה `func` תייצר תווית `.func$labelName`.
דוגמאות:

<i>High-level source code</i>	<i>Compiled low-level pseudo code</i>
<pre>if (cond) s1 else s2 ...</pre>	<pre>code for computing cond if-false-goto L1 code for executing s1 goto L2 label L1 code for executing s2 label L2 ...</pre>
<pre>while (cond) s1 ...</pre>	<pre>label L1 code for computing cond if-false-goto L2 code for executing s1 goto L1 label L2 ...</pre>

Function Calling Commands 8.2.2

פונקציות ב-VM משמשות להרחבת הפעולות ה-build-in של השפה. הפעולה שלהן היא זהה: הקורא לפעולה או לפונקציה צריך להכניס למחסנית את הארגומנטים לפעולה, הן שתיהן מוציאות את הארגומנטים האלו מהמחסנית ומכניסות אליה את התוצאה של הפעולה שלהן. השוני היחיד הוא שפונקציות נקראות בעזרת `call funcName`, בניגוד לפעולה ה-build-in שנקראת פשוט ע"י `actionName`.

פונקציות יכולות להשתמש במשתנים לוקאליים משלהן, שחיים בסקופ מתחילת הפונקציה עד פקודת return, ולאחר מכן המקום ששמר אותם ממוחזר. בנוסף, השפה תומכת בקריאות מקוננות לפונקציות, ובקריאות רקורסיביות. מה שמאפשר לנו להשתלט על כל הפעולות הדרושות לקריאה לפונקציות זה העובדה שהמבנה של הקריאות הוא היררכי: ניתן לבצע קריאות מקוננות או רקורסיביות, אבל בכל שלב רק פונקציה אחת רצה, וכל הפונקציות לפני מחכות שתסיים. זהו תהליך בצורת LIFO, last-in-first-out, שמאפיין גם מחסנית, לכן ניתן יהיה לנצל את פעולת המחסנית לביצוע הפעולה הזו: כאשר פונקציה g קוראת לפונקציה f, נשמור (push) את ה"עולם" של g למחסנית, ונעבור להריץ את f. כאשר נסיים את הריצה של f, נחזיר (pop) את ה"עולם" של g ונמשיך להריץ אותה.



מכאן נשתמש במונח frame-מסגרת, כדי להתייחס ל"עולם" של פונקציה, כלומר למשתנים הלוקאליים שלה, הארגומנטים שלה וסביבת העבודה שלה. בנוסף, בפרק 7 התייחסנו למונח stack - מחסנית כזיכרון הנוכחי שמשמש בפונקציה, ועליו ניתן היה לבצע פעולות push, pop, ... מעתה נתייחס למושג stack - מחסנית כמחסנית הגלובאלית: איזור הזיכרון שמכיל את המסגרות של הפונקציה הנוכחית ושל הפונקציות שמעלה שרשרת הקריאות, ולזיכרון העובד של הפונקציה הנוכחית נקרא working stack. המונחים האלו קשורים, במובן שה-working stack של הפונקציה הנוכחית תמיד נמצא בהתחלת המחסנית הגלובאלית.

מימוש של קריאת לפונקציה f מפונקציה g יכיל את הפעולות הבאות:

- שמירת המסגרת של g במחסנית הגלובאלית.
- הקצאת מקום במחסנית למשתנים הלוקאליים של f.
- קפיצה לקוד של f.
- הרצת הקוד של f.
- חזרה לשורה הבאה בקוד של g.

הקפיצה לקוד של f תמומש באמצעות תווית: הקוד של f מתחיל בהגדרה: function f numLocals, וצריך פשוט לתרגם את זה לתווית (f), ואז נוכל להגיע לשם ע"י הפעולה @f.

כאשר אנחנו נתקלים בקריאה לפונקציה f בתוך פונקציה g (call f numArgs), אנחנו יודעים שלאחר הרצת f (לאחר פקודת ה return), נרצה לחזור לשורה בפונקציה g שאחרי שורת הקריאה לפונקציה. נוכל לסמן תווית במקום של הכתובת הזו, לשמור את שם התווית כחלק מהמסגרת של g, ואז הפעולה של החזרה לשורה הבאה בקוד של g (return) ב-f תהיה להוציא את שם התווית מהמחסנית (יחד עם שאר המסגרת) ולקפוץ אליה.

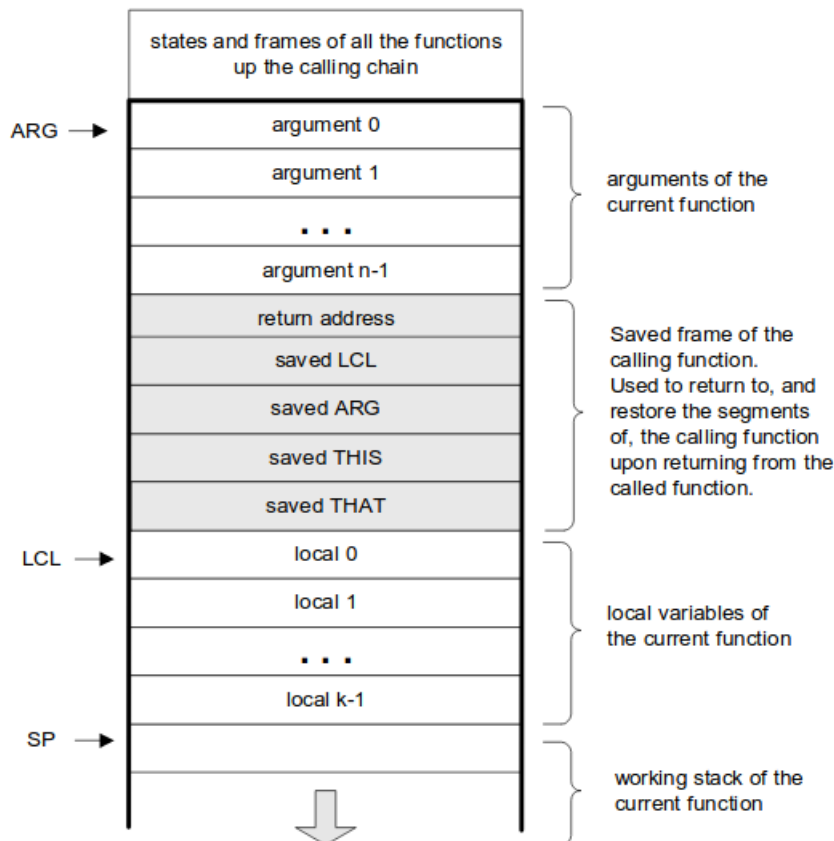
פונקציות ב-vm משתמשות בפקודות הבאות:

function f n	Here starts the code of a function named f, which has n local variables;
call f m	Call function f, stating that m arguments have already been pushed onto the stack;
return	Return to the calling function.

Implementation 8.3

The Global Stack & Function Calling Commands 8.3.1

הזיכרון בתכנית vm מנוהל ע"י מחסנית גלובאלית. בכל פעם שפונקציה נקראת, מוסיפים בלוק חדש למחסנית הגלובאלית-ית, שמכיל את ערכי הארגומנטים (פרמטרים) של הפונקציה הנקראת, קבוצת פויינטרים ששומרים את המצב של הפונקציה הקוראת לפני הקריאה (כולל כתובת חזרה), מקטע עבור המשתנים הלוקאליים של הפונקציה הנקראת (מאותחלים ל-0) ומחסנית עבודה ריקה עבור הפונקציה הנקראת.



מבחינת פלטפורמת ה-Hack, הדבר הזה ממומש כמו בפרק 7: בנוסף לכל הפעולות של push, pop וכו' שעובדים על מחסנית העבודה של פונקציה מסויימת, קריאת call "תעמיס" את המחסנית במשתנים המתאימים (כמתואר בטבלה) ואז התכנית תקפוץ לקוד של הפונקציה הנקראת. הפרוטוקול מתואר בפסאודו-קוד בטבלה הבאה, וכל פעולה בקוד הזה ניתן לתרגם לאסמבלי בצורה פשוטה (באותו אופן כמו בפרק 7):

<i>VM command</i>	<i>VM-on-Hack Implementation action (pseudo code)</i>
<u>Calling a function:</u> call f n	push return-address // (using label below) push LCL // save LCL of calling function push ARG // save ARG of calling function push THIS // save THIS of calling function push THAT // save THAT of calling function ARG = SP-n-5 // reposition ARG (n=number of args) LCL = SP // reposition LCL goto f // transfer control (return-address) // label for the return address
<u>Function declaration:</u> function f k	(f) // declare label for function entry repeat k times: // k=number of local variables PUSH 0 // initialize all of them to 0
<u>Returning from a function:</u> return	FRAME=LCL // FRAME is a temporary variable RET=*(FRAME-5) // save return address in a temp. var *ARG=pop() // reposition return value for caller SP=ARG+1 // restore SP for caller THAT=*(FRAME-1) // restore THAT of calling function THIS=*(FRAME-2) // restore THIS of calling function ARG=*(FRAME-3) // restore ARG of calling function LCL=*(FRAME-4) // Restore LCL of calling function goto RET // GOTO the return-address

כעת נוכל גם להרחיב את טבלת הסמלים שלנו כדי שתכיל גם את הסמלים הסטנדרטיים לתוויות, הצהרות על פונקציות וכתובות החזרה:

<i>Symbol</i>	<i>Usage</i>
“functionName:label” symbols	Each “label <i>b</i> ” command in a function <i>f</i> should generate a globally unique symbol <i>f:b</i> where <i>f</i> is the function name and <i>b</i> is the label symbol within the function’s code. When translating “goto <i>b</i> ” and “if-goto <i>b</i> ” commands into the target language, the full label specification <i>f:b</i> should be used instead of <i>b</i> .
“functionName” labels	Each function <i>f</i> should generate a symbol <i>f</i> that refers to its entry point in the instruction memory of the target architecture.
return address symbols	Each function call should generate a unique symbol that serves as a return address, i.e. the location of the command following the call command in the instruction memory of the target architecture.

Initialization 8.3.2

כאשר מריצים את המתרגם (VMtranslator) על תכנית VM (קובץ או תיקייה עם מספר קבצי vm) הוא מוציא קובץ פלט asm יחיד שמכיל את כל התכנית. הקובץ הזה חייב לאתחל את הזיכרון RAM לפי המיפוי הסטנדרטי שהגדרנו בפרק 7, כדי שהתכנית תתחיל לרוץ כנדרש. הוא עושה זאת בעזרת שתי פקודות: ראשית אתחול SP להיות 256, ולאחר מכן קריאה לפונקציה Sys.init. זוהי פונקציה של מערכת ההפעלה (שנבנה בפרק 12) שקוראת לפונקציה main (בתוך התכנית, כלומר חייבת להיות כזו) שאחראית להרצת התכנית, ואז נכנסת ללולאה אינסופית (בסיום הרצת main). כלומר, השורות הראשונות בכל קובץ פלט יהיו:

- SP=256
- call Sys.init

מתורגמות לקוד אסמבלי (באותו אופן כמו פקודת השמה ו-call אחרות).

Implementation API 8.3.3

בחלק זה נרחיב את התכנית VMtranslator מפרק 7. היא עדיין מורכבת משלושה מודולים (מחלקות): VMtranslator, Parser, Code.

* יש API מפורט בספר בפרק 8. התוספות מתרחשות בשני המודולים הראשונים:

Code: מתרגם פקודות אריתמטיות וגישה לזיכרון (push, pop), וכן פקודות program-flow וטיפול בפונקציות.

Parser: קורא את קובץ הקלט (vm) שורה-שורה. בכל שלב מפרסר את השורה ומאפשר גישה נוחה לכל אחד מהמרכיבים של הפקודה הכתובה בה: פקודה אריתמטית, פקודת push או פקודת pop, פקודות program-flow, פקודות קריאה לפונקציה וגישה לארגומנטים 1 ו-2 של הפקודה.

High-Level Language (Jack) 9

Background 9.1

בפרק זה נכיר את המפרט של שפת התכנות Jack, שעבורה נכתוב בפרקים 10 ו-11 קומפיילר. זוהי שפת high-level מבוססת עצמים (דומה ל-JAVA) עם סינטקס יחסית פשוט ובלי תמיכה בירושה. תכנית ב-Jack מורכבת ממחלקות - classes, לכל מחלקה מוקדש קובץ עם אותו שם ועם סיומת jack. בתכנית חייבת להיות מחלקה Main עם פונקציה main בתוכה, שאחראית להרצת התכנית. פעולות על אובייקטים נקראות מתודות - methods, ופעולות ברמת המחלקה (סטטיות) נקראות פונקציות - functions. דוגמא - מחלקת Fraction:

```
/** A Fraction (partial implementation) */
class Fraction {

    field int numerator, denominator;

    /** Construct a new (reduced) fraction given a numerator
    and denominator */
    constructor Fraction new(int a, int b){
        let numerator = a;
        let denominator = b;
        do reduce(); // if a/b is not reduced, reduce it.
        return this;
    }

    /** reduce this fraction - internal method (from the outside,
    a fraction always seems reduced.) */
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {
            let numerator = numerator / g;
            let denominator = denominator / g;
        }
        return;
    }

    /** compute the gcd of a and b - internal service function */
    function int gcd(int a, int b){
        var int r;
        // apply Euclid's algorithm
        while (~ (b = 0)) {
            let r = a - (b * (a / b)); // r = remainder of division a/b
            let a = b;
            let b = r;
        }
        return a;
    }

    method int getNumerator(){
        return numerator;
    }

    method int getDenominator(){
        return denominator;
    }

    /** Return the sum of this fraction and the other one */
    method Fraction plus(Fraction other){
        var int sum;
        let sum = (numerator * other.getDenominator()
            + (other.getNumerator() * denominator));
        return Fraction.new(sum, denominator * other.getDenominator());
    }

    // more methods: minus, times, div, ...

    /** print this fraction */
    method void print() {
        do Output.printInt(numerator);
        do Output.printString("/");
        do Output.printInt(denominator);
        return;
    }
} // end Fraction
```

Jack Specification 9.2

General 9.2.1

תכנית Jack היא סדרה של סימנים (tokens) מסודרים בסדר מוסכם, וכן הערות ושורות ריקות שמהם מתעלמים. הסימנים הם symbols, reserved words, constants, identifiers כמפורט בטבלה:

White space and comments	Space characters, newline characters, and <i>comments</i> are ignored. The following <i>comment</i> formats are supported: <pre>// comment to end of line /* comment until closing */ /** API documentation comment */</pre>												
Symbols	() : arithmetic grouping, and parameter/argument-lists grouping [] : array indexing; { } : program structure and statement grouping; , : variable list separator; ; : statement terminator; = : assignment and comparison operator; . : class membership; + - * / & ~ < > : operators.												
Reserved words	<table> <tr> <td>class, constructor, method, function:</td><td>Program components</td></tr> <tr> <td>int, boolean, char, void:</td><td>Primitive types</td></tr> <tr> <td>var, static, field:</td><td>Variable declarations</td></tr> <tr> <td>let, do, if, else, while, return:</td><td>Statements</td></tr> <tr> <td>true, false, null:</td><td>Constant values</td></tr> <tr> <td>this:</td><td>Object reference</td></tr> </table>	class, constructor, method, function:	Program components	int, boolean, char, void:	Primitive types	var, static, field:	Variable declarations	let, do, if, else, while, return:	Statements	true, false, null:	Constant values	this:	Object reference
class, constructor, method, function:	Program components												
int, boolean, char, void:	Primitive types												
var, static, field:	Variable declarations												
let, do, if, else, while, return:	Statements												
true, false, null:	Constant values												
this:	Object reference												
Constants	Positive <i>Integer</i> constants are in standard decimal notation, e.g. "1984". Negative numbers like "-13" are not constants, but rather a unary minus operator applied to an integer constant. <i>String</i> constants are enclosed within two quote (") characters and may contain any characters except <i>newline</i> or <i>double-quote</i>. (These characters can be obtained using the function calls <code>String.newLine()</code> and <code>String.doubleQuote()</code>). <i>Boolean</i> constants can be <code>true</code> or <code>false</code>. The constant <code>null</code> signifies a null reference.												
Identifiers	Identifiers are composed from arbitrarily long sequences of letters (A-Z, a-z), digits (0-9), and "_". The first character must be a letter or "_". The language is case sensitive.												

היחידה הבסיסית של תוכנית Jack היא מחלקה - class. כל מחלקה נמצאת בקובץ נפרד ומקומפלת בנפרד לקובץ VM (בפרקים 10,11). הגדרת מחלקה מתבצעת באמצעות:

```
class name {
    field and static variable declarations // must precede the subroutine declarations
    subroutine declarations // a sequence of constructor, method, and function declarations
}
```

לאחר ההצהרה { class name נוכל להוסיף 0 או יותר שדות - fields ומשתנים סטטיים - static variables למחלקה. לאחר מכן נוכל להוסיף 0 או יותר שגרות - subroutines: מתודות - methods (פעולות על אובייקט, אינסטנס, מה-מחלקה), פונקציות - functions (פעולות ברמת המחלקה, סטטיות) ובנאים - constructors (בונים אובייקטים, אינסטנסים, מהמחלקה).
הגדרת שגרה (subroutine) מתבצעת באמצעות אחד מהבאים:

```
constructor type name (parameter-list) {
    declarations
    statements
}

method type name (parameter-list) {
    declarations
    statements
}

function type name (parameter-list) {
    declarations
    statements
}
```

לכל שגרה מוגדר שם (name) וסוג ערך ההחזרה שלה (type). פונקציות ומתודות יכולות לא להחזיר ערך, ואז נסמן את ה-type כ-void, אחרת נוכל להחזיר כל סוג פרימיטיבי (בהמשך), כל סוג שמוגדר ע"י מחלקה בסיפרייה הסטנדרטית של Jack (בהמשך) וכל סוג שמוגדר ע"י מחלקה בתכנית הנוכחית. בנאים יכולים להיות עם כל שם שנרצה (נהוג לתת להם את השם new) אך ערך ההחזרה שלהם חייב להיות אובייקט מהמחלקה.

לכל שגרה ניתן להוסיף פרמטרים - parameters ברשימה בסוגריים אחרי השם.

לאחר ההצהרה { subroutine type name (parameter-list) נוכל להוסיף 0 או יותר משתנים לוקאליים - local variables (משתנים שחיים רק בתוך הסקופ של הפונקציה), ולאחר מכן 0 או יותר פעולות - statements (פעולות שונות שניתן לבצע בשפה, בהמשך).

כמו ב-JAVA, תכנית Jack מורכבת ממספר קבצי jack שמתארים כל אחד מחלקה בתכנית. מחלקה אחת בתוך התכנית חייבת להיקרא Main, וחייבת להיות בה פונקציה בשם main. כאשר התכנית תרוץ, הפקודה הראשונה שתקרא תהיה קריאה לפונקציה Main.main, ומשם התוכנית צריכה להתחיל ולסיים לרוץ.

חייבים להצהיר על משתנים ב-Jack ועל הסוג שלהם לפני שמשתמשים בהם. יש 4 סוגי הגדרות משתנים: שדות - field, משתנים סטטיים - static, משתנים לוקאליים - local ופרמטרים - parameter, כל אחד מהם מוגדר לסקופ מסוים:

Var. Kind	Definition and Description	Declared in:	Scope
Static variables	static type name1, name2, ... ; Only one copy of each static variable exists, and this copy is shared by all the object instances of the class. (like <i>private static variables</i> in Java)	Class declaration.	The class in which they are declared.
Field variables	field type name1, name2, ... ; Every object instance of the class has a private copy of the field variables. (like <i>private object variables</i> in Java)	Class declaration.	The class in which they are declared, but cannot be used in functions.
Local variables	var type name1, name2, ... ; Local variables are allocated on the stack when the subroutine is called and are freed when it returns. (like <i>local variables</i> of Java).	Subroutine declaration.	The subroutine in which they are declared.
Parameters	type name1, type name2, ... e.g.: function void drive (Car c, int miles) { ... } Used as inputs of subroutines.	Appear in parameter-lists as part of subroutine declarations.	The subroutine in which they are defined.

סוגי משתנים: לכל משתנה יש סוג - type שנשאר איתו לכל זמן החיים שלו, ומוגדר בזמן הצהרת המשתנה. משתנים יכולים להיות פרימיטיביים - primitive (int, boolean, char) או מסוג-מחלקה מסויים - object type, כלומר הסוג שלהם הוא שם המחלקה שהם מייצגים אובייקט שלה. המחלקה שמממשת את הסוג הזה יכולה להיות חלק מהתוכנית או חלק מהספרייה הסטנדרטית של Jack (כמו Array או String).

סוגים פרימיטיביים: ב-Jack יש 3 סוגים פרימיטיביים:
 - int: מספר באורך 16 ביט בשיטת ה- 2's complement (מפרק 2), כלומר מספרים עם סימן.
 - boolean: true או false.
 - char: אות unicode, למשל 'c'.
 למשתנים אלו מוקצה זיכרון כאשר מצהירים עליהם.

סוגי מחלקות: כל מחלקה ב-Jack (עם בנאי) מגדירה סוג-מחלקה. כמו ב-Java, הצהרה על משתנה מסוג-מחלקה מסויים מקצה זיכרון לפויינטר (תא בזיכרון שיחזיק את הכתובת של הזיכרון ששומר את האובייקט עצמו, אם אחד כזה יוצב במשתנה). זיכרון לשמירת האובייקט עצמו מוקצה כאשר המתכנת מייצר אותו באמצעות הבנאי של המחלקה:

```
// The following assumes the existence of Car and Employee classes.
// Car objects have model and licensePlate fields.
// Employee objects have name and car fields.
var Employee e, f; // creates variables e, f that contain a null reference
var Car c; // creates a variable c that contains a null reference
...
let c = Car.new("Jaguar", "007") // constructs a new Car object
let e = Employee.new("Bond", c) // constructs a new Employee object
// At this point c and e hold the references to these two objects.
let f = e; // only the reference is copied
```

המחלקה הסטנדרטית של Jack מספקת שני סוגי-מחלקות: Array, String.

1. String: מחלקה built-in בתוך הספרייה הסטנדרטית של Jack. הקומפיילר מזהה את הרצף "aaa" ומתייחס אליו כתוכן של אובייקט סטרינג. ניתן לגשת ולשנות את תוכן הסטרינג, ולייצר סטרינגים בדרכים שונות, באמצעות הפונקציות של המחלקה (בהמשך).

2. Array: מחלקה built-in בתוך הספרייה הסטנדרטית של Jack. מערכים הם חד מימדיים ומתחילים תמיד באינדקס 0. לתאים של מערכים אין סוג מוגדר, וניתן לאחסן במערך אחד איברים מסוגים שונים (בפרט ניתן, למשל, לבנות מערך של מערכים - מטריצה). כאשר מצהירים על מערך מוקצה זיכרון לרפרנס (פויינטר) למערך, והקצאת זיכרון למערך עצמו מתבצעת כאשר משתמשים בפונקציה new של המחלקה: Array.new(length) ומציבים את ערך ההחזרה במשתנה מסוג Array הרצוי. גישה לתא במערך מתבצעת באמצעות הפקודה arr[i]. דוגמא:

```
/** Computes the average of a sequence of integers */
class Main {
    function void main() {
        var Array a;
        var int length;
        var int i, sum;

        let length = Keyboard.readInt("HOW MANY NUMBERS? ");
        let a = Array.new(length);
        let i = 0;
        while (i < length) {
            let a[i] = Keyboard.readInt("ENTER THE NEXT NUMBER: ");
            let i = i + 1;
        }
        let i = 0;
        let sum = 0;
        while (i < length) {
            let sum = sum + a[i];
            let i = i + 1;
        }
        do Output.printString("THE AVERAGE IS: ");
        do Output.printInt(sum / length);
        do Output.println();
        return;
    }
}
```

המרת סוגים - Type Conversions: שפת Jack היא weakly-typed. מפרט השפה לא מפרט את ההתנהגות במקרה שמנסים להציב ערך מסוג אחד למשתנה מסוג אחר (באופן כללי) וקומפיילרים שונים יכולים להתייחס אל פעולות כאלה בדרכים שונות. למרות זאת, למקרים הספציפיים הבאים קומפיילר Jack נדרש להתייחס בדרך המתוארת:

- char ו-int מומרים אוטומטית מאחד לשני אם נדרש, בהתאם למפרט Unicode. למשל:

```
var char c; var int i;
let c = 33; // c = 'A'
let i = 'A'; // i = 33
```

- int יכול להיות מומר לסוג-מחלקה (פויינטר), וההתייחסות אליו תהיה כאל כתובת בזיכרון. למשל:

```
var Array arr; // declaration
var Array arr2; // declaration
let arr = Array.new(200); // initialization, the compiler set arr base addr
let arr[100] = 77; // RAM[arrBaseAddr + 100] = 77
let arr2 = 5000 // set arr2 base addr to 5000
let arr2[100] = 77; // RAM[5000 + 100] = 77
```

- משתנה מסוג-מחלקה יכול להיות מומר למערך. ההמרה הזו מאפשרת לגשת לשדות האובייקט באותו אופן שניגשים לתאים במערך ולהיפך. למשל:

```
// Assumes that class Complex has two int fields: re & im
var Complex num; var Array arr;
let arr = Array.new(2);
let arr[0] = 7; let arr[1] = 8;
let num = arr; // num == Complex(7, 8)
```

פעולות שונות שניתן לבצע בשפה:

Statement	Syntax	Description
let	let <i>variable</i> = <i>expression</i> ; or let <i>variable</i> [<i>expression</i>]= <i>expression</i> ;	An assignment operation. The variable may be static, local, field, or a parameter. Alternatively the variable may be an array entry.
if	if (<i>expression</i>) { <i>statements</i> } else { <i>statements</i> }	Typical <i>if</i> statement with an optional <i>else</i> clause. The brackets are mandatory even if <i>statements</i> is a single statement.
while	while (<i>expression</i>) { <i>statements</i> }	Typical <i>while</i> statement. The brackets are mandatory even if <i>statements</i> is a single statement.
do	do <i>function-or-method-call</i> ;	Used to call a function or method for their effect, ignoring the value they return, if any. The function or method call follows the syntax described in section 2.6.
return	return <i>expression</i> ; or return ;	Used to return values from subroutines. The second form must be used by functions and methods that return a void value. Constructors must return the expression <i>this</i> .

ביטויים בשפה (משתמשים ב- expressions). מוגדרים רקורסיבית באופן הבא:

A *Jack expression* is one of the following:

- ❑ A *constant*
- ❑ A *variable name* in scope (the variable may be *static*, *field*, *local*, or a *parameter*)
- ❑ The `this` keyword, denoting the current object. Cannot be used in functions.
- ❑ An *array element* using the syntax `name[expression]`, where *name* is a variable name of type `Array` in scope.
- ❑ A *subroutine call* that returns a non-void type (see Section 2.6).
- ❑ An expression prefixed by one of the unary operators `{-,~}`:
 - expression*: arithmetic negation
 - ~*expression*: boolean negation (bitwise for integers)
- ❑ An expression of the form *expression operator expression* where *operator* is one of the binary operators `{+,-,*,/,&,|,<,>,=}`:

<code>+ - * /</code>	:	integer arithmetic operators
<code>& </code>	:	boolean <code>And</code> , <code>Or</code> (bitwise for integers) operators
<code>< > =</code>	:	comparison operators
- ❑ An expression in parenthesis: (*expression*)

חשוב: סדר הערכת ביטויים לא מוגדרת בשפה, וכל קומפיילר יכול לממש אותה באופן שונה. בביטויים מורכבים, כדי להבטיח שנקבל את הביטוי שאנחנו רוצים, יש להשתמש בסוגריים במקומות המתאימים.

קריאות לשגרות מתבצעות לפי הסינטקס `functionName(argument-list)`. מספר וסוגי הארגומנטים חייבים להתאים לאלו של רשימת הפרמטרים של הפונקציה, שמופרטת בהצהרת הפונקציה. הסוגיים חייבים להופיע אפילו אם רשימת הארגומנטים ריקה. ארגומנט יכול להיות ביטוי בסיסי, ויכול להיות גם ביטוי מסובך שיש צורך בחישוב רקורסיבי כדי להעריך אותו. פירוט הסינטקס:

- בתוך מחלקה, מתודות נקראות באמצעות הסינטקס `methodName(argument-list)`.
 - פונקציות ובנאים נקראים תמיד בשימוש בשם המלא שלהם, כלומר `className.subroutineName(argument-list)`.
 - מחוץ למחלקה, מתודות נקראות דרך אובייקט קיים `var` מסוג המשתנה (שעליו תפעל המתודה) באמצעות הסינטקס `var.methodName(argument-list)`.
- דוגמא:

```
class Foo {
  // Some subroutine declarations - code omitted
  ...
  method void f() {
    var Bar b;          // Declares a local variable of class type Bar
    var int i;           // Declares a local variable of primitive type int
    ...
    do g(5,7)            // Calls method g of class Foo (on this object)
    do Foo.p(2)           // Calls function p of class Foo
    do Bar.h(3)           // Calls function h of class Bar
    let b = Bar.r(4);      // Calls constructor or function r of class Bar
    do b.q()              // Calls method q of class Bar (on object b)
    let i = w(b.s(3), Foo.t()) // Calls method w on this object,
                           // method s on object b and function
                           // or constructor t of class Foo
    ...
  }
}
```

בנייה ומחיקה של אובייקטים: בניית אובייקט מורכבת משני שלבים. ראשית מכריזים על משתנה מסוג המחלקה באמצעות הקריאה:

```
var objectType varName;
```

למשל משתנה `c` מסוג `Circle`: `var Circle c`

פקודה זו גורמת לקומפיילר להקצות זיכרון לפוינטר לאובייקט (שיקושר למשתנה `c` מעתה), ולא לאובייקט עצמו. הקצאה הזיכרון לאובייקט מתבצעת באמצעות קריאה לבנאי של המחלקה `Circle`. כלומר, למחלקה שממשתנה סוג-מחלקה חייב להיות לפחות בנאי אחד. בנאים יכולים לקבל איזה שם שנרצה, אבל נהוג לקרוא להם `new`. כעת, כדי להשלים את בניית האובייקט והשמתו בכתובת המוצבת במשתנה `c` נבצע את הקריאה:

```
let varName=ClassName.constructorName(parameter-list);
```

למשל, `let c = Circle.new(x,y,50)` כאשר `(x,y)` מסמנים פיקסל על המסך ו-50 זה רדיוס המעגל (אנחנו מניחים שהמחלקה `Circle` קיימת ויש לה בנאי `new` עם רשימת פרמטרים מתאימה).

פקודה זו תגרום לקריאה לבנאי. כעת הקומפיילר יקצה, באמצעות מערכת ההפעלה, מספיק מקום בזיכרון כדי לאחסן בו אובייקט מהמחלקה הזו. מערכת ההפעלה תחזיר כתובת בסיס למקטע הזיכרון שהוקצה עבור האובייקט, והקומפיילר יאחסן את הכתובת הזו ב-`this`, והערך ב-`this` יאוחסן במשתנה, הפוינטר, שמקושר לאובייקט (במקרה שלנו במשתנה `c`, כלומר במקום בזיכרון שמוקצה ל-`c`, רגיסטר אחד, תאוחסן כתובת הבסיס של המשתנה החדש שיצרנו). לאחר מכן, תוכן הבנאי יתחיל לרוץ ובד"כ יאתחל את שדות האובייקט החדש כדי שיהיו במצב וואלידי (וגישה אליהם לא תגרום לשגיאה).

כאשר סיימנו להשתמש באובייקט מסויים, נרצה לשחרר את הזיכרון בו הוא מאוחסן, לשימוש עתידי עבור אובייקטים אחרים. ניתן לעשות זאת באמצעות קריאה לפונקציה של הספרייה הסטנדרטית:

```
Memory.deAlloc(varName);
```

בעיקרון, בכל מחלקה שמגדירה סוג-מחלקה (כלומר שיש לה בנאי) נכתוב מתודת dispose שמטרתה למחוק אוביי-קטים מהמחלקה בצורה בטוחה, ולהבטיח שכל הזיכרון שהוקצה לאובייקט משוחרר.
דוגמא - רשימה מקושרת:

```
/** A List class (partial implementation) */
class List {
    field int data;
    field List next;

    constructor List new(int car, List cdr){
        let data = car;
        let next = cdr;
        return this;
    }
    ...

    method void dispose() {
        if (~next = null) {
            do next.dispose();
        }
        do Memory.deAlloc(this);
        return;
    }
}

// create a list holding the numbers (2,3,5):
function void create235(){
    var List v;
    let v=List.new(5,null);
    let v=List.new(2,List.new(3,v));
    // ...
    do v.dispose();
    return;
}
```

The Jack Standard Library 9.2.7

שפת Jack מגיעה עם ספרייה סטנדרטית, שניתן לראות אותה כממשק של מערכת ההפעלה. זהו אוסף של מחלקות Jack שחייב להיות מצורף לכל תוכנית Jack. הספרייה מכילה את המחלקות הבאות:

- Math: מספקת פעולות מתמטיות בסיסיות.
 - String: מממשת את הסוג String ומספקת פעולות בסיסיות על סטרינגים.
 - Array: מממשת את הסוג Array ומספקת בנייה ומחיקה של מערכים.
 - Output: מטפלת בפלט למסך בצורת טקסט.
 - Screen: מטפלת בפלט גרפי למסך.
 - Keyboard: מטפלת בקלט המשתמש דרך המקלדת.
 - Memory: מטפלת בפעולות זיכרון (הקצאת זיכרון, שחרור זיכרון ועוד).
 - Sys: מספקת מספר פעולות שקשורות להרצת התוכנית.
- ה-API המפורט של כל אחת מהמחלקות בספרייה הזו מצורף כ-pdf לסיכום (JackStandardLib.pdf).

Compiler I: Syntax Analysis 10

Background 10.1

בפרק 9 פירטנו את המפרט של שפת Jack. בפרט זה נתחיל את בניית הקומפיילר משפה זו לשפת VM. נסיים את הבנייה בפרק 11, מה שישלים את בניית המתרגם (קומפיילר) משפת Jack עבור מחשב ה-Hack שלנו. החלק הזה של תהליך הקומפילציה מ-Jack-- (מעבר מ-Jack ל-VM) נקרא ה-front-end של הקומפיילר, והחלק הקודם שבנינו בפרקים 7,8 (מעבר מ-VM לאסמבלי) נקרא ה-back-end של הקומפיילר.

בניית ה-front-end של הקומפיילר מורכבת משני חלקים. ראשית, על הקומפיילר להבין את הסינטקס של הקלט, כלומר לפרק את התכנית למבנה הבסיסי של הסינטקס של השפה, ושנית, על הקומפיילר לתרגם את הפירוק הזה לשפת היעד. לפעולה הראשונה אנחנו קוראים Syntax Analysis, ובה נתמקד בפרק זה. לפעולה השנייה, בה נתמקד בפרק הבא, אנחנו קוראים Code Generation.

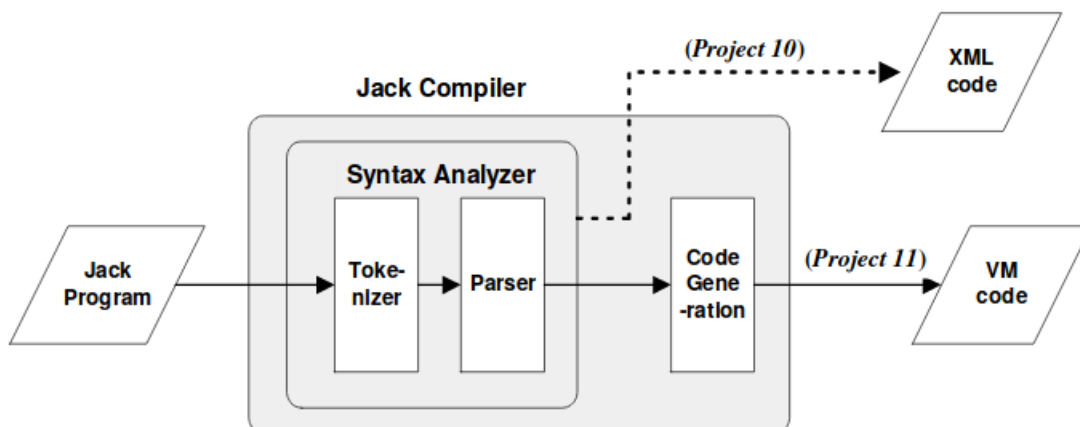
כאשר מדברים על "הבנת הסינטקס" מתכוונים לפעולה שאין בה ייצור קוד של שפת היעד. במקום זאת, ה-syntax analyzer מפרק את קוד הקלט למבנה הסינטקטי הבסיסי שלו ומוציא קובץ XML שמתאר את המבנה הזה כעץ.

המשימה של עיבוד סינטקס - Syntax Analysis - מפוצלת בדרך כלל לשני מרכיבים:

- tokenizing: פירוק קוד הקלט לאטומים הבסיסיים (או tokens, טוקנים) מהם מורכבת השפה.

- parsing: התאמת סדרת הטוקנים לחוקי הסינטקס של השפה.

שתי המשימות האלו בלתי תלויות בשפת היעד. ואכן, בפרק זה נתייחס רק ל-Jack ולפירוט הסינטקס שלה באופן פורמלי, ורק בפרק הבא נתמקד בתרגום התוצר של ה-syntax analyzer לשפת היעד VM. דרך הפעולה של הקומפיילר והחלוקה לפרקים מתוארת באיור הבא:



בחלק זה המטרה היא לקבל קובץ jack ולפרק את הטקסט שבו לאטומים הבסיסיים של השפה: Tokens, טוקנים. דוגמא כללית:



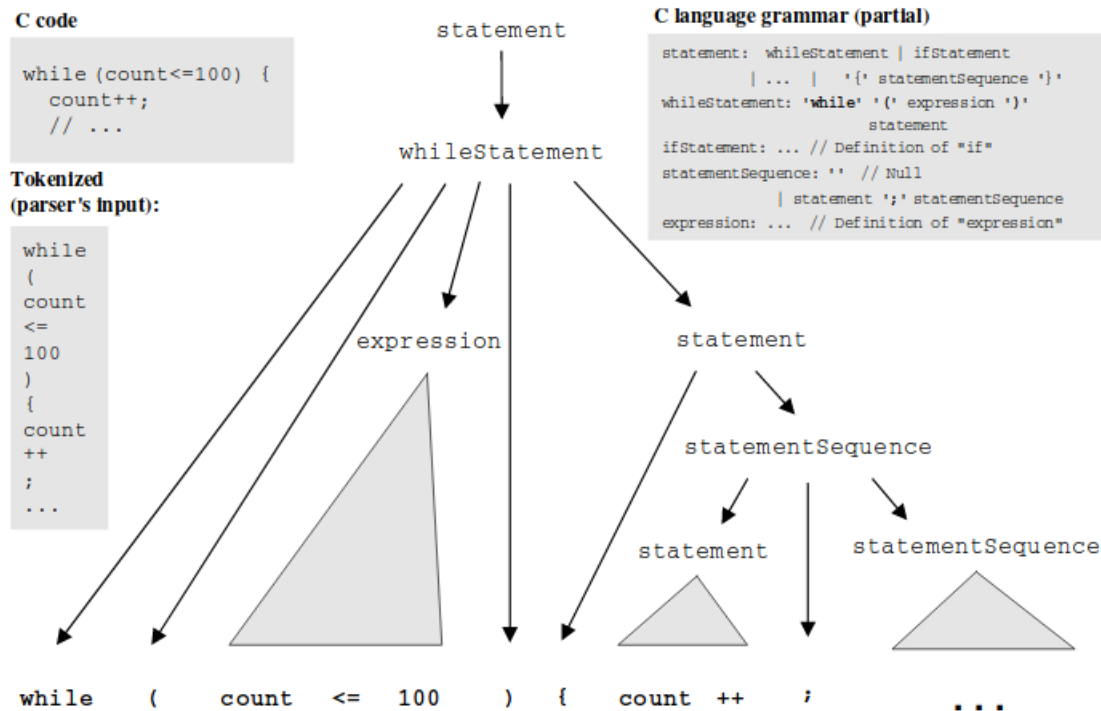
הטוקנים ב-Jack:

Lexical elements:	The Jack language includes five types of terminal elements (tokens):
keyword:	'class' 'constructor' 'function' 'method' 'field' 'static' 'var' 'int' 'char' 'boolean' 'void' 'true' 'false' 'null' 'this' 'let' 'do' 'if' 'else' 'while' 'return'
symbol:	'{' '}' '(' ')' '[' ']' '.' ',' ';' '+' '-' '*' '/' '%' '&' ' ' '<' '>' '=' '~'
integerConstant:	A decimal number in the range 0 .. 32767.
StringConstant:	'"' A sequence of Unicode characters not including double quote or newline "'
identifier:	A sequence of letters, digits, and underscore ('_') not starting with a digit.

Grammers & Parsing 10.1.2

בחלק זה המטרה, בהינתן סדרת הטוקנים שמתקבלת מה-tokenizer, לקבץ ולסדר את הטוקנים במבנים פורמליים (כמו הצהרות משתנים, ביטויים ופקודות שונות בשפה Jack) שיאפשרו ל-Code Generator שנבנה בפרק הבא לתרגם את הטוקנים לתוכנית פועלת בשפת VM, בצורה פשוטה. התהליך הזה נקרא פרסור - parsing. הרמז המשמעותי לדרך הפעולה שלנו בחלק הזה הוא ששפות התכנות, ושפות פורמליות באופן כללי, הן שפות חסרות הקשר. בקצרה, זה אומר שהדקדוק שלהן מורכב מסט של חוקים שמנחים אותנו בבניית אלמנטים מורכבים בשפה (נקראים non-terminals) באמצעות האלמנטים הפשוטים (נקראים terminals - אותיות), ושהחוקים המופעלים ליצירת non-terminal מסויים (למשל statement) בשפה לא תלויים בהקשר שלו, כלומר אין משמעות למה היה כתוב לפני או אחרי האלמנט הזה, אלא רק מה הסיווג שלו.

המבנה של דקדוק חסר הקשר הוא היררכי, לכן ניתן לייצג את פלט הפרסור כעץ גזירה. לדוגמא, עבור פרסור שפת C:



ישנם מספר אלגוריתמים ליצירת עצי גזירה כאלו. אנחנו נשתמש בשיטת top-down Recursive Descent Parsing.

Recursive Descent Parsing: ישנם מספר אלגוריתמים ליצירת עצי גזירה כאלו. אנחנו נשתמש בשיטת top-down Recursive Descent Parsing. בשיטה זו מנסים לפרסר את הטוקנים באופן רקורסיבי, באמצעות המבנה המקוון שמרכיב את הדקדוק של Jack.

דרך פעולתו של פרסר שפועל בשיטה הזו יכולה להיות מפושטת בצורה הבאה:

ראשית נצייד את הפרסר עם דרך לעבד כל terminal בדקדוק. כעת, לכל non-terminal בדקדוק, נצייד את הפרסר במתודה שאחראית לפרסור האלמנט הזה. אם האלמנט מורכב רק מ-terminals, המתודה פשוט תעבד אותם ותמשיך לקרוא הלאה מה-tokenizer. אחרת, לכל בלוק non-terminal בתוך האלמנט הזה, נעבד אותו באופן רקורסיבי ונמשיך עד שנסיים את עץ הגזירה עבור המשתנה הזה.

LL(0) Grammar: הדרך שתיארנו לפרסור היא פשוטה ואלגנטית. הבעיה היחידה שיכולה לעלות זה כאשר יש אלטרנטיבות לתרגום non-terminal מסויים, והאלטרנטיבה שנבחרת נקבעת לא רק לפי הטוקן הבא, אלא יותר מאחד. ככל שנצטרך להסתכל יותר קדימה כדי לבחור את האלטרנטיבה, כך הפרסור יהפוך להיות יותר מורכב. למזלנו, הדקדוק של Jack הוא כמעט LL(0). הבעיה היחידה נמצאת בפרסור של expression, שם יש צורך להסתכל על שני tokens קדימה בחלק מהמקרים.

Lexical elements:	The Jack language includes five types of terminal elements (tokens):
keyword:	'class' 'constructor' 'function' 'method' 'field' 'static' 'var' 'int' 'char' 'boolean' 'void' 'true' 'false' 'null' 'this' 'let' 'do' 'if' 'else' 'while' 'return'
symbol:	'{' '}' '(' ')' '[' ']' '.' ',' ';' '+' '-' '*' '/' '%' ' ' '<' '>' '=' '~'
integerConstant:	A decimal number in the range 0 .. 32767.
StringConstant	'"' A sequence of Unicode characters not including double quote or newline '"'
identifier:	A sequence of letters, digits, and underscore ('_') not starting with a digit.
Program structure:	A Jack program is a collection of classes, each appearing in a separate file. The compilation unit is a class. A class is a sequence of tokens structured according to the following context free syntax:
class:	'class' className '{' classVarDec* subroutineDec* '}'
classVarDec:	('static' 'field') type varName (',' varName)* ';'
type:	'int' 'char' 'boolean' className
subroutineDec:	('constructor' 'function' 'method') ('void' type) subroutineName '(' parameterList ')' subroutineBody
parameterList:	((type varName) (',' type varName)*)?
subroutineBody:	'{' varDec* statements '}'
varDec:	'var' type varName (',' varName)* ';'
className:	identifier
subroutineName:	identifier
varName:	identifier
Statements:	
statements:	statement*
statement:	letStatement ifStatement whileStatement doStatement returnStatement
letStatement:	'let' varName '(' expression ')' '?' '=' expression ';'
ifStatement:	'if' '(' expression ')' '{' statements '}' ('else' '{' statements '}')?
whileStatement:	'while' '(' expression ')' '{' statements '}'
doStatement:	'do' subroutineCall ';'
ReturnStatement	'return' expression? ';'
Expressions:	
expression:	term (op term)*
term:	integerConstant stringConstant keywordConstant varName varName '[' expression ']' subroutineCall '(' expression ')' unaryOp term
subroutineCall:	subroutineName '(' expressionList ')' (className varName) '.' subroutineName '(' expressionList ')'
expressionList:	(expression (',' expression)*)?
op:	'+' '-' '*' '/' '%' ' ' '<' '>' '='
unaryOp:	'-' '~'
KeywordConstant:	'true' 'false' 'null' 'this'

התכנית JackAnalyzer מורכבת משלושה מודולים (מחלקות): JackAnalyzer, JackTokenizer, CompilationEngine. * יש API מפורט בספר בפרק 10.

JackTokenizer: מתעלם משורות ריקות והערות. מחלק את קובץ הקלט לטוקנים לפי המפרט של שפת Jack.

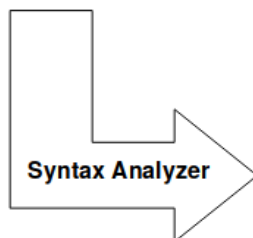
CompilationEngine: הפרסר. מקבל קלט מה-JackTokenizer ומפרסר אותו לפי השיטה שתוארה קודם ולפי מבנה שפת Jack. לכל non-terminal יש פונקציית קומפילציה שמפרסרת אותו וקורסיבית.

JackAnalyzer: המודול המרכזי, מריץ את התכנית: פותח את הקובץ החדש עם סיומת xml, מנחה את ה-JackTokenizer לעבור על קובץ ה-jack, ואת ה-CompilationEngine לפרסר את סדרת הטוקנים שיוצאת מה-JackTokenizer.

דוגמא לפעולת ה-JackAnalyzer:

Analyzer's input (Jack code)

```
Class Bar {
  method Fraction foo(int y) {
    var int temp; // a variable
    let temp = (xxx+12)*-63;
    ...
  }
}
```



Analyzer's output (XML code)

```
<class>
  <keyword> class </keyword>
  <identifier> Bar </identifier>
  <symbol> { </symbol>
    <subroutineDec>
      <keyword> method </keyword>
      <identifier> Fraction </identifier>
      <identifier> foo </identifier>
      <symbol> ( </symbol>
      <parameterList>
        <keyword> int </keyword>
        <identifier> y </identifier>
      </parameterList>
      <symbol> ) </symbol>
      <subroutineBody>
        <symbol> { </symbol>
          <varDec>
            <keyword> var </keyword>
            <keyword> int </keyword>
            <identifier> temp </identifier>
            <symbol> ; </symbol>
          </varDec>
          <statements>
            <letStatement>
              <keyword> let </keyword>
              <identifier> temp </identifier>
              <symbol> = </symbol>
              <expression>
                <term>
                  <symbol> ( </symbol>
                  <expression>
                    <term>
                      <identifier> xxx </identifier>
                    </term>
                  <symbol> + </symbol>
                  <term>
                    <integerConstant> 12
                  </integerConstant>
                </term>
              </expression>
            </letStatement>
            <symbol> ) </symbol>
          </statements>
        </symbol>
      </subroutineBody>
    </subroutineDec>
  </symbol>
</class>
```

Compiler II: Code Generation 11

Background 11.1

בחלק זה נשלים את בניית ה- front-end של הקומפיילר: ה-code-generation.

Data Translation 11.1.1

תכניות משתמשות במספר גדול של סוגים - types - של משתנים, פרימיטיביים כמו int ומורכבים כמו מערכים או אובייקטים. בנוסף, ישנם מספר זנים - kinds - של משתנים (ארגומנטים, משתנים לוקאליים, סטטיים או שדות), כל אחד מהזנים חי בסקופ מסויים ויש לו אורך חיים מסויים. כל סוג וזן של משתנה מנוהל באופן אחר ע"י הקומפיילר, לכן בפרט הקומפיילר חייב לדעת מה הסוג והזן של כל משתנה בתוכנית. לשם כך הוא שומר טבלת סמלים, שמטרתה, בכל פעם שהקומפיילר נתקל בהצהרה על משתנה, לשמור את שמו, הסוג שלו והזן שלו.

Symbol Table & Handling Variables: בכל פעם שהקומפיילר נתקל בהצהרה על משתנה הוא צריך לשמור אותו כדי שיוכל לזהות אותו בהמשך. לשם כך הוא משתמש בטבלה שמורכבת מ-4 עמודות: שם המשתנה, הסוג שלו, הזן שלו והמספר הסידורי שלו בתוך הזן הזה (כלומר אם הוא static והיו כבר שני משתנים סטטיים במחלקה, המספר הסידורי שלו הוא 3). לכל מחלקה יש טבלה (גלובאלית), שמחזיקה את השדות והמשתנים הסטטיים של המחלקה, ולכל פונקציה יש טבלה (לוקאלית) שמחזיקה את הארגומנטים והמשתנים הלוקאליים. במקרה שהקומפיילר נתקל בשם של משתנה v באיזו פונקציה f במחלקה c, הוא קודם יחפש את v בטבלה של הפונקציה f, ואם לא ימצא אז יחפש בטבלה של המחלקה c. ברגע שהוא מוצא את המשתנה בטבלה המתאימה, הוא יכול פשוט להחליף את שמו ב-kind index (למשל 2 local), שזה איך שה-VM מכיר את המשתנה הזה בתוך הפונקציה (או המחלקה) הזו.

Handling Arrays: מערכים כמעט תמיד מאוחסנים במקום רציף בזיכרון. כאשר מגדירים מערך bar, הקומפיילר רק מקצה מקום לפוינטר למערך (תא שיחזיק בעתיד את כתובת הבסיס של המערך בערימה), ולאחר מכן יש צורך להקצות מקום למערך עצמו באמצעות מערכת ההפעלה: `bar=Array.new(length)`. בפקודה זו הקומפיילר מפעיל את פונקציית alloc של המחלקה Memory, שדואגת להקצות מספר מספיק של תאים למערך ולהכניס את כתובת הבסיס למקום שהוקצה למשתנה bar.

מספר דוגמאות לפעולות על מערכים: ראשית, נניח שכתובת לתא אליו נרצה לגשת נמצאת במשתנה הלוקאלי הראשון (נסמן ב-p). הגישה לתא הזה בשפת VM תתקבל באופן הבא:

```
push local 0
pop pointer 1 // THAT=RAM[4]=p, that 0 = RAM[THAT]=RAM[p]
```

כעת, כדי לשנות את הערך בתא הזה, נניח לקבוע 19, נבצע את הפקודה הבאה:

```
push constant 19
pop that 0 // RAM[p] = 19
```

כעת, נניח שאנחנו רוצים לבצע את הפעולה $\text{bar}[k]=19$. מכיוון שהמשתנה bar מכיל את כתובת הבסיס של המערך, ניתן לתרגם את הפקודה לנוטציה $*(\text{bar}+k)=19$, כלומר "אחסן את 19 בתא שנמצא בכתובת $\text{bar}+k$ בזיכרון. בשפת VM הפעולה תתבצע באופן הבא (נניח ש- bar הוא המשתנה הלוקאלי הראשון ו- k הוא הפרמטר הראשון של הפונקציה):

```
push local 0
push argument 0
add // now the first element in the stack is bar+k, the target address
pop pointer 1 // THAT = bar+k, that 0 = RAM[THAT]=RAM[bar+k]
push constant 19
pop that 0 // RAM[RAM[THAT]+0]=RAM[bar+k]=19
לחלופין, נוכל לכתוב את זה בצורה הבאה: ***לא נכון!!! לבדוק למה זה לא עובד.
push local 0 // now the first element in the stack is bar, the base address of the array
pop pointer 1 // THAT = bar, that 0 = RAM[THAT]=RAM[bar]
push constant 19
pop that k // RAM[RAM[THAT]+k]=RAM[bar+k]=19
```

Handling Objects: אובייקט של מחלקה מסוימת מחזיק שדות וסט פעולות - מתודות. הטיפול בכל אחד מהם שונה. נתאר את הטיפול בשדות, שדומה לטיפול במערכים: כמו במערכים, כאשר מצהירים על משתנה מסוג-מחלקה מסויים, מוקצה זיכרון רק לפוינטר (תא בזיכרון שמקושר למשתנה, ובהמשך תוצב בו כתובת הבסיס של האובייקט), והזיכרון לאובייקט עצמו (כלומר לשדות האובייקט) מוקצה רק בקריאה לבנאי.

לכן, כאשר מקמפלים בנאי של מחלקה, הקומפיילר ראשית בודק מה מספר השדות של אובייקט במחלקה הזו (נניח n), ואז מייצר את הקוד הדרוש כדי להקצות זיכרון למספר הזה של שדות (הזיכרון הזה מוקצה ברצף ב-RAM, כמו במערכים): $\text{Memory.alloc}(n)$. את כתובת הבסיס למקטע הזיכרון שהוקצה לאובייקט מכניסים לפוינטר this שניתן לגשת אליו דרך $\text{pointer } 0$. כלומר בסה"כ קימפול בנאי תמיד מתחיל בספירת מספר השדות (n) ואז שלוש השורות הבאות:

```
push constant n
call Memory.alloc 1 // the first element in the stack = the base address of the new object
pop pointer 0 // store the base address of the new object in THIS = RAM[3] = pointer 0
```

גישה לשדה של האובייקט תתבצע באופן דומה לגישה לתאים במערך: נניח שיש לנו מחלקה Complex שמממשת מספר מרוכב. יש לה שני שדות: re ו- im . נניח שבתוך מתודה $\text{add}(\text{int})$ של המחלקה, אנחנו רוצים לממש את הפעולה: $\text{im} = \text{im} + c$, כאשר c הוא ארגומנט (יחיד) למתודה. אז הקומפיילר יפעל באופן הבא:

- הוא יבחן את טבלת הסמלים של המתודה וימצא ש- im הוא השדה השני של האובייקט (שמאוחסן כעת ב- this , עוד על זה בעוד רגע), ו- c הוא הארגומנט הראשון של המתודה.

- כעת הוא יסתכל על הפעולה בתור $\text{field } 1 = \text{field } 1 + \text{argument } 0$.

- $\text{field } 1$ יתורגם: כתובת הבסיס של מקטע הזיכרון השמור לאובייקט נמצא ב- this , והשדה השני ($\text{field } 1$) יהיה בתא

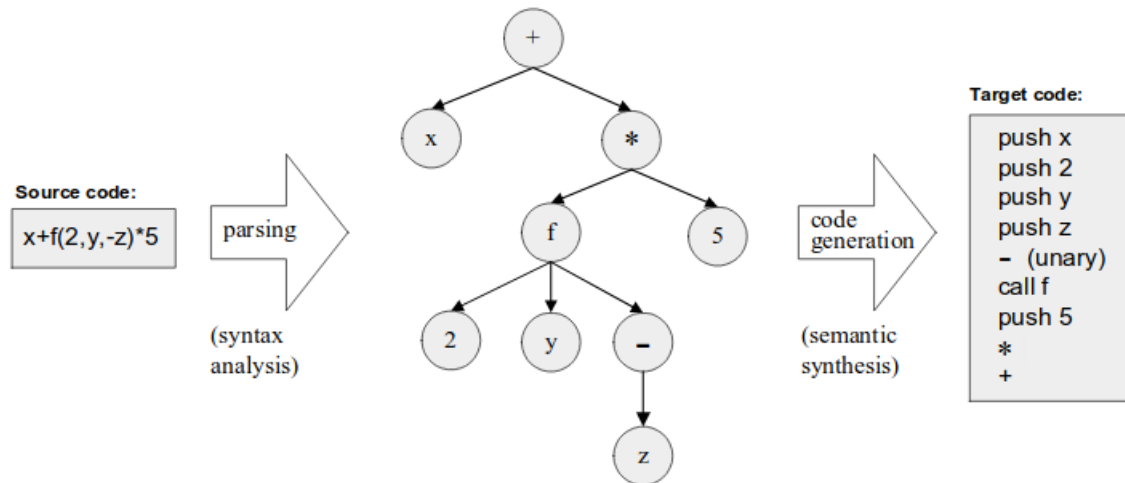
השני במקטע הזה. לכן, $\text{field } 1$ יתורגם ל- $*(\text{this}+1)$, שמתורגם באותו אופן כמו מערכים.

כעת, נניח שאנחנו רוצים להפעיל את המתודה add על אובייקט b מסוג Complex (שכבר בנינו, כלומר הגדרנו משתנה a מסוג Complex , הפעלנו את הבנאי של Complex והצבנו את ערך ההחזרה שלנו ב- b). הקריאה תתבצע ב-Jack באמצעות הפקודה $b.\text{add}(5)$. בניגוד לשדות, שיש עותק שלהם בכל אובייקט, יש רק עותק אחד של כל מתודה, למשל add , שמשמשים את כל האינסטנסים של המחלקה Complex . כדי לגרום לכך שהתוכנית תדע על איזה אובייקט להפעיל את המתודה בקריאה מסוימת, הקומפיילר יוסיף את האובייקט הנדרש בתוך ארגומנט למתודה. למשל, במקרה שלנו, ל- add יהיו בעצם שני ארגומנטים: Complex , ואז int . כאשר תתבצע הקריאה $b.\text{add}(5)$, הקומפיילר יתרגם את זה לקריאה $\text{Complex.add}(b, 5)$, ובשפת vm (פסאודו-קוד) נקבל:

```
push b; push 5; call Complex.add 2
```

נתאר את תהליך התרגום של פקודות high-level לשפת VM. נשארו לנו שני סוגים של פקודות לתרגם: expressions ו-flow-control.

Evaluating Expressions: נניח שנרצה לתרגם את הערכת הביטוי $x + g(2, y, -5) * 5$. ראשית, עלינו "להבין" את המבנה הסינטקטי של הביטוי, למשל באמצעות עץ גזירה, שזה מה שעשינו בפרק הקודם באמצעות ה-syntax analyzer. כעת נוכל לעבור על העץ ולתרגם את הפעולות בו לקוד VM, שבסופו הערך העליון במחסנית יהיה הערך של הביטוי הדרוש.
דוגמא:



האלגוריתם שעליו נתבסס למעבר על העץ ותרגום לשפת VM, שהיא שפה מבוססת-מחסנית, הוא "הדפסת עץ בנוצציית postfix", שמכונה גם Right Polish Notation או RPN. בסינטקס RPN, אופרטור $f(x, y)$ יכתב כ- "`x, y, f`", או, בשפת VM:

```
push x
push y
call f
```

באותו אופן, אופרטור $x + y$ יכתב כ- "`x, y, +`", או בשפת VM:

```
push x
push y
add
```

באופן כללי נתאר את האסטרטגיה להערכת expressions באמצעות הכללים הבאים:

Code(exp):

```
if exp is a number n      then output "push n"
if exp is a variable v    then output "push v"
if exp = (exp1 op exp2)   then Code(exp1); Code(exp2); output "op"
if exp = op(exp1)         then Code(exp1); output "op"
if exp = f(exp1 ... expN) then Code(exp1) ... Code(expN); output "call f"
```

Translating Flow Control: ב-Jack ישנם שני סוגי פקודות flow-control: if-else ו-while. את שניהם מממשים באמצעות פקודות VM if-goto ו-goto. פסאודו קוד:

Source code	Generated code
if (cond)	code for computing ~cond
s1	if-goto L1
else	code for executing s1
s2	goto L2
...	label L1
	code for executing s2
	label L2
	...
while (cond)	label L1
s1	code for computing ~cond
...	if-goto L2
	code for executing s1
	goto L1
	label L2
	...

11.2 Specification & Implementation

11.2.1 Standard Mapping over the VM

הקומפיילר מתרגם כל קובץ jack (מחלקה) לקובץ vm שמכיל פונקציה VM אחת לכל בנאי, פונקציה ומתודה.

- שמות קבצים ופונקציות: קובץ jack בשם aaa (שמכיל את המחלקה aaa) בתכנית מתורגם לקובץ vm עם אותו שם.
- שגרה בשם foo בקובץ aaa מקומפלת לפונקציית VM בשם aaa.foo.
- פונקציה או בנאי ב-Jack עם k ארגומנטים מקומפלים לפונקציית VM עם k ארגומנטים.
- מתודה ב-Jack עם k ארגומנטים מקומפלת לפונקציית VM עם k+1 ארגומנטים, כאשר הארגומנט הראשון (argument 0) תמיד מתייחס לאובייקט עליו מופעלת המתודה - this. כמו שתואר קודם, הקומפיילר אחראי להכניס את האובייקט למחסנית כארגומנט הראשון (ולאחר מכן את שאר הארגומנטים) לפני הקריאה למתודה.

הקצאת זיכרון וגישה לזיכרון:

- למשתנים לוקאליים של שגרת Jack מוקצה מקום במקטע הזיכרון local, ודרך שם ניגשים אליהם.
- לארגומנטים של שגרת Jack מוקצה מקום במקטע הזיכרון argument, ודרך שם ניגשים אליהם.
- למשתנים סטטיים של מחלקה aaa מוקצה מקום במקטע הזיכרון static, ודרך שם ניגשים אליהם. המקטע הזה משותף לכל המחלקה.
- בתוך פונקציית VM שמתאימה למתודה או בנאי של מחלקה, הגישה לאובייקט עליו פועלים - this, מתבצעת ע"י הצבעה למקטע הוירטואלי this, אליו ניגשים באמצעות pointer 0, ואז גישה לשדה הספציפי לפי האינדקס המתאים i, אי שלילי) באמצעות this i.
- בתוך פונקציית VM הגישה לתאי מערך מתבצעת ע"י הצבעה למקטע הוירטואלי that, אליו ניגשים באמצעות pointer 1, ואז גישה לתא הספציפי לפי האינדקס המתאים i, אי שלילי) באמצעות that i.

קריאות לפונקציות:

- לפני שקוראים לפונקציית VM, הפונקציה הקוראת (שהיא בעצמה פונקציית VM) חייבת להכניס למחסנית (push) את הארגומנטים הדרושים. במקרה של קריאה למתודה, ראשית נכניס את האובייקט (המשתנה שמצביע עליו, כלומר שמחזיק את כתובת הבסיס של מקטע הזיכרון שלו) למחסנית ואז את שאר הארגומנטים.
- כאשר מקמפלים מתודה לפונקציית VM, הקומפילר חייב להגדיר את כתובת הבסיס של this, שנמצאת ב-pointer-0, לכתובת של האובייקט הנוכחי, שנמצאת ב-0 argument.

החזרת ערך מפונקציה וקימפול שגרות void:

- קימפול פקודת return של שגרת Jack שאינה void מתבצעת באמצעות הכנסת הערך המוחזר למחסנית (push) וכתיבת פקודת return.
- מכיוון שכל פונקציית vm חייבת לדחוף ערך החזרה למחסנית ולהסתיים ב-return, עבור מתודות ופונקציות Jack שהן void, כלומר אין להן ערך החזרה, נבצע את הפעולה הבאה: נכניס למחסנית את הקבוע 0 ונכתוב את הפקודה return.
- לאחר קימפול פקודת do aaa, כאשר aaa היא פונקציית או מתודת void, הפונקציה הקוראת צריכה להוציא מהמחסנית (pop) את הערך הראשון שיש בה (קבוע, "ערך החזרה") ולהתעלם ממנו.

קבועים: הקבועים null ו- false מתורגמים לקבועים 0. הקבוע true מתורגם לקבוע 1-, שמתקבל במחסנית ע"י push constant 1 ואז neg.

- שימוש במערכת ההפעלה: מערכת ההפעלה הבסיסית (הסיפרייה הסטנדרטית) של Jack מממושים בקבצי vm, וחיבים להימצא בתוכנית לאחר קימפול התוכנית, בזמן ההרצה שלה. בדרך זו, כל הפונקציות ממערכת ההפעלה בהן משתמשים בתוכנית יזוהו. בנוסף, על הקומפילר להשתמש במספר פונקציות ממערכת הפעלה בעצמו:
- כפל וחילוק מתורגמים לשימוש בפונקציות Math.multiply ו-Math.divide.
- ב-Jack מחרוזות נכתבות פשוט כקבועים, למשל x="cccc". כאשר הקומפילר נתקל בסטרינג קבוע כזה, הוא צריך, בעזרת מערכת ההפעלה, לייצר אובייקט מסוג סטרינג באמצעות קריאה לבנאי x=String.new(length) (בתרגום ל-VM), ולהוסיף כל אות c בסטרינג לאובייקט באמצעות קריאה למתודה x.appendChar('c') (בתרגום ל-VM).
- בנאים צריכים להשתמש בפונקציה Memory.alloc(size) כדי להקצות זיכרון לאובייקט שהם מייצרים.

Implementation 11.2.2

התכנית JackCompiler מורכבת מחמישה מודולים (מחלקות), חלק מהם חדשים וחלק הם הרחבות של המודולים שנבנו בפרק 10: JackCompiler, JackTokenizer, CompilationEngine, SymbolTable, VMWriter. יש API מפורט בספר בפרק 11.

SymbolTable: מחזיק את טבלאות הסמלים עבור המחלקה הנוכחית: אחת כללית למחלקה ואחת לשגרה הנוכחית.

VMWriter: מחלקה סטטית, מסוגלת לכתוב פקודות VM לקובץ הפלט, בסינטקס VM.

JackTokenizer: המודול מהפרק הקודם.

CompilationEngine: מבצע את הקומפילציה עצמה. בפרק זה מרחיבים את כל אחת מהמתודות שנבנו בפרק הקודם (בהן רק זיהינו את האלמנט וכתבנו אותו בפורמט xml). כעת כל אחת מהמתודות האלו צריכה לבצע את כל תהליך הקימפול של האלמנט, כאשר היא מקבלת טוקנים באמצעות ה-JackTokenizer וכותבת לקובץ הפלט באמצעות ה-VMWriter, ומחזיקה אובייקט SymbolTable של המחלקה הנוכחית כדי לשמור את המשתנים והמאפיינים שלהם.

JackCompiler: המודול המרכזי, מריץ את התכנית: פותח את הקובץ החדש עם סיומת vm, מנחה את ה-CompilationEngine לפרסר את סדרת הטוקנים שיוצאת מה-JackTokenizer.

Operating System 12

Background 12.1

עד לשלב זה בנינו את החומרה של מחשב ה-Hack, ובנינו קומפיילר משפת תכנות high-level - Jack, כך שניתן לכתוב תוכנות ב-Jack, לקמפל אותן ולהריץ אותן על פלטפורמת ה-Hack. המרכיב היחיד החסר הוא מערכת ההפעלה - operating system, או OS. מערכת ההפעלה כוללת מספר שירותים שמאפשרים למתכנת ב-Jack להימנע מהתעסקות בפרטים הקטנים בפעולות פשוטות יחסית כמו כפל וחילוק, הצגת צורות על המסך, קבלת קלט מהמשתמש, ניהול זיכרון ועוד. מערכת ההפעלה שלנו מורכבת ממספר מחלקות Jack, כל אחת מספקת קבוצת שירותים דומים באמצעות שגרות Jack.

מערכות הפעלה באופן כללי כתובות בשפת high-level ומקומפלות לקוד בינארי, וכך גם מערכת ההפעלה שלנו: ניתן לכתוב את כולה ב-Jack. עם זאת, בניגוד לתוכניות Jack כלליות, מערכת ההפעלה חייבת להיות מודעת לפלטפורמה עליה היא רצה - מחשב ה-Hack, וזאת מכיוון שהמטרה שלה היא להתאים את הפרטים הקטנים (שנובעים מהתעסקות ישירה עם החומרה) של פעולות סטנדרטיות.

אחד הדברים החשובים מבחינת מערכת ההפעלה הוא יעילות: כל קוד שנכתוב ב-Jack ונרצה להריץ על מערכת ההפעלה יצטרך להשתמש במערכת ההפעלה, ולכן אם היא תהיה מסורבלת זה יזיק לכל תוכנית שנכתוב בהמשך. בהמשך הפרק נציג מספר אלגוריתמים בהם משתמשים בד"כ למימוש חלק מהשירותים שמערכת ההפעלה מציעה. אלגוריתמים אלו מספקים את דרישת היעילות וטוב לדעת אותם באופן כללי.

General 12.2

ראשית, לכל מחלקה תהיה פונקציה מאתחלת - `init`, שאחראית לאתחל משתנים סטטיים במחלקה. המימוש הספציפי תלוי במחלקה.

Math - Mathematical Operation 12.3

Math

This class enables various mathematical operations.

- Function `int abs(int x)`: Returns the absolute value of `x`.
- Function `int multiply(int x, int y)`: Returns the product of `x` and `y`.
- Function `int divide(int x, int y)`: Returns the integer part of the `x/y`.
- Function `int min(int x, int y)`: Returns the minimum of `x` and `y`.
- Function `int max(int x, int y)`: Returns the maximum of `x` and `y`.
- Function `int sqrt(int x)`: Returns the integer part of the square root of `x`.

חיבור ופעולות פשוטות אחרות בד"כ ממומשות ברמת החומרה (ב-ALU). ניתן גם לממש כפל, חילוק ופעולות דומות (מעט יותר מורכבות) בחומרה, אך ניתן גם לממש אותן בתוכנה, באמצעות מערכת ההפעלה.

Multiplication 12.3.1

מתבצע באמצעות "כפל ארוך":

Long multiplication

$$\begin{array}{r}
 x \quad \quad 1 \ 0 \ 1 \ 1 = 1 \ 1 \\
 y \quad * \quad 1 \ 0 \ 1 = \underline{5} \quad j\text{-th bit of } y \\
 \quad \quad 1 \ 0 \ 1 \ 1 \\
 \quad \quad 0 \ 0 \ 0 \ 0 \\
 \quad 1 \ 0 \ 1 \ 1 \\
 x \cdot y \quad 1 \ 1 \ 0 \ 1 \ 1 \ 1 = 5 \ 5
 \end{array}$$

multiply(x, y):

// Where $x, y \geq 0$

$sum = 0$

$shiftedX = x$

for $j = 0 \dots (n-1)$ do

if $(j\text{-th bit of } y) = 1$ then

$sum = sum + shiftedX$

$shiftedX = shiftedX * 2$

האלגוריתם פועל ב- $O(n)$ פעולות חיבור, כאשר n הוא מספר הביטים של x ו- y .

מימוש:

- הבדיקה $(j\text{-th bit of } y = 1)$ דורשת גישה לביטים ספציפיים של y . לשם כך נחזיק במחלקה Math מערך סטטי בשם `twoToThe`, כאשר ב- `init` שלה המערך יאותחל למערך באורך 16 ביטים, ובכל תא i נשים את 2^i . לאחר מכן, הבדיקה הנ"ל מתורגמת פשוט ל- $(twoToThe[j] \& y)$.
 - $shiftedX * 2$ ניתן לחשב ע"י `left shift` של `shiftedX` או ע"י חיבור `shiftedX` לעצמו.

Division 12.3.2

מתבצע באמצעות "חיבור ארוך":

divide(x, y):

// Integer part of x / y , where $x \geq 0$ and $y > 0$

if $y > x$ return 0

$q = divide(x, 2 * y)$

if $(x - 2 * q * y) < y$

return $2 * q$

else

return $2 * q + 1$

האלגוריתם פועל ב- $O(n)$ פעולות חיבור, כאשר n הוא מספר הביטים של x ו- y .

מימוש:

- הפונקציה עצמה בודקת מה הסימנים של x ו- y ולפי זה קוראת לפונקציית עזר עם הסימנים המתאימים.
 - פונקציית העזר מממשת בדיוק את הפסאודו-קוד, כאשר כפל ב-2 מתורגם ל- `left shift` של q או ע"י חיבור q לעצמו.
 - בבדיקה `return 0` : if $(y > x)$ נוסף גם בדיקה $y < 0$, כדי להימנע מ-overflow.

מתבצע באמצעות חיפוש בינארי של y עבור $y^2 \leq x < (y+1)^2$ כזה יהיה החלק השלם של $\text{sqrt}(x)$, עבור $0 \leq x \leq 2^n$.

```
sqrt(x):
  // Compute the integer part of  $y = \sqrt{x}$ . Strategy:
  // Find an integer  $y$  such that  $y^2 \leq x < (y+1)^2$  (for  $0 \leq x < 2^n$ )
  // By performing a binary search in the range  $0 \dots 2^{n/2} - 1$ .
   $y = 0$ 
  for  $j = n/2 - 1 \dots 0$  do
    if  $(y + 2^j)^2 \leq x$  then  $y = y + 2^j$ 
  return  $y$ 
```

האלגוריתם פועל ב- $O(n)$ פעולות חיבור, כאשר n הוא מספר הביטים של x .

מימוש:

- נאתחל $y = 0, j = 7$. כל עוד $j \geq 0$: נחשב $\text{num} = y + 2^j$ ו- $\text{temp} = \text{num} \cdot \text{num}$.
 - אם $0 < \text{temp} \leq x$, נציב $y = \text{num}$ (הבדיקה של חיוביות נועדה למנוע overflow).
 - נעדכן $j = j - 1$.

Memory

This class allows direct access to the main memory.

- Function `int peek(int address)`: Returns the value of the main memory at this address.
- Function `void poke(int address, int value)`: Sets the value of the main memory at this address to the given value.
- Function `Array alloc(int size)`: Allocates the specified space on the heap and returns a reference to it.
- Function `void deAlloc(Array o)`: De-allocates the given object and frees its memory space.

המחלקה הזו אחראית על ניהול הזיכרון בתכנית. הקצאת זיכרון למערכים ואובייקטים בערימה (מכתובת 2048 עד כתובת 16383) מתבצעת במהלך זמן הריצה (Dynamic Memory Allocation). זה מתנהל במחלקה זו. יש שני אלגוריתמים להקצאת זיכרון: בסיסי ומשופר.

Basic Memory Allocation Algorithm 12.4.1

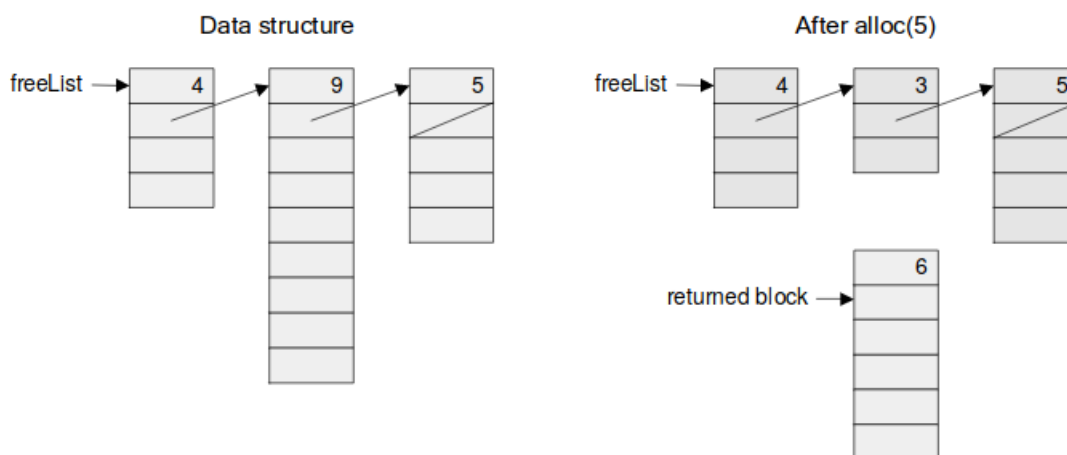
המחלקה מחזיקה מערך (מצביע) סטטי `free` שמצביע לתחילת החלק הפנוי בערימה, כלומר החלק שעדיין לא הוקצה.

```
Initialization: free = heapBase

// Allocate a memory block of size words.
alloc(size):
    pointer = free
    free = free + size
    return pointer

// De-allocate the memory space of a given object.
deAlloc(object):
    do nothing
```

Improved Memory Allocation Algorithm 12.4.2



Initialization:

```

freeList = heapBase
freeList.length = heapLength
freeList.next = null

```

// Allocate a memory space of *size* words.

alloc(size):

```

Search freeList using best-fit or first-fit heuristics
to obtain a segment with segment.length > size
If no such segment is found, return failure
(or attempt defragmentation)
block = needed part of the found segment
(or all of it, if the segment remainder is too small)
Update freeList to reflect the allocation
block[-1] = size + 1 // Remember block size, for de-allocation
Return block

```

// Deallocate a decommissioned *object*.

deAlloc(object):

```

segment = object - 1
segment.length = object[-1]
Insert segment into the freeList

```

המחלקה מחזיקה מערך (מצביע) סטטי freeList לרשימה מקושרת של מקטעי זיכרון. כל מקטע מכיל שני "שדות" שמאוחסנים בתא הראשון והשני של המקטע: אורך המקטע - length, ומצביע למקטע הבא ברשימה המקושרת - next. כלומר, segment[0]=length, segment[1]=next.

ה-freeList מאותחלת עם length=14335 (אורך הערימה) ו-next=null. כאשר אנחנו רוצים להקצות זיכרון באורך size, האלגוריתם מחפש ברשימה המקושרת מקטע בגודל מתאים. יש שני דרכים לחיפוש הזה:

- first-fit: מחפשים את המקטע הראשון עם segment.length > size.
 - best-fit: מחפשים את המקטע עם האורך המינימלי כך שמתקיים segment.length > size.
 כעת, בהנחה שמצאנו מקטע כזה, נסמן את כתובת הבסיס שלו ב-segment. בכל מעבר נרצה לשמור את המקטע הקודם, נסמן את כתובת הבסיס שלו ב-prev. נעדכן את ה-next של prev:

```
prev.next = segment.next
```

ונחזיר את הבלוק:

```
block = segment + 1
```

כאשר התא block[-1] שומר את גודל הבלוק שהקצאנו לצורך שחרור הזיכרון (בהמשך). כדי להימנע מסיוס הזיכרון, אם גודל המקטע שמצאנו גדול בהרבה מה-size (נניח, size+4 > segment.length), נרצה להוציא רק חלק מהבלוק. לכן, במקרה הזה, נחזיר

```
block = segment + length - size
```

ונעדכן את אורך המקטע:

```
segment.length = length - size - 1
```

את prev.next נשאיר כמו שהיה (מצביע על segment).

כעת, כאשר נתבקש לשחרר חלק בזיכרון - deAlloc(obj), האלגוריתם יוסיף את הבלוק שהוקצה ל-obj ל-freeList (בהתחלה):

```
segment = obj - 1
segment.next = freeList
freeList = segment
```

peek, poke 12.4.3

הפונקציות האלו נועדו לגעת בכתובות ספציפיות ב-RAM מבלי להתעסק עם מצביעים.

```
// To create a Jack-level "proxy" of the RAM:
var Array memory;
let memory = 0;
// From this point on we can use code like:
let x = memory[j] // Where j is any RAM address
let memory[j] = y // Where j is any RAM address
```

Array 12.5

Array

This class enables the construction and disposal of arrays.

- Function **Array.new**(int size): Constructs a new array of the given size.
- Method **void dispose()**: Disposes this array.

המימוש די ברור, אך צריך לשים לב לדבר אחד: השגרה `Array.new` אינה בנאי אלא פונקציה. לכן, הזיכרון המוקצה למערך חדש צריך להיות מוקצה מפורשות ע"י `Memory.alloc(size)` ומשוחרר ישירות ע"י `Memory.deAlloc(this)`.

String

This class implements the `String` data type and various string-related operations.

- Constructor `String new(int maxLength)`: Constructs a new empty string (of length zero) that can contain at most `maxLength` characters.
- Method `void dispose()`: Disposes this string.
- Method `int length()`: Returns the length of this string.
- Method `char charAt(int j)`: Returns the character at location `j` of this string.
- Method `void setCharAt(int j, char c)`: Sets the `j`'th element of this string to `c`.
- Method `String appendChar(char c)`: Appends `c` to this string and returns this string.
- Method `void eraseLastChar()`: Erases the last character from this string.
- Method `int intValue()`: Returns the integer value of this string (or at least of the prefix until a non numeric character is found).
- Method `void setInt(int j)`: Sets this string to hold a representation of `j`.
- Function `char backSpace()`: Returns the backspace character.
- Function `char doubleQuote()`: Returns the double quote (") character.
- Function `char newLine()`: Returns the newline character.

מחרוזות ממומשות כמערכים. כאשר המחרוזות נוצרות, הבנאי מאתחל מערך באורך מקסימלי למחרוזות. הגודל של המערך לא משתנה, למרות שניתן להוסיף תווים אליו (באמצעות הוספת קוד ה-ASCII של התו לתא המתאים במערך) ולהוריד תווים ממנו. כלומר ישנו שדה אחד ששומר את האורך המקסימלי של המחרוזות, ושדה שני ששומר את האורך הנוכחי שלה (ואז הורדת תו מסוף המחרוזת שקול להורדת האורך הנוכחי ב-1). הפעולות היחידות שמעט מורכבות כאן הן מעבר ממחרוזת ל-`int` ולהיפך (הפונקציות `setInt` ו-`intValue`).

String Representation of Numbers 12.6.1

המחשב שומר תווים במחשב בקוד ה-ASCII שלהם. למשל, עבור הספרות:

Character:	'0'	'1'	'2'	'3'	'4'	'5'	'6'	'7'	'8'	'9'
ASCII code:	48	49	50	51	52	53	54	55	56	57

כדי לעבור בין הייצוג המספרי של ספרה לייצוג בקוד ASCII, יש להוסיף למספר 48 - הקוד של '0'. כדי לעבור בין מספרים (עם מספר ספרות) למחרוזות, נשתמש באלגוריתמים הבאים:

```
// Convert a non-negative number to a string
int2String(n):
    lastDigit = n % 10
    c = character representing lastDigit
    if n < 10
        return c (as a string)
    else
        return int2String(n / 10).append(c)
```

```
// Convert a string to a non-negative number
string2Int(s):
    v = 0
    for i = 1 ... length of s do
        d = integer value of the digit s[i]
        v = v * 10 + d
    return v
// (Assuming that s[1] is the most
// significant digit character of s.)
```

כאשר במשתנים `c` (משמאל) ו-`d` (מימין) דרושה המרה בין מחרוזת ל-`int` או להיפך.

Screen

This class allows drawing graphics on the screen. Column indices start at 0 and are left-to-right. Row indices start at 0 and are top-to-bottom. The screen size is hardware-dependant (in the Hack platform: 256 rows by 512 columns).

- function void `init()`: for internal use only.
- function void `clearScreen()`: erases the entire screen.
- function void `setColor(boolean b)`: sets a color (white=false, black=true) to be used for all further `drawXXX` commands.
- function void `drawPixel(int x, int y)`: draws the (x,y) pixel.
- function void `drawLine(int x1, int y1, int x2, int y2)`: draws a line from pixel (x1,y1) to pixel (x2,y2).
- function void `drawRectangle(int x1, int y1, int x2, int y2)`: draws a filled rectangle whose top left corner is (x1, y1) and bottom right corner is (x2,y2).
- function void `drawCircle(int x, int y, int r)`: draws a filled circle of radius $r \leq 181$ around (x,y).

מטרת המחלקה הזו היא לחסוך מהמתכנת פעולות גישה מורכבות לפיקסלים במפת הזיכרון של המסך כאשר הוא רוצה לצייר דברים. במקום זאת, מממשים במחלקה הזו מספר פונקציות שימושיות לציור עצמים על המסך. כתובת הבסיס של מפת הזיכרון של המסך בפלטפורמת ה-Hack היא 48361. גישה למפת הזיכרון של המסך יכולה להתבצע ע"י הגדרת מערך סטטי במחלקה, `screen`, ואז הצבת כתובת הבסיס בו. נשמור משתנה סטטי נוסף `color - (int)`, שמטרתו לסמן האם הצבע בו אנחנו רוצים לצבוע כעת הוא לבן (0) או שחור (1).

Pixel Drawing 12.7.1

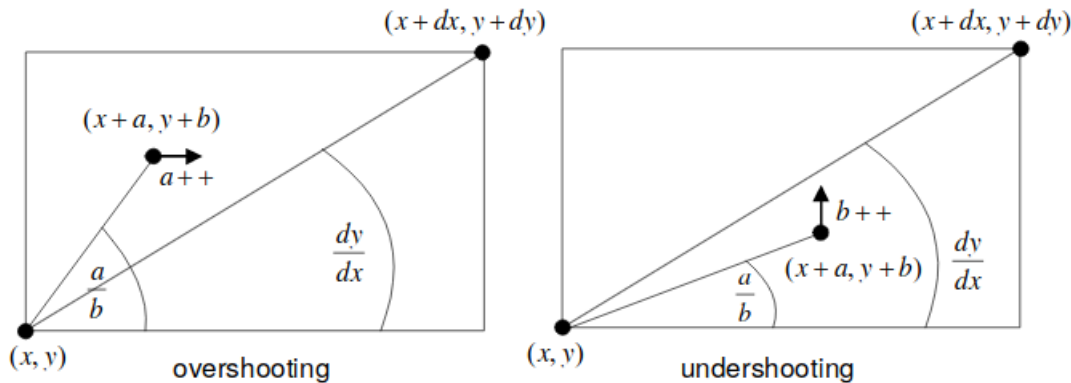
`drawPixel(x, y):`

```
// Hardware-specific.
// Assuming a memory mapped screen:
Write a predetermined value in the RAM
location corresponding to screen location (x, y).
```

פיקסל על המסך מיוצג ע"י שתי קואורדינטות: $(column, row)$. הפיקסל (0,0) מייצג את התא העליון השמאלי במסך, כלומר ציר ה-x ממוספר משמאל לימין (כרגיל) וציר ה-y ממוספר מלמעלה למטה (הפוך). יש 256 שורות במסך, בכל אחת 512 פיקסלים (32 רגיסטרים בשורה, 16 ביטים ברגיסטר). צביעת פיקסל מסויים על המסך דורשת גישה לתא שמכיל את הפיקסל במפת הזיכרון של המסך, ופעולה לוגית שתדליק או תכבה את הפיקסל בהתאם לדרישה.

מימוש:

- הגישה לתא המתאים מתבצעת באמצעות: $addr = (32 \cdot y) + (\frac{x}{16})$ (הכתובת ביחס למפת הזיכרון של המסך).
 - נחשב את מיקום הפיקסל בתוך התא ונייצר את המספר שהייצוג הבינארי שלו רק את המיקום הזה דלוק:
 $value = \text{Math.two_to_the}(x \& 15)$. נותן את מספר הביט עבור הפיקסל, והפונקציה `Math.two\_to\_the(i)` מח-
 זירה את 2^i .
 - עדכון הביט: תלוי בצבע. אם הצבע הנדרש לבן (0), הביטוי `screen[addr] = screen[addr] & (~value)` יתן לנו 0 בביט המתאים. אחרת, הביטוי `screen[addr] = screen[addr] | (value)` יתן לנו 1 (שחור) בביט המתאים. כל שאר הביטים בתא לא משתנים.



```

drawLine(x, y, x+dx, y+dy):
  // Assuming dx, dy > 0
  initialize (a,b) = (0,0)
  while a ≤ dx and b ≤ dy do
    drawPixel(x+a, y+b)
    if a/dx < b/dy then a++ else b++

```

```

// To test whether a/dx < b/dy, maintain a variable adyMinusbdx,
// and test if it becomes negative.
Initialization:      set adyMinusbdx = 0
When a++ is performed: set adyMinusbdx = adyMinusbdx + dy
When b++ is performed: set adyMinusbdx = adyMinusbdx - dx

```

מימוש:

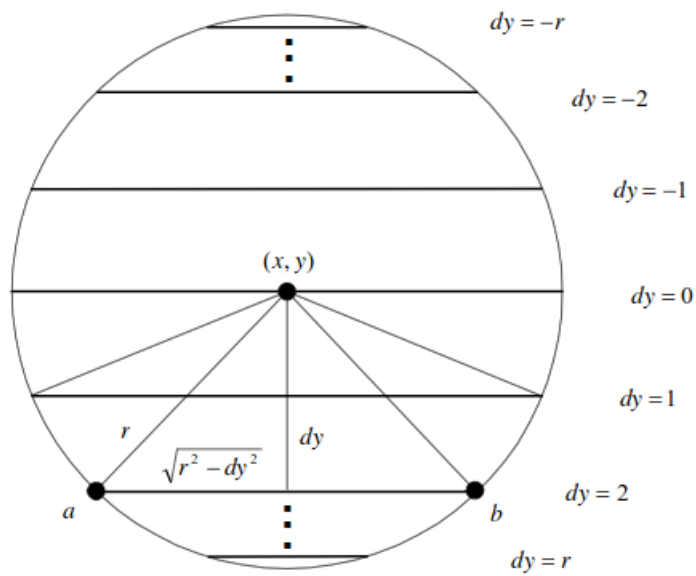
- אם $x_2 < x_1$ נחליף בין הנקודות.
- אם $dx = 0$ או $dy = 0$ נרצה לצייר קווים ישרים, ונעשה זאת בפונקציות עזר (חוסך זמן ריצה) - בסוף.
- אם $y_1 < y_2$ נעבוד לפי הפסאודו קוד. אחרת נשנה אותו קצת: את dy נחשב כ- $y_2 - y_1$ ובכל פעם שכתוב $b++$ נשנה ל- $b--$.

מימוש קו אופקי (Horizontal): קלט: x_1, x_2, y

- ראשית נחשב את הכתובות של התאים של הגבולות במפת הזיכרון של המסך.
- נעדכן את המילה הראשונה משמאל באותו אופן כמו שמעדכנים פיקסל: מחשבים את הערך המקביל ליצוג הבינארי בו כל הביטים מימין לפיקסל הדרוש דלוקים, ואז מבצעים Or או AND כתלות בצבע הנדרש.
- נעדכן את כל המילים מימין: פשוט נשים 1- בתאים.
- נעדכן את המילה האחרונה באותו אופן שעידכנו את המילה הראשונה, ביחס לפיקסל הימני.

מימוש קו אנכי (Vertical): קלט: x, y_1, y_2

- ראשית נחשב את הכתובות של התאים של הגבולות במפת הזיכרון של המסך. נחשב את מספר הביט המתאים בתא בהתאם ל- x .
- נעבור שורה שורה (באמצעות הוספת 32 לכתובת הנוכחית בכל שלב) ונדליק את הביט המתאים כמו שמעדכנים פיקסל (הערך שבו משתמשים והפעולה שמבצעים עליו ועל הערך בכתובת נכונים לכל השורות, ונקבעים לפי x ולפי הצבע הנוכחי).



$$\text{point } a = (x - \sqrt{r^2 - dy^2}, y + dy)$$

$$\text{point } b = (x + \sqrt{r^2 - dy^2}, y + dy)$$

drawCircle(x, y, r):

for each $dy \in -r \dots r$ do

drawLine from $(x - \sqrt{r^2 - dy^2}, y + dy)$ to $(x + \sqrt{r^2 - dy^2}, y + dy)$

מימוש:

- משתמשים ב- drawHorizontal.

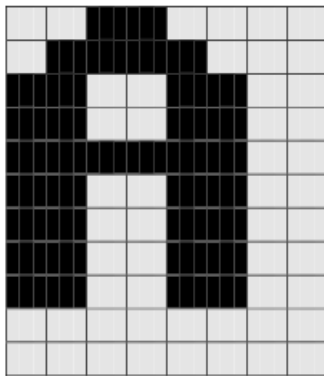
- מגבילים את הרדיוס ל181 כדי למנוע overflow.

Output

This class allows writing text on the screen.

- Function void **moveCursor**(int i, int j): Moves the cursor to the j'th column of the i'th row, and erases the character located there.
- Function void **printChar**(char c): Prints c at the cursor location and advances the cursor one column forward.
- Function void **printString**(String s): Prints s starting at the cursor location, and advances the cursor appropriately.
- Function void **printInt**(int i): Prints i starting at the cursor location, and advances the cursor appropriately.
- Function void **println**(): Advances the cursor to the beginning of the next line.
- Function void **backSpace**(): Moves the cursor one column back.

אחראי על הדפסת תווים למסך. המסך (256 שורות על 512 עמודות) מחולק לרשת של 23 שורות טקסט, בכל אחת 64 תווים (כל תו הוא 11 שורות ו-8 פיקסלים בשורה, ונשארות 3 שורות מיותרות במסך שלא נכנסות לרשת). לכל תו מייצרים bitmap - דרך קבועה להצגה שלו על המסך. לדוגמא, עבור האות 'A':



בנוסף, יש צורך בסמן (cursor) שמסמן איפה ברשת אנחנו נמצאים כרגע, כלומר איפה תיכתב האות הבאה. ניתן לייצג אותו על ידי אינדקס שורה - line (בין 0 ל-22) ועמודה - column (בין 0 ל-63). כתיבת אות במיקום הנוכחי מתקבלת ע"י כתיבת ה-bitmap של האות בפיקסלים שבשורות $line \cdot 11, \dots, line \cdot 11 + 10$ ועמודות $column \cdot 8, \dots, column \cdot 8 + 7$ ועדכון ה-cursor למשבצת הבאה.

כתיבת שורה חדשה תעדכן את ה-cursor ל- $line = line + 1$ ול- $column = 0$. אם הוא הגיע לסוף המסך, נאפס את הערכים שלו ל-(0,0).
מימוש:

- התווים אצלנו ממומשים באמצעות bitmaps לכל תו, כלומר מחזיקים מערך של מערכים, בכל תא במערך bitmap של תו מסויים, כאשר bitmap כוללת 11 ערכים, $map[i]$ הוא מספר שמתאים לשורה ה-i במשבצת.
- נחזיק (כמו ב-Screen משתנה סטטי $screen = 16284$ שמצביעה לתחילת מפת הזיכרון של המסך. הכתובת של הרגיסטר העליון עבור שורת הטקסט i והעמודה j מתקבלת ע"י $addr = 11 \cdot 32 \cdot i + \frac{j}{2}$.
- נשים לב כי משבצת ברשת כוללת 8 עמודות ו-11 שורות של מפת הזיכרון של המסך. כלומר, כל אות תופסת חצי (ימני או שמאלי) מ-11 הרגיסטרים שמתאימים לפיקסלים. יש לכך שתי השלכות: על עדכון המיקום של ה-cursor ועל הדפסת תו.

- כדי לטפל בהזזת ה-cursor נשמור מערך סטטי mask של שני תאים: תא 0 יכיל את 255~ ותא 1 יכיל את 255. כעת, כאשר נרצה להזיז את ה-cursor למשבצת הבאה, נבדוק האם מספר העמודה איזוגי (כלומר אנחנו בצד ימין) ע"י $right = col \& 1$ (אם זה שווה 1 אז אנחנו בימין), ולפי הערך הזה נעבור על כל שורה במשבצת ונאפס את האיזור המתאים באמצעות $screen[addr] = screen[addr] \& mask[right]$.

- כדי לכתוב במשבצת בה ה-cursor נמצא, אם מספר העמודה איזוגי זה אומר שאנחנו בצד שמאל של הרגיסטרים, אחרת אנחנו בצד ימין. נבדוק את הזוגיות ע"י $right = col \& 1$ (אם זה שווה 1 אז אנחנו בימין). כעת, לכל אחת מהשורות $0 \leq i \leq 10$ במשבצת נקבל את $num = map[i]$ עבור האות הרצויה. אם אנחנו בימין, נכפיל את הערך ב-256 ונציב ב-num כדי להעביר את הביטים הדולקים לימין ונעדכן $screen[addr] = screen[addr] + num$. אם אנחנו בשמאל בעדכן $screen[addr] = num$.

Keyboard - Input Handling 12.9

Keyboard

This class allows reading inputs from the keyboard.

- Function `char keyPressed()`: Returns the character of the currently pressed key on the keyboard; if no key is currently pressed, returns 0.
- Function `char readChar()`: Waits until a key is pressed on the keyboard and released, and then echoes the key to the screen and returns the character of the pressed key.
- Function `String readLine(String message)`: Prints the message on the screen, reads the next line (until a newline character) from the keyboard, echoes the line to the screen, and returns its value. This method handles user backspaces.
- Function `int readInt(String message)`: Prints the message on the screen, reads the next line (until a newline character) from the keyboard, echoes the line to the screen, and returns the integer until the first non numeric character in the line. This method handles user backspaces.

מפת הזיכרון של המקלדת נמצאת בכתובת 67542. נשמור מערך סטטי keyboard שמחזיר את הכתובת הזו וכך נוכל לגשת לתא הזה פשוט ע"י `keyboard[0]`.

Detecting Keyboard Input 12.9.1

```
keyPressed():
// Depends on the specifics of the keyboard interface
if a key is presently pressed on the keyboard
    return the ASCII value of the key
else
    return 0
```

מימוש: פשוט מחזירים את `keyboard[0]`.

```

readChar():
    // Read and echo a single character
    display the cursor
    while no key is pressed on the keyboard
        do nothing // wait till a key is pressed
    c = code of currently pressed key
    while a key is pressed
        do nothing // wait for the user to let go
    print c at the current cursor location
    move the cursor one position to the right
    return c

```

מימוש:

- כל עוד `keyboard[0]=0`, אל תעשה כלום (חכה שהמשתמש ילחץ על מקש).
- אם `keyboard[0]!=0`, תעד את התו שנכתב במשתנה לוקאלי `c`.
- כל עוד `keyboard[0]!=0`, אל תעשה כלום (חכה שהמשתמש יעזוב את המקש).
- אם `keyboard[0]=0`, החזר את `c` וסיים.

Reading a String 12.9.3

```

readLine():
    // Read and echo a "line" (until newline)
    s = empty string
    repeat
        c = readChar()
        if c = newline character
            print newline
            return s
        else if c = backspace character
            remove last character from s
            move the cursor 1 position back
        else
            s = s.append(c)

```

מימוש: (מקס' אורך הקלט: 64 תווים).

- אתחל מחרוזת באורך 64 תווים.
- כל עוד לא נקרא התו `c=128` (newline) המשך לקרוא באמצעות `readchar` והוסף את התו למחרוזת. אם נקרא התו `c=129` (backspace) מחק את התו האחרון מהמחרוזת ומהמסך.
- כאשר נקרא התו `newline`, הדפס שורה חדשה והחזר את המחרוזת.

Sys

This class supports some execution-related services.

- Function void **halt**(t): Halts the program execution.
- Function void **error**(int errorCode): Prints the error code on the screen and halts.
- Function void **wait**(int duration): Waits approximately *duration* milliseconds and returns.

המחלקה הזו נועדה לאתחול מערכת ההפעלה בעת הפעלת המחשב ולהפעלת התוכנית הנוכחית. נזכור כי בשלב התרגום מ-VM לאסמבלי (פרק 8), הוספנו קוד אתחול בתחילת סדרת ההוראות שמפעיל את הפונקציה Sys.init. - Sys.init: הפונקציה הזו מאתחלת את כל המחלקות של מערכת ההפעלה (ע"י קריאה ל-init, יש לשים לב לסדר כדי שלא נאתחל מחלקה שמשתמשת בקוד שלה במחלקה שעוד לא אתחלנו). לאחר מכן היא קוראת לפונקציה Main.main, שחייבת להיות בתוכנית עצמה ומתחילה את הרצת התוכנה. בסוף היא נכנסת ללולאה אינסופית. סדר הפקודות: Memory, Math, Screen, Output, Keyboard, Main.main, Sys.halt

- Sys.halt: מכניסה את התוכנית ללולאה אינסופית.

- Sys.wait(t): משהה את פעולת התוכנית למשך t מילישניות. כדי לממש את זה מספיק להריץ לולאה למשך t איטרציות, כאשר בכל איטרציה יש לולאה מקוננת שרצה בערך 100 איטרציות.